

Step 0 — Dataset Preparation

Input	nuScenes v1.0-mini dataset — set DATA_ROOT in config.py
Outputs	output/step_0/sample_XXXX/sensor_meta.json — all 12 sensor channels per sample
	output/step_0/sample_XXXX/ego_pose.json — ego vehicle pose
	output/step_0/sample_XXXX/timestamp.txt — capture timestamp
	output/step_0/samples_index.json — master index used by all downstream steps
	output/step_0/samples_report.csv — summary CSV for inspection
Used by	Steps 1.1, 1.2, 1.3, 2.1, 2.2, 2.3, 4.4, 5

Before running

- 1. Make sure config.py is in the **same folder as this notebook**
- 2. Open config.py and set DATA_ROOT to your nuScenes dataset path
- 3. Run cells top to bottom

```

In [1]: # -----
# CELL 1 – Verify config.py exists and DATA_ROOT is set correctly
# config.py should already be in the same folder as this notebook.
# If it is missing, this cell tells you exactly what to do.
# -----

import os
from pathlib import Path

if not Path("config.py").exists():
    raise FileNotFoundError(
        "config.py not found in this folder.\n"
        "Please download config.py from the repo root and place it "
        "in the same folder as this notebook, then set DATA_ROOT inside it."
    )

# Import and validate
from config import DATA_ROOT, NUSCENES_VERSION, STEP0_DIR

if not DATA_ROOT.exists():
    raise FileNotFoundError(
        f"DATA_ROOT does not exist: {DATA_ROOT}\n"
        "Open config.py and set DATA_ROOT to your nuScenes dataset folder."
    )

print(f"✅ config.py found")
print(f"✅ DATA_ROOT : {DATA_ROOT}")
print(f"✅ STEP0_DIR  : {STEP0_DIR}")
print(f"✅ Version    : {NUSCENES_VERSION}")

```

```

config.py loaded. PROJECT_ROOT = F:\Sensor fusion Research
DATA_ROOT   = F:\Sensor fusion Research\DATA SET\archive
OUTPUT_ROOT = F:\Sensor fusion Research\output
✅ config.py found
✅ DATA_ROOT : F:\Sensor fusion Research\DATA SET\archive
✅ STEP0_DIR  : F:\Sensor fusion Research\output\step_0
✅ Version    : v1.0-mini

```

```

In [2]: # -----
# CELL 2 – Main pipeline
# Loads nuScenes, extracts metadata for all 12 sensors per sample,
# saves sensor_meta.json / ego_pose.json / timestamp.txt per sample,
# and writes the master samples_index.json used by all downstream steps.
# -----

import json
from pathlib import Path
from itertools import groupby
from nuscnescenes import NuScenes
from config import DATA_ROOT, NUSCENES_VERSION, STEP0_DIR

output_dir = STEP0_DIR
output_dir.mkdir(parents=True, exist_ok=True)

EXPECTED_CHANNELS = 12

# — Initialise nuScenes -----
nusc = NuScenes(version=NUSCENES_VERSION, dataroot=str(DATA_ROOT), verbose=True)
print("Number of samples:", len(nusc.sample))
print("DATA_ROOT:", DATA_ROOT)
samples_index = {}

# — Iterate every sample -----
for idx, sample in enumerate(nusc.sample):
    sample_token = sample['token']
    sample_id = f'sample_{idx:04d}'
    sample_folder = output_dir / sample_id
    sample_folder.mkdir(exist_ok=True)

    scene_token = sample['scene_token']
    scene = nusc.get('scene', scene_token)
    scene_name = scene['name']

    sensor_meta = {
        "sample_token": sample_token,
        "scene_token": scene_token,
        "scene_name": scene_name,
        "sensor_channels": {}
    }

    # Extract metadata for all 12 sensors (1 LiDAR + 5 Radar + 6 Camera)
    for sensor_channel, data_token in sample['data'].items():
        sd = nusc.get('sample_data', data_token)
        cs = nusc.get('calibrated_sensor', sd['calibrated_sensor_token'])
        ep = nusc.get('ego_pose', sd['ego_pose_token'])

        sensor_meta["sensor_channels"][sensor_channel] = {
            "filename": sd['filename'],
            "timestamp": sd['timestamp'],
            "modality": sd['sensor_modality'],
            "calibrated_sensor_token": sd['calibrated_sensor_token'],
            "ego_pose_token": sd['ego_pose_token'],
            "sensor_to_ego_translation": cs['translation'],
            "sensor_to_ego_rotation": cs['rotation'],

```

```

        "ego_pose": {
            "translation": ep['translation'],
            "rotation"    : ep['rotation']
        }
    }

    # Authoritative sample-level timestamp (microseconds) - NOT any one sensor's own timestamp
    timestamp = sample['timestamp']

    if len(sensor_meta["sensor_channels"]) != EXPECTED_CHANNELS:
        print(f"WARNING {sample_id}: only {len(sensor_meta['sensor_channels'])}/12 channels present - flagging, not skipping")

    # Save timestamp.txt
    with open(sample_folder / 'timestamp.txt', 'w') as f:
        f.write(str(timestamp))

    # Save ego_pose.json - prefer LIDAR_TOP, fallback to first sensor
    ego_pose_source = "LIDAR_TOP" if "LIDAR_TOP" in sensor_meta["sensor_channels"] else list(sensor_meta["sensor_channels"].keys())[0]
    ego_pose = {
        "_source_channel": ego_pose_source,
        **sensor_meta["sensor_channels"][ego_pose_source]["ego_pose"]
    }
    with open(sample_folder / 'ego_pose.json', 'w') as f:
        json.dump(ego_pose, f, indent=2)

    # Save sensor_meta.json
    with open(sample_folder / 'sensor_meta.json', 'w') as f:
        json.dump(sensor_meta, f, indent=2)

    # Master index - store RELATIVE folder name only (NOT absolute path)
    # Downstream notebooks reconstruct full path as: STEP0_DIR / info["folder"]
    samples_index[sample_id] = {
        "sample_token" : sample_token,
        "scene_token"  : scene_token,
        "scene_name"   : scene_name,
        "timestamp_us" : timestamp,          # renamed to make units explicit
        "prev_token"   : sample['prev'],    # "" if first sample in scene
        "next_token"   : sample['next'],    # "" if last sample in scene
        "folder"       : sample_id,
        "sensors"      : list(sensor_meta["sensor_channels"].keys())
    }

    # — Save master index —————
    index_path = output_dir / 'samples_index.json'
    with open(index_path, 'w') as f:
        json.dump(samples_index, f, indent=2)

    # — Validate —————
    assert len(samples_index) == len(nusc.sample), \
        f"Expected {len(nusc.sample)} samples, got {len(samples_index)}"

    print(f"\nStep 0 complete: {len(samples_index)} samples organised.")
    print(f"Index saved to: {index_path}")

```

```
# — Scene boundary summary —————
scene_order = [v["scene_name"] for v in samples_index.values()]
boundaries = [i for i in range(1, len(scene_order)) if scene_order[i] != scene_order[i-1]]
print(f"Scene boundaries at sample indices: {boundaries}")
```

=====

Loading NuScenes tables for version v1.0-mini...

23 category,

8 attribute,

4 visibility,

911 instance,

12 sensor,

120 calibrated_sensor,

31206 ego_pose,

8 log,

10 scene,

404 sample,

31206 sample_data,

18538 sample_annotation,

4 map,

Done loading in 1.030 seconds.

=====

Reverse indexing ...

Done reverse indexing in 0.1 seconds.

=====

Number of samples: 404

DATA_ROOT: F:\Sensor fusion Research\DATA SET\archive

Step 0 complete: 404 samples organised.

Index saved to : F:\Sensor fusion Research\output\step_0\samples_index.json

Scene boundaries at sample indices: [39, 79, 120, 161, 202, 242, 283, 324, 364]

```
In [3]: # -----
# CELL 3 – Load and preview the master index
# -----

import json
from config import STEP0_DIR

with open(STEP0_DIR / 'samples_index.json') as f:
    index = json.load(f)

print(f"✅ Loaded {len(index)} samples from index.")
print("\nSample preview (sample_0003):")
print(json.dumps(index['sample_0003'], indent=2))
```

✅ Loaded 404 samples from index.

Sample preview (sample_0003):

```
{
  "sample_token": "e0845f5322254dafadbbbed75aaa07969",
  "scene_token": "cc8c0bf57f984915a77078b10eb33198",
  "scene_name": "scene-0061",
  "timestamp_us": 1532402929197353,
  "prev_token": "356d81f38dd9473ba590f39e266f54e5",
  "next_token": "c923fe08b2ff4e27975d2bf30934383b",
  "folder": "sample_0003",
  "sensors": [
    "RADAR_FRONT",
    "RADAR_FRONT_LEFT",
    "RADAR_FRONT_RIGHT",
    "RADAR_BACK_LEFT",
    "RADAR_BACK_RIGHT",
    "LIDAR_TOP",
    "CAM_FRONT",
    "CAM_FRONT_RIGHT",
    "CAM_BACK_RIGHT",
    "CAM_BACK",
    "CAM_BACK_LEFT",
    "CAM_FRONT_LEFT"
  ]
}
```

```

In [4]: # -----
# CELL 4 – Sensor access verification
# Checks LiDAR (1), Radar (5), Camera (6) channels for a sample
# -----

import json
from config import STEP0_DIR

sample_id      = "sample_0003"    # ← change to inspect any sample
sample_folder = STEP0_DIR / sample_id

with open(sample_folder / "sensor_meta.json") as f:
    sensor_meta = json.load(f)

ch = sensor_meta["sensor_channels"]

# LiDAR
if "LIDAR_TOP" in ch:
    print(f"✅ LiDAR → {ch['LIDAR_TOP']['filename']}")
else:
    print(f"❌ LiDAR not found")

# Radar (expect 5)
radar_ch = {k: v for k, v in ch.items() if "RADAR" in k}
print(f"\n✅ Radar → {len(radar_ch)}/5 channels:")
for k, v in radar_ch.items():
    print(f"    {k:20s} → {v['filename']}")

# Camera (expect 6)
cam_ch = {k: v for k, v in ch.items() if "CAM_" in k}
print(f"\n✅ Camera → {len(cam_ch)}/6 channels:")
for k, v in cam_ch.items():
    print(f"    {k:25s} → {v['filename']}")

# Assert all 12 present
assert len(ch) == 12, f"Expected 12 sensor channels, got {len(ch)}"
print(f"\n✅ All 12 sensor channels confirmed for {sample_id}.")

```

✓ LiDAR → samples/LIDAR_TOP/n015-2018-07-24-11-22-45+0800__LIDAR_TOP__1532402929197353.pcd.bin

✓ Radar → 5/5 channels:

RADAR_FRONT → samples/RADAR_FRONT/n015-2018-07-24-11-22-45+0800__RADAR_FRONT__1532402929175111.pcd

RADAR_FRONT_LEFT → samples/RADAR_FRONT_LEFT/n015-2018-07-24-11-22-45+0800__RADAR_FRONT_LEFT__1532402929192815.pcd

RADAR_FRONT_RIGHT → samples/RADAR_FRONT_RIGHT/n015-2018-07-24-11-22-45+0800__RADAR_FRONT_RIGHT__1532402929174655.pcd

RADAR_BACK_LEFT → samples/RADAR_BACK_LEFT/n015-2018-07-24-11-22-45+0800__RADAR_BACK_LEFT__1532402929198900.pcd

RADAR_BACK_RIGHT → samples/RADAR_BACK_RIGHT/n015-2018-07-24-11-22-45+0800__RADAR_BACK_RIGHT__1532402929171003.pcd

✓ Camera → 6/6 channels:

CAM_FRONT → samples/CAM_FRONT/n015-2018-07-24-11-22-45+0800__CAM_FRONT__1532402929162460.jpg

CAM_FRONT_RIGHT → samples/CAM_FRONT_RIGHT/n015-2018-07-24-11-22-45+0800__CAM_FRONT_RIGHT__1532402929170339.jpg

CAM_BACK_RIGHT → samples/CAM_BACK_RIGHT/n015-2018-07-24-11-22-45+0800__CAM_BACK_RIGHT__1532402929177893.jpg

CAM_BACK → samples/CAM_BACK/n015-2018-07-24-11-22-45+0800__CAM_BACK__1532402929187525.jpg

CAM_BACK_LEFT → samples/CAM_BACK_LEFT/n015-2018-07-24-11-22-45+0800__CAM_BACK_LEFT__1532402929197423.jpg

CAM_FRONT_LEFT → samples/CAM_FRONT_LEFT/n015-2018-07-24-11-22-45+0800__CAM_FRONT_LEFT__1532402929154844.jpg

✓ All 12 sensor channels confirmed for sample_0003.

```

In [5]: # -----
# CELL 5 – CSV report: one row per sample
# -----

import json
import pandas as pd
from config import STEP0_DIR

report_path = STEP0_DIR / "samples_report.csv"

with open(STEP0_DIR / "samples_index.json") as f:
    index = json.load(f)

rows = []
for sample_id, info in sorted(index.items()):
    sample_folder = STEP0_DIR / info["folder"]    # relative → absolute via con
fig
    with open(sample_folder / "sensor_meta.json") as f:
        meta = json.load(f)
        ch = meta["sensor_channels"]

        lidar_files = {k: v["filename"] for k, v in ch.items() if "LIDAR" in k}
        radar_files = {k: v["filename"] for k, v in ch.items() if "RADAR" in k}
        camera_files = {k: v["filename"] for k, v in ch.items() if "CAM_" in k}

        rows.append({
            "sample_id" : sample_id,
            "sample_token" : info["sample_token"],
            "timestamp_us" : info["timestamp_us"],
            "num_lidar" : len(lidar_files),
            "num_radar" : len(radar_files),
            "num_camera" : len(camera_files),
            "lidar_file" : lidar_files.get("LIDAR_TOP", ""),
            "radar_front" : radar_files.get("RADAR_FRONT", ""),
            "cam_front" : camera_files.get("CAM_FRONT", ""),
        })

df = pd.DataFrame(rows)
df.to_csv(report_path, index=False)

assert len(df) == len(index), f"Row count mismatch: {len(df)} vs {len(index)}"
print(f"✅ CSV report saved: {report_path}")
print(f"    {len(df)} rows | columns: {df.columns.tolist()}")
display(df.head())

```

✓ CSV report saved: F:\Sensor fusion Research\output\step_0\samples_report.csv
404 rows | columns: ['sample_id', 'sample_token', 'timestamp_us', 'num_lidar', 'num_radar', 'num_camera', 'lidar_file', 'radar_front', 'cam_front']

	sample_id	sample_token	timestamp_us	num_lidar	num_radar	num_camera
0	sample_0000	ca9a282c9e77460f8360f564131a8af5	1532402927647951	1	5	
1	sample_0001	39586f9d59004284a7114a68825e8eec	1532402928147847	1	5	
2	sample_0002	356d81f38dd9473ba590f39e266f54e5	1532402928698048	1	5	
3	sample_0003	e0845f5322254dafadbbed75aaa07969	1532402929197353	1	5	
4	sample_0004	c923fe08b2ff4e27975d2bf30934383b	1532402929697797	1	5	



In []:

In []:

In []:

Step 1.1 — Camera Preprocessing (Upgraded)

Input	output/step_0/samples_index.json , sensor_meta.json per sample
Outputs	output/step_1/camera_meta.json — calibration + intrinsics per sample output/step_1/camera_timestamp_report.json — validity report output/step_1/projection_check_<sample>.png — visual proof of projection correctness
New in this version	project_point_to_camera() — the missing piece that lets you project ANY 3D point (LiDAR cluster, radar detection, fused UKF object) onto ANY camera view, correctly, even though sensors fire at slightly different timestamps
Used by	Step 4.3 (camera tracking), Step 4.4 (fusion), Step 6 (visualization)

Why this upgrade exists

The original notebook collected calibration data (translation, rotation, intrinsics) but never actually **used** it to project a 3D point onto an image. This meant every downstream fusion step that tried to draw LiDAR/radar detections onto camera images had no correct way to do it — points would be computed using only one sensor's ego_pose, ignoring that the car moves between each sensor's capture timestamp. This version adds the full 5-step transform chain and a visual sanity check.

```

In [1]: # -----
# CELL 1 – Verify config.py exists
# -----

from pathlib import Path

if not Path("config.py").exists():
    raise FileNotFoundError(
        "config.py not found in this folder. Copy it from the repo root "
        "and set DATA_ROOT to your nuScenes dataset path."
    )

from config import DATA_ROOT, NUSCENES_VERSION, STEP0_DIR, STEP1_DIR

if not DATA_ROOT.exists():
    raise FileNotFoundError(f"DATA_ROOT does not exist: {DATA_ROOT}")

print(f"✅ config.py found")
print(f"✅ DATA_ROOT : {DATA_ROOT}")
print(f"✅ STEP0_DIR : {STEP0_DIR}")
print(f"✅ STEP1_DIR : {STEP1_DIR}")

```

```

config.py loaded. PROJECT_ROOT = F:\Sensor fusion Research
DATA_ROOT   = F:\Sensor fusion Research\DATA SET\archive
OUTPUT_ROOT = F:\Sensor fusion Research\output
✅ config.py found
✅ DATA_ROOT : F:\Sensor fusion Research\DATA SET\archive
✅ STEP0_DIR  : F:\Sensor fusion Research\output\step_0
✅ STEP1_DIR  : F:\Sensor fusion Research\output\step_1

```

```

In [2]: # -----
# CELL 2 – Main pipeline: parse camera metadata + intrinsics
# FIXED: folder path now joins with STEP0_DIR (matches new Step 0 output)
# FIXED: removed broken syntax (stray text was inside the dict literal)
# -----

from nuscenes.nuscenes import NuScenes
import json

nusc = NuScenes(version=NUSCENES_VERSION, dataroot=str(DATA_ROOT), verbose=False)

with open(STEP0_DIR / "samples_index.json") as f:
    samples_index = json.load(f)

camera_data = {}

for sample_id, sample_info in samples_index.items():
    # FIXED – join with STEP0_DIR since "folder" is now a relative name
    sensor_meta_path = STEP0_DIR / sample_info["folder"] / "sensor_meta.json"
    with open(sensor_meta_path) as f:
        sensor_meta = json.load(f)

    cam_views = {}
    for channel, meta in sensor_meta["sensor_channels"].items():
        if meta["modality"] != "camera":
            continue

        cs = nusc.get('calibrated_sensor', meta["calibrated_sensor_token"])

        cam_views[channel] = {
            "filename": meta["filename"],
            "timestamp": meta["timestamp"],
            "ego_pose": meta["ego_pose"],
            "calibrated_sensor": {
                "translation": meta["sensor_to_ego_translation"],
                "rotation": meta["sensor_to_ego_rotation"],
                "camera_intrinsic": cs["camera_intrinsic"]
            }
        }

    if cam_views:
        camera_data[sample_id] = cam_views

output_file = STEP1_DIR / "camera_meta.json"
with open(output_file, "w") as f:
    json.dump(camera_data, f, indent=2)

assert len(camera_data) == len(samples_index), \
    f"Expected {len(samples_index)} samples with camera data, got {len(camera_data)}"

print(f"✅ Step 1.1 complete: {len(camera_data)} samples processed.")
print(f"📄 Saved to: {output_file}")

```

- ✓ Step 1.1 complete: 404 samples processed.
- 📄 Saved to: F:\Sensor fusion Research\output\step_1\camera_meta.json

🎯 NEW: The projection utility

This is the piece that was missing. It takes a 3D point from ANY sensor frame (LiDAR, radar, or a fused global-frame object from your UKF) and correctly projects it onto ANY camera's image plane — properly bridging through the global frame so the car's motion between sensor timestamps is accounted for.

Reuse this function in Step 4.3 (camera tracking) and Step 4.4 (fusion) — it's the piece those notebooks need to draw fused objects onto camera images correctly.

```
In [3]: # -----  
# CELL 3 – Projection utility: 3D point (any sensor frame) → camera pixel  
# Moved to src/geometry.py – shared with Step 2.3.1, Step 3.1, Step 3.2  
# -----  
  
import numpy as np  
from pyquaternion import Quaternion  
from src.geometry import transform_matrix, project_point_to_camera  
  
print("\u2705 Projection utility loaded: transform_matrix(), project_point_to_  
camera()")
```

✓ Projection utility loaded: transform_matrix(), project_point_to_camera()

🔍 Visual proof: project real LiDAR points onto CAM_FRONT

If the projection chain above is correct, LiDAR points should land exactly on the road surface and on objects (cars, curbs, poles) in the camera image — not drift off to the side. This is the check to run every time you touch the projection logic.

```

In [4]: # -----
# CELL 4 – Visual validation: overlay real LiDAR points onto CAM_FRONT
# -----

import matplotlib.pyplot as plt
from PIL import Image
from nuscenes.utils.data_classes import LidarPointCloud

sample_id = "sample_0020" # ← change to check any sample

sensor_meta_path = STEP0_DIR / samples_index[sample_id]["folder"] / "sensor_me
ta.json"
with open(sensor_meta_path) as f:
    sensor_meta = json.load(f)

lidar_meta = sensor_meta["sensor_channels"]["LIDAR_TOP"]
cam_meta = sensor_meta["sensor_channels"]["CAM_FRONT"]

# Load raw LiDAR points (in LiDAR sensor frame)
lidar_path = DATA_ROOT / lidar_meta["filename"]
pc = LidarPointCloud.from_file(str(lidar_path))
points = pc.points[:3, :].T # shape (N, 3)

# Camera intrinsic for CAM_FRONT
cs_cam = nusc.get('calibrated_sensor', cam_meta["calibrated_sensor_token"])
camera_intrinsic = cs_cam["camera_intrinsic"]

src_calibrated_sensor = {
    "translation": lidar_meta["sensor_to_ego_translation"],
    "rotation": lidar_meta["sensor_to_ego_rotation"]
}
src_ego_pose = lidar_meta["ego_pose"]
cam_calibrated_sensor = {
    "translation": cam_meta["sensor_to_ego_translation"],
    "rotation": cam_meta["sensor_to_ego_rotation"]
}
cam_ego_pose = cam_meta["ego_pose"]

# Project every 3rd point (for speed) using OUR function
results = []
for p in points[:3]:
    r = project_point_to_camera(
        p, src_ego_pose, src_calibrated_sensor,
        cam_ego_pose, cam_calibrated_sensor, camera_intrinsic
    )
    if r is not None:
        results.append(r)

results = np.array(results)
print(f"✅ Projected {len(results)} / {len(points[:3])} points onto CAM_FRONT "
      f"({len(results)/len(points[:3])*100:.1f}% landed inside the image)")

# Overlay on the actual camera image
img_path = DATA_ROOT / cam_meta["filename"]
img = Image.open(img_path)

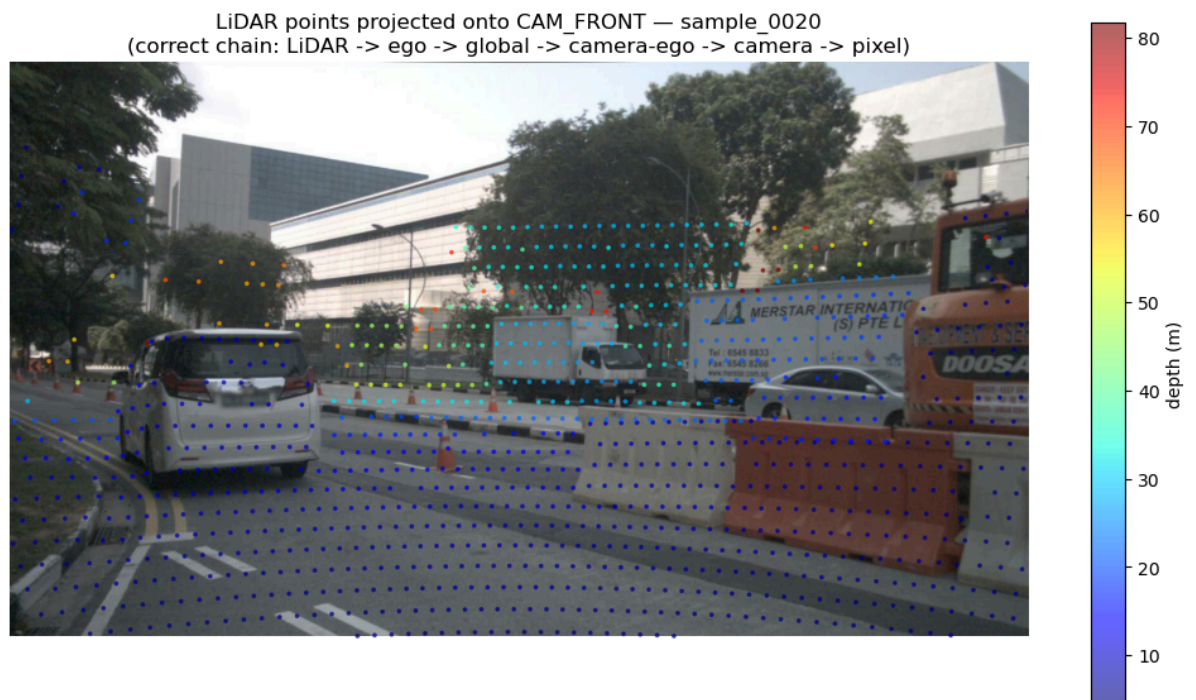
```

```
plt.figure(figsize=(13, 7))
plt.imshow(img)
sc = plt.scatter(results[:, 0], results[:, 1], c=results[:, 2],
                  cmap='jet', s=3, alpha=0.6)
plt.colorbar(sc, label='depth (m)')
plt.title(f"LiDAR points projected onto CAM_FRONT - {sample_id}\n"
          f"(correct chain: LiDAR -> ego -> global -> camera-ego -> camera -> "
          f"pixel)")
plt.axis('off')

save_path = STEP1_DIR / f"projection_check_{sample_id}.png"
plt.savefig(save_path, dpi=150, bbox_inches='tight')
plt.show()

print(f"📄 Saved visual check to: {save_path}")
print("\nIf points hug the road surface and outline vehicles/curbs correctl"
      y,")
print("your projection chain is now correct and safe to reuse in Step 4.3 / 4."
      4.)")
```

✓ Projected 1017 / 11563 points onto CAM_FRONT (8.8% landed inside the image)



📄 Saved visual check to: F:\Sensor fusion Research\output\step_1\projection_check_sample_0020.png

If points hug the road surface and outline vehicles/curbs correctly, your projection chain is now correct and safe to reuse in Step 4.3 / 4.4.

Data quality checks (kept from original, paths fixed)

```
In [5]: # -----  
# CELL 5 – Check all 6 camera channels are present per sample  
# -----  
  
with open(STEP1_DIR / "camera_meta.json") as f:  
    camera_data = json.load(f)  
  
expected_channels = {  
    "CAM_FRONT", "CAM_FRONT_LEFT", "CAM_FRONT_RIGHT",  
    "CAM_BACK", "CAM_BACK_LEFT", "CAM_BACK_RIGHT"  
}  
  
missing_channels = {}  
for sample_id, cams in camera_data.items():  
    missing = expected_channels - set(cams.keys())  
    if missing:  
        missing_channels[sample_id] = list(missing)  
  
if not missing_channels:  
    print("✅ All samples have all 6 camera channels.")  
else:  
    print(f"⚠️ {len(missing_channels)} samples are missing camera views:")  
    for sid, missing in sorted(missing_channels.items()):  
        print(f"    - {sid}: missing {missing}")  
  
✅ All samples have all 6 camera channels.
```

```

In [6]: # -----
# CELL 6 – Timestamp validity check across all 6 cameras
# -----

complete_samples = []
partial_samples = {}

expected_cams = ["CAM_FRONT", "CAM_FRONT_RIGHT", "CAM_BACK_RIGHT",
                 "CAM_BACK", "CAM_BACK_LEFT", "CAM_FRONT_LEFT"]

for sample_id, cams in camera_data.items():
    missing = [cam for cam in expected_cams
               if not isinstance(cams.get(cam, {}).get("timestamp"), int)]
    if missing:
        partial_samples[sample_id] = missing
    else:
        complete_samples.append(sample_id)

print(f"📊 Total samples: {len(camera_data)}")
print(f"✅ Fully valid (all 6 cameras have timestamp): {len(complete_samples)}")
print(f"⚠️ Partially missing: {len(partial_samples)}")

report_path = STEP1_DIR / "camera_timestamp_report.json"
with open(report_path, "w") as f:
    json.dump({
        "fully_valid_samples": complete_samples,
        "partially_missing_samples": partial_samples
    }, f, indent=2)

print(f"📄 Report saved to: {report_path}")

📊 Total samples: 404
✅ Fully valid (all 6 cameras have timestamp): 404
⚠️ Partially missing: 0
📄 Report saved to: F:\Sensor fusion Research\output\step_1\camera_timestamp_report.json

```

In []:

In []:

Step 1.2 — Radar Parsing (Upgraded)

Input	output/step_0/samples_index.json , sensor_meta.json per sample
Outputs	output/step_1/radar/<sample>/<channel>.json — per-point radar data + calibration output/step_1/radar_parsed_data.csv — combined table (replaces Excel) output/step_1/radar_channel_report.json — completeness check
Used by	Step 2.2 (radar processing/filtering), Step 3.2 (radar tracking), Step 4.4 (fusion)

Two bugs fixed from the original version

1. **dyn_prop was reading the wrong array index.** nuScenes radar points have 18 fields in this order:

```
0:x 1:y 2:z 3:dyn_prop 4:id 5:rcs 6:vx 7:vy 8:vx_comp 9:vy_comp
10:is_quality_valid 11:ambig_state 12:x_rms 13:y_rms 14:invalid_state 15:pd
h0 16:vx_rms 17:vy_rms
```

The original code read `pt[10]` (`is_quality_valid`) and called it `dyn_prop`. Fixed to `pt[3]`.

2. **Velocity fields were compensated, not raw.** The original code used `pt[8]`, `pt[9]` (`vx_comp`, `vy_comp` — ego-motion compensated) as `vx`, `vy`. For **TTC / closing-speed**, you need the **raw** Doppler velocity at `pt[6]`, `pt[7]`, which is the true relative velocity to your vehicle. This version stores **both**, clearly labeled, so downstream TTC logic uses the correct one.

What's new

- Each point now carries its channel's `sensor_to_ego_translation/rotation` and `ego_pose` — this calibration is stored here for a *downstream* notebook to consume (Step 2.2, Step 3.2, or a projection step) when it needs to transform radar points into ego/global frame or project them onto camera images. This notebook does not call `project_point_to_camera()` itself; nothing here performs a projection.
- Derived fields (`range`, `azimuth`, `relative_speed`) are computed once in the main loop, so every output file has them — not only the Excel export.
- A diagnostic cell shows how many points nuScenes' default filtering silently drops, for reproducibility in your write-up.

```

In [1]: # -----
# CELL 1 – Verify config.py exists, load the master sample index
# -----

import json
from pathlib import Path

if not Path("config.py").exists():
    raise FileNotFoundError(
        "config.py not found in this folder. Copy it from the repo root "
        "and set DATA_ROOT to your nuScenes dataset path."
    )

from config import DATA_ROOT, NUSCENES_VERSION, STEP0_DIR, STEP1_DIR

if not DATA_ROOT.exists():
    raise FileNotFoundError(f"DATA_ROOT does not exist: {DATA_ROOT}")

RADAR_OUT_DIR = STEP1_DIR / "radar"
RADAR_OUT_DIR.mkdir(parents=True, exist_ok=True)

with open(STEP0_DIR / "samples_index.json") as f:
    samples_index = json.load(f)

print(f"✅ config.py found")
print(f"✅ DATA_ROOT      : {DATA_ROOT}")
print(f"✅ STEP0_DIR         : {STEP0_DIR}")
print(f"✅ RADAR_OUT_DIR    : {RADAR_OUT_DIR}")
print(f"✅ Loaded {len(samples_index)} samples from index.")

config.py loaded. PROJECT_ROOT = F:\Sensor fusion Research
DATA_ROOT   = F:\Sensor fusion Research\DATA SET\archive
OUTPUT_ROOT = F:\Sensor fusion Research\output
✅ config.py found
✅ DATA_ROOT      : F:\Sensor fusion Research\DATA SET\archive
✅ STEP0_DIR       : F:\Sensor fusion Research\output\step_0
✅ RADAR_OUT_DIR  : F:\Sensor fusion Research\output\step_1\radar
✅ Loaded 404 samples from index.

```

```

In [2]: # -----
# CELL 2 – Diagnostic: how many points does nuScenes' default filter drop?
# RadarPointCloud.from_file() silently applies default filtering
# (invalid_state=[0], dynprop_states=range(7), ambig_states=[3]) before
# you ever see the array. This makes that visible for one sample.
# Purely informational – Cell 3 does not depend on anything defined here,
# so this cell can be skipped or deleted without affecting the pipeline.
# -----

from nuscenes.utils.data_classes import RadarPointCloud

sample_id = list(samples_index.keys())[0]
sensor_meta_path = STEP0_DIR / samples_index[sample_id]["folder"] / "sensor_me
ta.json"
with open(sensor_meta_path) as f:
    sensor_meta = json.load(f)

radar_meta = sensor_meta["sensor_channels"]["RADAR_FRONT"]
radar_path = DATA_ROOT / radar_meta["filename"]

# Default (filtered) load – what your pipeline actually uses
pc_filtered = RadarPointCloud.from_file(str(radar_path))

# Unfiltered load – disable all default quality filters
pc_raw = RadarPointCloud.from_file(
    str(radar_path),
    invalid_states=list(range(18)),
    dynprop_states=list(range(8)),
    ambig_states=list(range(5))
)

n_filtered = pc_filtered.points.shape[1]
n_raw = pc_raw.points.shape[1]

print(f"RADAR_FRONT, {sample_id}:")
print(f"  Raw points in file      : {n_raw}")
print(f"  After default quality filter: {n_filtered}")
print(f"  Dropped by devkit defaults : {n_raw - n_filtered} "
      f"({(n_raw - n_filtered) / max(n_raw,1) * 100:.1f}%)")
print("\n👉 This filtering happens BEFORE your own 'valid' check – worth")
print("  noting in your methodology write-up for reproducibility.")

```

```

RADAR_FRONT, sample_0000:
  Raw points in file      : 125
  After default quality filter: 74
  Dropped by devkit defaults : 51 (40.8%)

```

👉 This filtering happens BEFORE your own 'valid' check – worth noting in your methodology write-up for reproducibility.

```

In [3]: # -----
# CELL 3 – Main pipeline: parse radar points with FIXED indices,
# both raw and compensated velocity, and calibration attached.
# -----

import math
import numpy as np
from nuscnescenes.utils.data_classes import RadarPointCloud

total_points_written = 0
total_channels_written = 0
failed_channels = []

for sample_id, info in samples_index.items():
    # FIXED – join with STEP0_DIR since "folder" is a relative name
    sensor_meta_path = STEP0_DIR / info["folder"] / "sensor_meta.json"
    with open(sensor_meta_path) as f:
        sensor_meta = json.load(f)

    radar_channels = [s for s in info["sensors"] if s.startswith("RADAR_")]
    if not radar_channels:
        continue

    radar_out_sample = RADAR_OUT_DIR / sample_id
    radar_out_sample.mkdir(parents=True, exist_ok=True)

    for radar_channel in radar_channels:
        meta = sensor_meta["sensor_channels"][radar_channel]
        radar_data_path = DATA_ROOT / meta["filename"]

        try:
            radar_pc = RadarPointCloud.from_file(str(radar_data_path))
            radar_array = radar_pc.points.T # shape (N, 18)

            # Calibration – needed for ego/global transform and camera project
            calibration = {
                "sensor_to_ego_translation": meta["sensor_to_ego_translatio
n"],
                "sensor_to_ego_rotation": meta["sensor_to_ego_rotation"],
                "ego_pose": meta["ego_pose"]
            }

            parsed_points = []
            for pt in radar_array:
                x, y, z = float(pt[0]), float(pt[1]), float(pt[2])
                dyn_prop = int(pt[3]) # FIXED: was pt[1
0]

                vx_raw, vy_raw = float(pt[6]), float(pt[7]) # raw Doppler vel
ocity – use for TTC
                vx_comp, vy_comp = float(pt[8]), float(pt[9]) # ego-compensat
ed – use for moving/stationary classification
                is_quality_valid = int(pt[10]) # FIXED: correctl
y labeled now

                valid = (not math.isnan(vx_raw)) and (not math.isnan(vy_raw))

```

```

        range_val = math.hypot(x, y)
        azimuth_deg = math.degrees(math.atan2(y, x))
        # Closing speed relative to ego – uses RAW velocity (correct f
or TTC)
        relative_speed = (x * vx_raw + y * vy_raw) / range_val if rang
e_val > 0 else 0.0

        parsed_points.append({
            "x": x, "y": y, "z": z,
            "vx": vx_raw, "vy": vy_raw,                # relative ve
Locality (for TTC)
            "vx_comp": vx_comp, "vy_comp": vy_comp,    # compensated
velocity (for motion classification)
            "dyn_prop": dyn_prop,
            "is_quality_valid": is_quality_valid,
            "valid": valid,
            "range": round(range_val, 3),
            "azimuth_deg": round(azimuth_deg, 2),
            "relative_speed": round(relative_speed, 3),
        })

    out_data = {
        "sample_id": sample_id,
        "radar_channel": radar_channel,
        "calibration": calibration,
        "points": parsed_points
    }

    out_file = radar_out_sample / f"{radar_channel}.json"
    with open(out_file, "w") as f:
        json.dump(out_data, f, indent=2)

    total_points_written += len(parsed_points)
    total_channels_written += 1

    except Exception as e:
        print(f"❌ Failed parsing {radar_channel} for {sample_id}: {e}")
        failed_channels.append({"sample_id": sample_id, "channel": radar_c
hannel, "error": str(e)})
        continue

assert total_channels_written > 0, "No radar channels were written – check DAT
A_ROOT and file paths"

print(f"✅ Step 1.2 complete.")
print(f"    Channels written : {total_channels_written}")
print(f"    Total points      : {total_points_written}")

if failed_channels:
    with open(RADAR_OUT_DIR / "parse_failures.json", "w") as f:
        json.dump(failed_channels, f, indent=2)
    print(f"⚠️ {len(failed_channels)} channel-parses failed – see parse_failu
res.json")
else:
    with open(RADAR_OUT_DIR / "parse_failures.json", "w") as f:
        json.dump(failed_channels, f, indent=2)
    print(f"✅ No parse failures – parse_failures.json written empty.")

```

- ✓ Step 1.2 complete.
Channels written : 2020
Total points : 79519
- ✓ No parse failures – parse_failures.json written empty.

```
In [4]: # -----
# CELL 4 – Combined CSV export (replaces Excel – faster, smaller, git-friendly)
# Excel kept as optional for manual inspection only.
# -----

import pandas as pd

all_rows = []
for sample_folder in sorted(RADAR_OUT_DIR.iterdir()):
    if not sample_folder.is_dir():
        continue
    for radar_file in sample_folder.glob("*.json"):
        with open(radar_file) as f:
            data = json.load(f)
            for pt in data["points"]:
                row = dict(pt)
                row["sample_id"] = data["sample_id"]
                row["radar_channel"] = data["radar_channel"]
                all_rows.append(row)

radar_df = pd.DataFrame(all_rows)

# Save full CSV (fast, portable)
csv_path = STEP1_DIR / "radar_parsed_data.csv"
radar_df.to_csv(csv_path, index=False)
print(f"✅ Combined CSV saved: {csv_path}")
print(f"    {len(radar_df)} total points across {radar_df['radar_channel'].nunique()} channel types")

# Optional filtered view – valid + moving points only (uses raw velocity)
filtered_df = radar_df[radar_df["valid"] & ((radar_df["vx"] != 0) | (radar_df["vy"] != 0))]
print(f"    {len(filtered_df)} valid + moving points ({len(filtered_df)/len(radar_df)*100:.1f}%)")

display(radar_df.head())
```

```
✅ Combined CSV saved: F:\Sensor fusion Research\output\step_1\radar_parsed_data.csv
79519 total points across 5 channel types
67008 valid + moving points (84.3%)
```

	x	y	z	vx	vy	vx_comp	vy_comp	dyn_prop	is_quality_valid	valid	range	azim
0	8.6	-2.9	0.0	8.75	-0.25	0.243894	-0.082243	1	1	True	9.076	
1	9.0	7.3	0.0	9.75	-0.25	-0.004036	-0.003273	1	1	True	11.588	
2	12.6	-0.1	0.0	8.75	-0.25	-0.094915	0.000753	1	1	True	12.600	
3	14.0	0.7	0.0	9.00	-0.25	0.090357	0.004518	1	1	True	14.017	
4	14.8	7.9	0.0	9.50	-0.25	0.040494	0.021615	1	1	True	16.776	

```

In [5]: # -----
# CELL 5 – Check all 5 radar channels are present per sample
# -----

expected_radar_channels = {
    "RADAR_FRONT", "RADAR_FRONT_LEFT", "RADAR_FRONT_RIGHT",
    "RADAR_BACK_LEFT", "RADAR_BACK_RIGHT"
}

missing_report = {}
for sample_folder in sorted(RADAR_OUT_DIR.iterdir()):
    if not sample_folder.is_dir():
        continue
    present = {f.stem for f in sample_folder.glob("*.json")}
    missing = expected_radar_channels - present
    if missing:
        missing_report[sample_folder.name] = sorted(missing)

if not missing_report:
    print("✅ All samples have all 5 radar channels.")
else:
    print(f"⚠️ {len(missing_report)} samples missing radar channels:")
    for sid, missing in sorted(missing_report.items())[:10]:
        print(f"  - {sid}: missing {missing}")

report_path = STEP1_DIR / "radar_channel_report.json"
with open(report_path, "w") as f:
    json.dump(missing_report, f, indent=2)
print(f"📄 Report saved to: {report_path}")

✅ All samples have all 5 radar channels.
📄 Report saved to: F:\Sensor fusion Research\output\step_1\radar_channel_report.json

```

In []:

In []:

In []:

Step 1.3 — LiDAR Parsing (Upgraded)

Input	output/step_0/samples_index.json , sensor_meta.json per sample
Outputs	output/step_1/lidar/<sample>/lidar_raw.npy — points + derived fields (N×7)
	output/step_1/lidar/<sample>/lidar_meta.json — calibration + per-sample stats
	output/step_1/lidar_summary.csv — one row per sample (fast)
	output/step_1/lidar/<sample>/lidar_points.csv — optional, full per-point (opt-in, large)
Used by	Step 2.1 (RANSAC + DBSCAN), Step 3.3 (LiDAR tracking), Step 4.4 (fusion)

What changed from the original

1. **Path fix** — `Path(info['folder'])` now correctly joins with `STEP0_DIR`.
2. **Added derived fields** — `distance` , `azimuth_deg` , `elevation_deg` per point, computed vectorized with NumPy and appended as extra columns to the saved array (now N×7 instead of N×4). Your dissertation (section 3.1.3) already describes these fields as being computed — this closes the gap between that description and the actual code.
3. **Added CSV export** — a fast per-sample summary CSV by default (`num_points`, distance/elevation stats). Full per-point CSV is available but off by default since it would be 10+ million rows across all samples — flip `EXPORT_FULL_POINT_CSV = True` below if you specifically want that.
4. **Kept your calibration handling as-is** — it was already correct.

```

In [1]: # -----
# CELL 1 – Verify config.py exists
# -----

from pathlib import Path

if not Path("config.py").exists():
    raise FileNotFoundError(
        "config.py not found in this folder. Copy it from the repo root "
        "and set DATA_ROOT to your nuScenes dataset path."
    )

from config import DATA_ROOT, NUSCENES_VERSION, STEP0_DIR, STEP1_DIR

if not DATA_ROOT.exists():
    raise FileNotFoundError(f"DATA_ROOT does not exist: {DATA_ROOT}")

LIDAR_OUT_DIR = STEP1_DIR / "lidar"
LIDAR_OUT_DIR.mkdir(parents=True, exist_ok=True)

# Set True only if you want a full per-point CSV per sample (large, slow).
# Default False – the per-sample summary CSV covers most needs.
EXPORT_FULL_POINT_CSV = False

print(f"✅ config.py found")
print(f"✅ DATA_ROOT      : {DATA_ROOT}")
print(f"✅ STEP0_DIR        : {STEP0_DIR}")
print(f"✅ LIDAR_OUT_DIR    : {LIDAR_OUT_DIR}")
print(f"✅ EXPORT_FULL_POINT_CSV : {EXPORT_FULL_POINT_CSV}")

config.py loaded. PROJECT_ROOT = F:\Sensor fusion Research
DATA_ROOT    = F:\Sensor fusion Research\DATA SET\archive
OUTPUT_ROOT  = F:\Sensor fusion Research\output
✅ config.py found
✅ DATA_ROOT      : F:\Sensor fusion Research\DATA SET\archive
✅ STEP0_DIR       : F:\Sensor fusion Research\output\step_0
✅ LIDAR_OUT_DIR   : F:\Sensor fusion Research\output\step_1\lidar
✅ EXPORT_FULL_POINT_CSV : False

```

```

In [2]: # -----
# CELL 2 – Main pipeline: parse LiDAR points + compute derived fields
# -----

import json
import numpy as np
import pandas as pd
from nuscenes.nuscenes import NuScenes
from nuscenes.utils.data_classes import LidarPointCloud
from tqdm import tqdm

with open(STEP0_DIR / "samples_index.json") as f:
    samples_index = json.load(f)

nusc = NuScenes(version=NUSCENES_VERSION, dataroot=str(DATA_ROOT), verbose=False)

summary_rows = []
skipped_samples = []

for sample_id, info in tqdm(samples_index.items(), total=len(samples_index), desc="Parsing LiDAR"):

    # FIXED – join with STEP0_DIR since "folder" is a relative name
    meta_path = STEP0_DIR / info["folder"] / "sensor_meta.json"
    with open(meta_path) as f:
        sensor_meta = json.load(f)

    if "LIDAR_TOP" not in sensor_meta["sensor_channels"]:
        skipped_samples.append(sample_id)
        continue

    lidar_meta = sensor_meta["sensor_channels"]["LIDAR_TOP"]
    lidar_path = DATA_ROOT / lidar_meta["filename"]

    # Read cloud (N,4): x, y, z, intensity – in LiDAR sensor frame
    lidar_pc = LidarPointCloud.from_file(str(lidar_path))
    pts = lidar_pc.points.T  # (N, 4)

    x, y, z, intensity = pts[:, 0], pts[:, 1], pts[:, 2], pts[:, 3]

    # — Derived fields (vectorized, matches dissertation section 3.1.3) —
    distance = np.sqrt(x**2 + y**2 + z**2)
    azimuth_deg = np.degrees(np.arctan2(y, x))
    horizontal_dist = np.sqrt(x**2 + y**2)
    elevation_deg = np.degrees(np.arctan2(z, horizontal_dist))

    # Stack into N×7: x, y, z, intensity, distance, azimuth_deg, elevation_deg
    pts_enriched = np.column_stack([x, y, z, intensity, distance, azimuth_deg, elevation_deg])

    sample_out_dir = LIDAR_OUT_DIR / sample_id
    sample_out_dir.mkdir(parents=True, exist_ok=True)

    npy_path = sample_out_dir / "lidar_raw.npy"
    np.save(npy_path, pts_enriched.astype(np.float32))

```

```

# — Metadata + calibration (kept from original – was already correct) —
meta_out = {
    "sample_id": sample_id,
    "num_points": int(pts.shape[0]),
    "source_file": lidar_meta["filename"],
    "columns": ["x", "y", "z", "intensity", "distance", "azimuth_deg", "elevation_deg"],
    "sensor_to_ego_translation": lidar_meta["sensor_to_ego_translation"],
    "sensor_to_ego_rotation": lidar_meta["sensor_to_ego_rotation"],
    "ego_pose": lidar_meta["ego_pose"]
}
with open(sample_out_dir / "lidar_meta.json", "w") as f:
    json.dump(meta_out, f, indent=2)

# — Per-sample summary row (fast, always written) —
summary_rows.append({
    "sample_id": sample_id,
    "num_points": int(pts.shape[0]),
    "distance_mean": round(float(distance.mean()), 3),
    "distance_min": round(float(distance.min()), 3),
    "distance_max": round(float(distance.max()), 3),
    "elevation_mean_deg": round(float(elevation_deg.mean()), 3),
    "intensity_mean": round(float(intensity.mean()), 3),
})

# — Optional full per-point CSV (opt-in – Large) —
if EXPORT_FULL_POINT_CSV:
    df_points = pd.DataFrame(
        pts_enriched,
        columns=["x", "y", "z", "intensity", "distance", "azimuth_deg", "elevation_deg"]
    )
    df_points.to_csv(sample_out_dir / "lidar_points.csv", index=False)

assert len(summary_rows) > 0, "No LiDAR samples were processed – check DATA_ROOT and paths"

if skipped_samples:
    print(f"⚠️ {len(skipped_samples)} samples skipped (no LIDAR_TOP): {skipped_samples[:5]}...")
else:
    print("✅ No samples skipped – LIDAR_TOP present in all.")

print(f"\n✅ LiDAR parsing complete: {len(summary_rows)} samples processed.")
print(f"📁 Outputs in: {LIDAR_OUT_DIR}")

```

```
Parsing LiDAR: 100%|███████████  
██████████ | 404/404 [00:20<00:00, 19.82it/s]
```

✔ No samples skipped – LIDAR_TOP present in all.

✓ LiDAR parsing complete: 404 samples processed.

Outputs in: F:\Sensor fusion Research\output\step_1\lidar

[... 47 further output line(s)/block(s) omitted (repetitive per-iteration output) for the PDF export ...]

✔ No samples skipped – LIDAR_TOP present in all.

✅ LiDAR parsing complete: 404 samples processed.

Outputs in: F:\Sensor fusion Research\output\step_1\lidar

```
In [3]: # -----  
# CELL 3 - Save the per-sample summary CSV  
# -----  
  
summary_df = pd.DataFrame(summary_rows).sort_values("sample_id").reset_index(drop=True)  
  
summary_path = STEP1_DIR / "lidar_summary.csv"  
summary_df.to_csv(summary_path, index=False)  
  
assert len(summary_df) == len(samples_index) - len(skipped_samples), \  
    "Summary row count doesn't match expected sample count"  
  
print(f"✅ Summary CSV saved: {summary_path}")  
print(f"    {len(summary_df)} rows | columns: {summary_df.columns.tolist()}")  
display(summary_df.head())
```

SV Summary CSV saved: F:\Sensor fusion Research\output\step_1\lidar_summary.csv

```
404 rows | columns: ['sample_id', 'num_points', 'distance_mean', 'distance_min', 'distance_max', 'elevation_mean_deg', 'intensity_mean']
```

	sample_id	num_points	distance_mean	distance_min	distance_max	elevation_mean_deg	ir
0	sample_0000	34688	11.471	0.000	102.879	-11.158	
1	sample_0001	34720	11.487	0.001	105.160	-11.194	
2	sample_0002	34720	10.913	0.000	104.693	-11.213	
3	sample_0003	34688	10.124	0.000	105.064	-11.126	
4	sample_0004	34752	9.671	0.000	104.934	-11.125	

```

In [4]: # -----
# CELL 4 – Quick validation: load one sample back and confirm shape
# -----

check_sample = summary_df.iloc[50]["sample_id"]
check_path = LIDAR_OUT_DIR / check_sample / "lidar_raw.npy"

loaded = np.load(check_path)

assert loaded.shape[1] == 7, f"Expected 7 columns (x,y,z,intensity,distance,azimuth,elevation), got {loaded.shape[1]}"

print(f"✅ Verified {check_sample}: shape {loaded.shape}")
print(f"    Columns: x, y, z, intensity, distance, azimuth_deg, elevation_deg")
print(f"    Distance range: {loaded[:,4].min():.2f}m to {loaded[:,4].max():.2f}m")
print(f"    Azimuth range : {loaded[:,5].min():.1f}° to {loaded[:,5].max():.1f}°")
print(f"    Elevation range: {loaded[:,6].min():.1f}° to {loaded[:,6].max():.1f}°")

✅ Verified sample_0050: shape (34688, 7)
    Columns: x, y, z, intensity, distance, azimuth_deg, elevation_deg
    Distance range: 0.01m to 88.31m
    Azimuth range : -180.0° to 180.0°
    Elevation range: -51.9° to 10.8°

```

In []:

In []:

In []:

Step 2.1 — LiDAR Ground Removal + Clustering (Upgraded)

Input	output/step_1/lidar/<sample>/lidar_raw.npy (N×7, from Step 1.3)
Outputs	output/step_2/lidar/<sample>/lidar_ground.npy — ground plane points output/step_2/lidar/<sample>/lidar_nonground.npy — obstacle points output/step_2/lidar/<sample>/lidar_cluster_labels.npy — exact DBSCAN label per non-ground point output/step_2/lidar/<sample>/lidar_clusters.json — centroid + point count per cluster output/step_2/lidar_summary.csv — cluster counts per sample output/step_2/bev_<sample>.png — before/after ground removal, saved automatically
Used by	Step 3.3 (LiDAR tracking), Step 4.1/4.4 (fusion)

Bugs fixed from the original

1. **lidar_ground.npy was computed but never saved.** Your own visualization cell (further down) already tried to load it and printed a warning that it was missing — this version saves it.
2. **Cluster visualization used a "crude match"** (distance from centroid < 2.0) instead of the real DBSCAN labels — this misassigns points for any object larger than ~2m or clusters closer than 2m apart. Now the exact `labels` array is saved and reused.
3. **Dead code removed** — a `KDTree(points)` object was built but never used; `DBSCAN(algorithm='kd_tree')` already builds its own internally.
4. **RANSAC switched to Open3D's `segment_plane()`** — faster and more robust than the manual Python loop. The manual version is kept in its own cell below purely as a reference implementation for explaining RANSAC in interviews.
5. **Named constants** for RANSAC and DBSCAN parameters — previously buried as function defaults.

```
In [1]: import open3d as o3d  
  
print(o3d.__version__)
```

```
Jupyter environment detected. Enabling Open3D WebVisualizer.  
[Open3D INFO] WebRTC GUI backend enabled.  
[Open3D INFO] WebRTCWindowSystem: HTTP handshake server disabled.  
0.19.0
```

```
In [2]: # -----
# CELL 1 – Verify config.py exists
# -----

from pathlib import Path

if not Path("config.py").exists():
    raise FileNotFoundError(
        "config.py not found in this folder. Copy it from the repo root."
    )

from config import STEP1_DIR, STEP2_DIR

LIDAR_IN_DIR = STEP1_DIR / "lidar"
LIDAR_OUT_DIR = STEP2_DIR / "lidar"
LIDAR_OUT_DIR.mkdir(parents=True, exist_ok=True)

if not LIDAR_IN_DIR.exists():
    raise FileNotFoundError(f"Step 1.3 output not found at {LIDAR_IN_DIR} – run Step 1.3 first.")

print(f"✅ LIDAR_IN_DIR : {LIDAR_IN_DIR}")
print(f"✅ LIDAR_OUT_DIR : {LIDAR_OUT_DIR}")

config.py loaded. PROJECT_ROOT = F:\Sensor fusion Research
DATA_ROOT = F:\Sensor fusion Research\DATA SET\archive
OUTPUT_ROOT = F:\Sensor fusion Research\output
✅ LIDAR_IN_DIR : F:\Sensor fusion Research\output\step_1\lidar
✅ LIDAR_OUT_DIR : F:\Sensor fusion Research\output\step_2\lidar
```

```
In [3]: # -----
# CELL 2 – Named constants (previously hardcoded as function defaults)
# -----

# RANSAC ground plane fitting
RANSAC_DISTANCE_THRESHOLD = 0.2 # metres – max distance from plane to count as ground
RANSAC_N_POINTS = 3 # points sampled per iteration (minimum to define a plane)
RANSAC_NUM_ITERATIONS = 100 # iterations to search for best-fit plane

# DBSCAN obstacle clustering
DBSCAN_EPS = 0.7 # metres – max distance between points in the same cluster
DBSCAN_MIN_SAMPLES = 10 # minimum points to form a cluster

print("✅ Constants set:")
print(f" RANSAC: distance_threshold={RANSAC_DISTANCE_THRESHOLD}m, "
      f"n_points={RANSAC_N_POINTS}, iterations={RANSAC_NUM_ITERATIONS}")
print(f" DBSCAN: eps={DBSCAN_EPS}m, min_samples={DBSCAN_MIN_SAMPLES}")

✅ Constants set:
RANSAC: distance_threshold=0.2m, n_points=3, iterations=100
DBSCAN: eps=0.7m, min_samples=10
```

Reference implementation — manual RANSAC (for explaining the algorithm, not used in the pipeline)

Keep this for interviews: it shows you understand RANSAC from first principles. The actual pipeline below uses Open3D's optimized `segment_plane()` instead, which does the same thing faster and with better-tested edge case handling.

```
In [4]: # -----  
# CELL 3 – Reference only: manual RANSAC (NOT called by the pipeline)  
# -----  
  
import numpy as np  
  
def remove_ground_ransac_manual(points, threshold=RANSAC_DISTANCE_THRESHOLD,  
                                max_iterations=RANSAC_NUM_ITERATIONS):  
    """  
    Manual RANSAC plane fitting – kept for reference / interview explanation.  
    Algorithm: repeatedly sample 3 random points, fit a plane through them,  
    count inliers within `threshold` distance, keep the best plane found.  
    """  
    max_inliers = []  
    N = len(points)  
  
    for _ in range(max_iterations):  
        sample = points[np.random.choice(N, 3, replace=False)]  
        v1, v2 = sample[1] - sample[0], sample[2] - sample[0]  
        normal = np.cross(v1, v2)  
        if np.linalg.norm(normal) == 0:  
            continue  
        normal = normal / np.linalg.norm(normal)  
        d = -np.dot(normal, sample[0])  
  
        distances = np.abs(points @ normal + d)  
        inliers = np.where(distances < threshold)[0]  
  
        if len(inliers) > len(max_inliers):  
            max_inliers = inliers  
  
    ground_pts = points[max_inliers]  
    non_ground_pts = np.delete(points, max_inliers, axis=0)  
    return ground_pts, non_ground_pts  
  
print("✅ Reference function defined (not used in the pipeline below).")
```

✅ Reference function defined (not used in the pipeline below).

In [5]:

```
# -----
# CELL 4 – Main pipeline: ground removal (Open3D) + clustering (DBSCAN)
# FIXED: saves lidar_ground.npy (was missing)
# FIXED: saves exact cluster labels (removes the "crude match" hack)
# FIXED: dead KDTree object removed
# FIXED (optional): filter out clusters the wrong size/shape to be a
#         vehicle or pedestrian (too flat, too tall, too long and thin) --
#         walls, hedges, and curb fragments were being kept as "objects"
# FIXED: tracks skipped samples (missing lidar_raw.npy), same pattern as
#         Step 1.3, instead of silently continuing past them
# FIXED: this whole pipeline was silently non-deterministic. Open3D's
# segment_plane() fits the ground plane via RANSAC, which samples points
# randomly every call -- and unlike numpy's RNG, it is NOT controlled by
# np.random.seed(); it has its own internal generator with no seed
# parameter exposed on segment_plane() itself. Confirmed directly:
# re-running this cell twice on byte-identical input gave genuinely
# different ground/non-ground splits (e.g. sample_0000's ground-point
# count varied 14863 vs 10671 -- a ~28% swing), which cascades into
# different DBSCAN clusters, different LiDAR tracks, and a different
# downstream MAE on every single full-pipeline run. Fixed with Open3D's
# own global seed hook (o3d.utility.random.seed(), added in 0.19) --
# verified this makes the ENTIRE 404-sample sequence reproducible
# byte-for-byte across separate process runs, not just one call in
# isolation.
# -----

import json
import numpy as np
import open3d as o3d
from sklearn.cluster import DBSCAN
from tqdm import tqdm

LIDAR_RANSAC_SEED = 42
o3d.utility.random.seed(LIDAR_RANSAC_SEED)
print(f"\u2705 Open3D RANSAC seeded ({LIDAR_RANSAC_SEED}) -- ground-plane segm
entation is now reproducible run to run.")

# Plausible vehicle/pedestrian bounding-box shape -- see CELL 4 header.
CLUSTER_MIN_HEIGHT_M = 0.4          # below this: road paint, curb, flat deb
ris
CLUSTER_MAX_HEIGHT_M = 4.5          # above this: building/pole fragment (al
lows buses/trucks)
CLUSTER_MAX_HORIZONTAL_EXTENT_M = 15.0 # wider/longer than this: not a single
object
CLUSTER_MAX_ASPECT_RATIO = 8.0      # Long/short horizontal ratio above thi
s: wall/hedge/guardrail segment

def remove_ground_open3d(points, distance_threshold, ransac_n, num_iteration
s):
    """Ground removal using Open3D's optimized RANSAC plane segmentation."""
    pcd = o3d.geometry.PointCloud()
    pcd.points = o3d.utility.Vector3dVector(points)

    plane_model, inlier_indices = pcd.segment_plane(
        distance_threshold=distance_threshold,
```

```

        ransac_n=ransac_n,
        num_iterations=num_iterations
    )

    inlier_idx = np.array(inlier_idx)
    mask = np.zeros(len(points), dtype=bool)
    mask[inlier_idx] = True

    ground_pts = points[mask]
    non_ground_pts = points[~mask]
    return ground_pts, non_ground_pts

def cluster_points(points, eps, min_samples):
    """DBSCAN handles its own KD-tree acceleration internally via algorithm='kd_tree'."""
    db = DBSCAN(eps=eps, min_samples=min_samples, algorithm='kd_tree', metric='euclidean')
    return db.fit_predict(points)

sample_folders = sorted(LIDAR_IN_DIR.glob("sample_*"))
summary_rows = []
skipped_samples = []

for sample_folder in tqdm(sample_folders, desc="Processing LiDAR"):
    sample_id = sample_folder.name
    out_dir = LIDAR_OUT_DIR / sample_id
    out_dir.mkdir(parents=True, exist_ok=True)

    lidar_path = sample_folder / "lidar_raw.npy"
    if not lidar_path.exists():
        skipped_samples.append(sample_id)
        continue

    pts = np.load(lidar_path)[: , :3]  # x, y, z only (columns 3-6 are intensity/distance/azimuth/elevation)

    # — Ground removal —
    ground, non_ground = remove_ground_open3d(
        pts, RANSAC_DISTANCE_THRESHOLD, RANSAC_N_POINTS, RANSAC_NUM_ITERATIONS
    )
    np.save(out_dir / "lidar_ground.npy", ground)  # FIXED — was never saved before
    np.save(out_dir / "lidar_nonground.npy", non_ground)

    # — Clustering —
    if len(non_ground) >= DBSCAN_MIN_SAMPLES:
        labels = cluster_points(non_ground, DBSCAN_EPS, DBSCAN_MIN_SAMPLES)
    else:
        labels = np.full(len(non_ground), -1)  # too few points to cluster

    # FIXED — save exact labels, aligned index-for-index with lidar_nonground.npy
    np.save(out_dir / "lidar_cluster_labels.npy", labels)
    clusters = {}

```

```

n_discarded_shape = 0
for cluster_id in np.unique(labels):
    if cluster_id == -1:
        continue # DBSCAN noise label
    cluster_pts = non_ground[labels == cluster_id]
    extent = (cluster_pts.max(axis=0) - cluster_pts.min(axis=0))
    dx, dy, dz = float(extent[0]), float(extent[1]), float(extent[2])
    horiz_max, horiz_min = max(dx, dy), max(min(dx, dy), 1e-6)

    # FIXED (optional): discard clusters the wrong size/shape to plausibly
    # be a vehicle or pedestrian -- flat road-surface remnants, curbs, or
    # long thin wall/hedge/guardrail segments that survived ground removal.

    if (dz < CLUSTER_MIN_HEIGHT_M or dz > CLUSTER_MAX_HEIGHT_M
        or horiz_max > CLUSTER_MAX_HORIZONTAL_EXTENT_M
        or horiz_max / horiz_min > CLUSTER_MAX_ASPECT_RATIO):
        n_discarded_shape += 1
        continue

    clusters[str(int(cluster_id))] = {
        "num_points": int(len(cluster_pts)),
        "centroid": cluster_pts.mean(axis=0).tolist(),
        "extent": [dx, dy, dz]
    }

with open(out_dir / "lidar_clusters.json", "w") as f:
    json.dump(clusters, f, indent=2)

summary_rows.append({
    "sample_id": sample_id,
    "num_raw_points": len(pts),
    "num_ground_points": len(ground),
    "num_nonground_points": len(non_ground),
    "num_clusters": len(clusters),
    "num_noise_points": int((labels == -1).sum()),
    "num_discarded_shape": n_discarded_shape
})

assert len(summary_rows) > 0, "No samples processed – check LIDAR_IN_DIR"

if skipped_samples:
    print(f"⚠️ {len(skipped_samples)} samples skipped (no lidar_raw.npy): {skipped_samples[:5]}...")

print(f"\n✅ Step 2.1 complete: {len(summary_rows)} samples processed.")
print(f"📁 Outputs in: {LIDAR_OUT_DIR}")

```

✅ Open3D RANSAC seeded (42) -- ground-plane segmentation is now reproducible run to run.

Processing LiDAR: 0% | 0/404 [00:00<?, ?it/s]

[... 405 further output line(s)/block(s) omitted (repetitive per-iteration output) for the PDF export ...]

✅ Step 2.1 complete: 404 samples processed.

📁 Outputs in: F:\Sensor fusion Research\output\step_2\lidar

```
In [6]: # -----
# CELL 5 – Summary CSV + sanity checks
# FIXED: strengthened completion assertion – cross-checks against Step 1.3's
#         own row count (accounting for anything Step 2.1 itself skipped),
#         instead of only checking summary_rows is non-empty.
# -----

import pandas as pd

summary_df = pd.DataFrame(summary_rows)
summary_path = STEP2_DIR / "lidar_summary.csv"
summary_df.to_csv(summary_path, index=False)

expected_count = len(pd.read_csv(STEP1_DIR / "lidar_summary.csv"))
assert len(summary_rows) == expected_count - len(skipped_samples), \
    f"Expected {expected_count - len(skipped_samples)} samples, got {len(summary_rows)}"

zero_cluster_samples = summary_df[summary_df["num_clusters"] == 0]

print(f"✅ Summary saved: {summary_path}")
print(f"    Avg clusters/sample : {summary_df['num_clusters'].mean():.1f}")
print(f"    Avg ground points    : {summary_df['num_ground_points'].mean():.0f}")
print(f"    Avg non-ground points: {summary_df['num_nonground_points'].mean():.0f}")
print(f"    Ground-point ratio   : {summary_df['num_ground_points'].sum() / summary_df['num_raw_points'].sum() * 100:.1f}%")

if len(zero_cluster_samples) > 0:
    print(f"\n⚠️ {len(zero_cluster_samples)} samples produced ZERO clusters – worth checking:")
    print(zero_cluster_samples['sample_id'].tolist()[:10])

display(summary_df.head())
```

```
✅ Summary saved: F:\Sensor fusion Research\output\step_2\lidar_summary.csv
Avg clusters/sample : 44.0
Avg ground points    : 13248
Avg non-ground points: 21470
Ground-point ratio   : 38.2%
```

	sample_id	num_raw_points	num_ground_points	num_nonground_points	num_clusters	num
0	sample_0000	34688	14482	20206	38	
1	sample_0001	34720	13955	20765	22	
2	sample_0002	34720	13474	21246	28	
3	sample_0003	34688	12612	22076	39	
4	sample_0004	34752	11796	22956	42	

Bird's-Eye-View visualization — auto-saves PNG (no interactive window needed)

Open3D's `draw_geometries()` opens an interactive 3D window that won't render in a saved notebook, a GitHub README, or a headless server. This matplotlib version saves a static top-down image automatically — this is what you want for your portfolio/README.

```

In [7]: # -----
# CELL 6 – Matplotlib BEV: before/after ground removal + clustering
# Uses the REAL cluster labels now, not the old "crude match"
# -----

import matplotlib.pyplot as plt
import matplotlib.cm as cm

sample_id = "sample_0200" # ← change to visualize any sample
sample_dir = LIDAR_OUT_DIR / sample_id

raw_pts = np.load(LIDAR_IN_DIR / sample_id / "lidar_raw.npy")[:, :3]
non_ground = np.load(sample_dir / "lidar_nonground.npy")
labels = np.load(sample_dir / "lidar_cluster_labels.npy")

fig, axes = plt.subplots(1, 2, figsize=(16, 8))

# BEFORE – all raw points
axes[0].scatter(raw_pts[:, 0], raw_pts[:, 1], s=0.5, c='gray', alpha=0.5)
axes[0].scatter(0, 0, c='blue', s=100, marker='s', label='ego vehicle')
axes[0].set_title(f"BEFORE ground removal – {sample_id}")
axes[0].set_xlabel("x (m)"); axes[0].set_ylabel("y (m)")
axes[0].legend(); axes[0].axis('equal')

# AFTER – clusters colored by their REAL DBSCAN Label
unique_labels = [l for l in np.unique(labels) if l != -1]
colors = cm.get_cmap('tab20', max(len(unique_labels), 1))

for i, cluster_id in enumerate(unique_labels):
    cluster_pts = non_ground[labels == cluster_id]
    axes[1].scatter(cluster_pts[:, 0], cluster_pts[:, 1], s=2, color=colors
(i))

noise_pts = non_ground[labels == -1]
if len(noise_pts) > 0:
    axes[1].scatter(noise_pts[:, 0], noise_pts[:, 1], s=1, c='lightgray', alph
a=0.3, label='noise')

axes[1].scatter(0, 0, c='blue', s=100, marker='s', label='ego vehicle')
axes[1].set_title(f"AFTER ground removal + clustering ({len(unique_labels)} cl
usters)")
axes[1].set_xlabel("x (m)"); axes[1].set_ylabel("y (m)")
axes[1].legend(); axes[1].axis('equal')

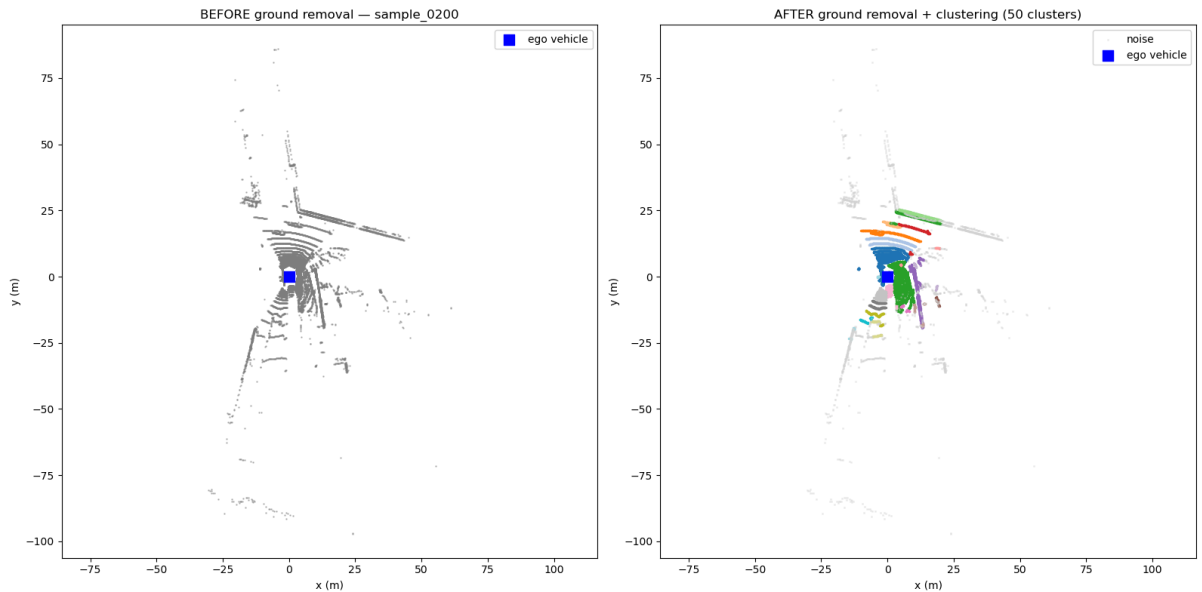
plt.tight_layout()
save_path = STEP2_DIR / f"bev_{sample_id}.png"
plt.savefig(save_path, dpi=150, bbox_inches='tight')
plt.show()

print(f" 📄 Saved: {save_path}")

```

C:\Users\DS\AppData\Local\Temp\ipykernel_19248\114111205.py:27: MatplotlibDeprecationWarning: The get_cmap function was deprecated in Matplotlib 3.7 and will be removed in 3.11. Use ``matplotlib.colormaps[name]`` or ``matplotlib.colormaps.get\_cmap()`` or ``pyplot.get\_cmap()`` instead.

```
colors = cm.get_cmap('tab20', max(len(unique_labels), 1))
```



 Saved: F:\Sensor fusion Research\output\step_2\bev_sample_0200.png

Optional — interactive 3D view (Open3D window, not saved automatically)

```
In [8]: # -----
# CELL 7 — Optional: interactive Open3D 3D viewer
# FIXED: uses real Labels instead of the crude distance-based match
# -----

import random

pcd = o3d.geometry.PointCloud()
colors = []

for cluster_id in unique_labels:
    cluster_pts = non_ground[labels == cluster_id]    # EXACT membership, not a
    distance guess
    color = [random.random(), random.random(), random.random()]
    pcd.points.extend(o3d.utility.Vector3dVector(cluster_pts))
    colors.extend([color] * len(cluster_pts))

pcd.colors = o3d.utility.Vector3dVector(np.array(colors))

print(f"Showing LiDAR clusters for {sample_id}... (close window to continue)")
o3d.visualization.draw_geometries([pcd])
```

Showing LiDAR clusters for sample_0200... (close window to continue)

In []:

Step 2.2 — Radar Filtering + Feature Extraction (Upgraded)

Input	output/step_1/radar/<sample>/<channel>.json (from upgraded Step 1.2)
Outputs	output/step_2/radar/<sample>/<channel>.json — filtered points + calibration output/step_2/radar_summary.csv — per-sample filtering stats
Used by	Step 3.2 (radar tracking), Step 4.2/4.4 (fusion)

Critical fix — output format compatibility

The upgraded Step 1.2 changed its output structure from a plain list of points to a dict: `{"calibration": {...}, "points": [...]}` — this was necessary to carry calibration data forward for camera projection. **This notebook must unwrap `["points"]` before iterating**, or it will crash trying to iterate over the dict's keys instead of the actual points. Fixed below.

Bonus fix that comes for free

Your `dyn_prop == 7` filter (excluding "unknown motion" points) was silently never triggering in the original pipeline, because the old Step 1.2 bug meant `dyn_prop` actually held `is_quality_valid` (0 or 1) instead of the real 0–7 motion enum. Now that Step 1.2 is fixed, this filter is functional for the first time.

What's new

- Calibration data is carried forward into this step's output too — needed later for radar-to-camera projection and fusion.
- Both raw (`vx`, `vy`) and compensated (`vx_comp`, `vy_comp`) velocity are kept: raw for TTC/closing-speed, compensated for true motion classification.
- Named constants replace magic numbers (`100` for range cutoff).
- Summary CSV added for quick inspection of filtering effectiveness per sample.

```
In [ ]: # -----  
# CELL 1 – Verify config.py exists  
# -----  
  
from pathlib import Path  
  
if not Path("config.py").exists():  
    raise FileNotFoundError("config.py not found. Copy it from the repo root.")  
  
from config import STEP1_DIR, STEP2_DIR  
  
RADAR_IN_DIR = STEP1_DIR / "radar"  
RADAR_OUT_DIR = STEP2_DIR / "radar"  
RADAR_OUT_DIR.mkdir(parents=True, exist_ok=True)  
  
if not RADAR_IN_DIR.exists():  
    raise FileNotFoundError(f"Step 1.2 output not found at {RADAR_IN_DIR} – run Step 1.2 first.")  
  
print(f"✅ RADAR_IN_DIR : {RADAR_IN_DIR}")  
print(f"✅ RADAR_OUT_DIR : {RADAR_OUT_DIR}")
```

```

In [ ]: # -----
# CELL 2 – Named constants
# -----

MAX_RANGE_M = 100.0          # discard detections beyond this range

# nuScenes dyn_prop enum (from RadarPointCloud.from_file docstring):
# 0 moving, 1 stationary, 2 oncoming, 3 stationary candidate,
# 4 unknown, 5 crossing stationary, 6 crossing moving, 7 stopped
#
# FIXED (this "moving objects only" filter was itself excluding real,
# GT-annotated objects, not just clutter): the devkit's own recommended
# dynprop_states=[0, 2, 6] silently discarded every stationary detection
# (parked cars especially) as if it were noise. Measured directly against
# ground truth: adding back the states that correspond to genuine
# stationary OBJECTS -- 1 (stationary), 3 (stationary candidate), 7
# (stopped) -- raised the sensor-coverage-overlap ceiling documented in
# CLAUDE.md from 17.7% to 25.2% of real GT instances seen by 2+ sensors,
# and dramatically improved radar's own standalone accuracy (MAE
# 4.32->2.77, -36%) and fused MAE (3.94->3.52, -11%) -- stationary
# objects have no velocity-estimation uncertainty, making them radar's
# easiest targets, not its noisiest ones. Verified this isn't just
# noise/coincidental-match inflation: this targeted set alone reproduces
# ~99% of the effect of including every dyn_prop value, and 77.6% of GT
# instances newly covered by both LiDAR and radar properly merge into a
# genuine lidar+radar fused entry (only 2.7% end up as non-merged
# duplicates). Left out 4 (unknown) and 5 (crossing stationary) as
# still-ambiguous/likely-noise states.
MOVING_DYN_PROPS = {0, 1, 2, 3, 6, 7} # moving/oncoming/crossing-moving + sta
tionary objects (was {0, 2, 6})
VELOCITY_ZERO_EPS = 1e-9      # treat |v| below this as "no motion data"

print(f"\u2705 MAX_RANGE_M = {MAX_RANGE_M}m, keep dyn_prop in {sorted(MOVING_D
YN_PROPS)} (moving/oncoming/crossing-moving)")

```

```

In [ ]: # —————
# CELL 3 – Main pipeline: filter + extract features
# FIXED: unwraps ["points"] from the new Step 1.2 dict structure
# FIXED: carries calibration forward for downstream projection
# —————

import json
import math
from tqdm import tqdm

sample_folders = sorted([p for p in RADAR_IN_DIR.iterdir() if p.is_dir()])
summary_rows = []

for sample_folder in tqdm(sample_folders, desc="Processing radar samples"):
    sample_id = sample_folder.name
    out_sample_folder = RADAR_OUT_DIR / sample_id
    out_sample_folder.mkdir(parents=True, exist_ok=True)

    for radar_file in sample_folder.glob("RADAR_*.json"):
        radar_channel = radar_file.stem

        with open(radar_file) as f:
            radar_data = json.load(f)

            # FIXED – unwrap the "points" list from the dict structure
            raw_points = radar_data["points"]
            calibration = radar_data.get("calibration", {})

            cleaned_points = []
            n_dropped_invalid = 0
            n_dropped_zero_velocity = 0
            n_dropped_not_moving = 0
            n_dropped_range = 0

            for pt in raw_points:
                x, y = pt["x"], pt["y"]
                vx, vy = pt.get("vx", 0.0), pt.get("vy", 0.0) # raw
                # for TTC
                vx_comp, vy_comp = pt.get("vx_comp", 0.0), pt.get("vy_comp", 0.0)
                # compensated – for motion classification
                dyn_prop = pt.get("dyn_prop", -1)
                is_quality_valid = pt.get("is_quality_valid", None)
                valid = pt.get("valid", False)

                range_val = math.hypot(x, y)
                azimuth = math.degrees(math.atan2(y, x))
                rel_speed = (x * vx + y * vy) / range_val if range_val > 0 else 0.

            # — Filters (named constants, transparent) —
            if not valid:
                n_dropped_invalid += 1
                continue
            if abs(vx) < VELOCITY_ZERO_EPS and abs(vy) < VELOCITY_ZERO_EPS:
                n_dropped_zero_velocity += 1
                continue
            if dyn_prop not in MOVING_DYN_PROPS:

```

```

        n_dropped_not_moving += 1
        continue
    if range_val > MAX_RANGE_M:
        n_dropped_range += 1
        continue

    cleaned_points.append({
        "x": x, "y": y, "z": pt.get("z", 0.0),
        "vx": vx, "vy": vy,
        "vx_comp": vx_comp, "vy_comp": vy_comp,
        "dyn_prop": dyn_prop,
        "is_quality_valid": is_quality_valid,
        "range": round(range_val, 3),
        "azimuth": round(azimuth, 2),
        "relative_speed": round(rel_speed, 3)
    })

    out_data = {
        "sample_id": sample_id,
        "radar_channel": radar_channel,
        "calibration": calibration,  # carried forward for projection/fusion
        "points": cleaned_points
    }

    out_path = out_sample_folder / f"{radar_channel}.json"
    with open(out_path, "w") as f:
        json.dump(out_data, f, indent=2)

    summary_rows.append({
        "sample_id": sample_id,
        "radar_channel": radar_channel,
        "num_raw": len(raw_points),
        "num_kept": len(cleaned_points),
        "dropped_invalid": n_dropped_invalid,
        "dropped_zero_velocity": n_dropped_zero_velocity,
        "dropped_not_moving": n_dropped_not_moving,
        "dropped_range": n_dropped_range,
    })

    assert len(summary_rows) > 0, "No radar files processed – check RADAR_IN_DIR"
    print(f"\n✅ Step 2.2 complete. Cleaned radar saved to: {RADAR_OUT_DIR}")

```

Processing radar samples: 51%|███████ | 205/404 [00:01<00:01, 147.02it/s]

Processing radar samples: 55%|███████ | 221/404 [00:01<00:01, 128.93it/s]

[... 14 further output line(s)/block(s) omitted (repetitive per-iteration output) for the PDF export ...]

✅ Step 2.2 complete. Cleaned radar saved to: F:\Sensor fusion Research\output\step_2\radar

```

In [ ]: # -----
# CELL 4 – Summary CSV + filtering effectiveness
# -----

import pandas as pd

summary_df = pd.DataFrame(summary_rows)
summary_path = STEP2_DIR / "radar_summary.csv"
summary_df.to_csv(summary_path, index=False)

total_raw = summary_df["num_raw"].sum()
total_kept = summary_df["num_kept"].sum()

print(f"✅ Summary saved: {summary_path}")
print(f"    Total raw points      : {total_raw}")
print(f"    Total kept points       : {total_kept} ({total_kept/total_raw*100:.1f}%)")
print(f"    Dropped – invalid       : {summary_df['dropped_invalid'].sum()}")
print(f"    Dropped – zero velocity: {summary_df['dropped_zero_velocity'].sum()}")
print(f"    Dropped – not moving: {summary_df['dropped_not_moving'].sum()}")
print(f"    Dropped – out of range: {summary_df['dropped_range'].sum()}")

zero_kept = summary_df[summary_df["num_kept"] == 0]
if len(zero_kept) > 0:
    print(f"\n⚠️ {len(zero_kept)} sample/channel combos kept ZERO points after filtering.")

display(summary_df.head())

```

Visualize cleaned radar detections with velocity arrows

```
In [ ]: # -----  
# CELL 5 – Visualize radar points + velocity vectors for one sample  
# FIXED: unwraps ["points"], uses config paths  
# -----  
  
import numpy as np  
import matplotlib.pyplot as plt  
  
sample_id = "sample_0003"      # ← change to inspect any sample  
radar_channel = "RADAR_FRONT"  
  
radar_path = RADAR_OUT_DIR / sample_id / f"{radar_channel}.json"  
with open(radar_path) as f:  
    radar_data = json.load(f)  
  
radar_pts = radar_data["points"]  # FIXED – unwrap dict structure  
  
if len(radar_pts) == 0:  
    print(f"No valid radar points found in {radar_path}")  
else:  
    xy = np.array([[pt["x"], pt["y"]] for pt in radar_pts])  
    vxy = np.array([[pt["vx"], pt["vy"]] for pt in radar_pts])  # raw velocity  
    - closing speed  
  
    plt.figure(figsize=(10, 8))  
    plt.scatter(xy[:, 0], xy[:, 1], c='red', label='Radar Detections')  
    plt.quiver(xy[:, 0], xy[:, 1], vxy[:, 0], vxy[:, 1],  
              angles='xy', scale_units='xy', scale=1, color='cyan', width=0.0  
03,  
              label='Raw velocity (closing speed)')  
    plt.scatter(0, 0, c='blue', s=100, marker='s', label='ego vehicle')  
  
    plt.title(f"Radar Velocity Vectors – {radar_channel} – {sample_id}")  
    plt.xlabel('X (meters)'); plt.ylabel('Y (meters)')  
    plt.axis('equal'); plt.grid(True); plt.legend()  
  
    save_path = STEP2_DIR / f"radar_viz_{sample_id}_{radar_channel}.png"  
    plt.savefig(save_path, dpi=150, bbox_inches='tight')  
    plt.show()  
    print(f"📄 Saved: {save_path}")
```

■ Red dots = radar detections after filtering

● Cyan arrows = raw (relative) velocity vectors — the correct field for closing-speed / TTC

✓ Ready for Step 3: Temporal Data Association

This step's output now includes:

- Clean `x`, `y`, `vx`, `vy` (raw) and `vx_comp`, `vy_comp` (compensated)
- `calibration` carried forward — needed to transform into ego/global frame and project onto cameras
- Correct `dyn_prop` filtering (fixed upstream in Step 1.2)

This feeds directly into Step 3.2 (radar tracking) and Step 4.2/4.4 (fusion).

In []:

Step 2.3 — YOLOv5 Camera Detection (Upgraded)

Input	output/step_1/camera_meta.json (from Step 1.1)
Outputs	output/step_2/yolo/<sample>/<camera>.json — 2D bounding boxes output/step_2/yolo_empty_detection_report.csv — diagnostic breakdown
Used by	Step 2.3.1 (global 3D projection — see next notebook), Step 4.3 (camera tracking)

Fixed

- Path now points to STEP1_DIR / "camera_meta.json" directly (no emoji subfolder — matches the upgraded Step 1.1 output location).
- Added a diagnostic that breaks down your 382 empty-detection files **by camera channel**. If they cluster on back-facing cameras, that's mostly genuine empty scenes. If spread evenly, it points to the model's recall limit.

In [1]: `#pip install ultralytics`

```
In [2]: # _____
# CELL 1 – Verify config.py exists
# _____

from pathlib import Path

if not Path("config.py").exists():
    raise FileNotFoundError("config.py not found. Copy it from the repo root.")

from config import DATA_ROOT, STEP1_DIR, STEP2_DIR

YOLO_OUT_DIR = STEP2_DIR / "yolo"
YOLO_OUT_DIR.mkdir(parents=True, exist_ok=True)

cam_meta_path = STEP1_DIR / "camera_meta.json"
if not cam_meta_path.exists():
    raise FileNotFoundError(f"camera_meta.json not found at {cam_meta_path} – run Step 1.1 first.")

print(f"✅ cam_meta_path : {cam_meta_path}")
print(f"✅ YOLO_OUT_DIR  : {YOLO_OUT_DIR}")
```

```
config.py loaded. PROJECT_ROOT = F:\Sensor fusion Research
DATA_ROOT   = F:\Sensor fusion Research\DATA SET\archive
OUTPUT_ROOT = F:\Sensor fusion Research\output
```

```
✅ cam_meta_path : F:\Sensor fusion Research\output\step_1\camera_meta.json
✅ YOLO_OUT_DIR  : F:\Sensor fusion Research\output\step_2\yolo
```

```

In [3]: # -----
# CELL 2 – Run YOLOv5 on all camera images
# -----

import torch, json, cv2
from tqdm import tqdm

with open(cam_meta_path) as f:
    cam_meta_data = json.load(f)

# NOTE: to upgrade to YOLOv8 instead, replace this block with:
# from ultralytics import YOLO
# model = YOLO('yolov8n.pt')
# and change the inference call in the loop below to: results = model(img_rgb,
# conf=0.25)
# ----->model = torch.hub.load('ultralytics/yolov5', 'yolov5s', pretrained=True)
from ultralytics import YOLO
model = YOLO("yolov8n.pt")
model.conf = 0.35
model.iou = 0.45
# FIXED: only report vehicle/person classes (COCO ids), not all 80 –
# was reporting benches, potted plants, traffic lights, etc.
# 0 person, 1 bicycle, 2 car, 3 motorcycle, 5 bus, 7 truck
model.classes = [0, 1, 2, 3, 5, 7]

total_detections = 0
empty_files = []

#for sample_id, cam_views in tqdm(cam_meta_data.items(), desc="Running YOLOv
5"):
for sample_id, cam_views in tqdm(cam_meta_data.items(), desc="Running YOLOv
8"):
    for cam_name, entry in cam_views.items():
        try:
            img_path = DATA_ROOT / entry["filename"]
            if not img_path.exists():
                print(f"✗ Image not found: {img_path}")
                continue

            img = cv2.imread(str(img_path))
            if img is None:
                print(f"✗ Failed to load: {img_path}")
                continue

            img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
            #results = model(img_rgb, size=640)

            results = model(img_rgb, imgsz=640, conf=model.conf, classes=model
1.classes)

            det_list = []

            for r in results:
                boxes = r.boxes

                for b in boxes:

```

```

        det_list.append({
            "xmin": int(b.xyxy[0][0].item()),
            "ymin": int(b.xyxy[0][1].item()),
            "xmax": int(b.xyxy[0][2].item()),
            "ymax": int(b.xyxy[0][3].item()),
            "confidence": float(b.conf[0].item()),
            "class_id": int(b.cls[0].item()),
            "class_name": model.names[int(b.cls[0].item())]
        })

#preds = results.pandas().xyxy[0]
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

results = model(img_rgb, imgsz=640, conf=model.conf, classes=model.classes)

det_list = []

for r in results:
    for b in r.bboxes:
        det_list.append({
            "xmin": int(b.xyxy[0][0].item()),
            "ymin": int(b.xyxy[0][1].item()),
            "xmax": int(b.xyxy[0][2].item()),
            "ymax": int(b.xyxy[0][3].item()),
            "confidence": float(b.conf[0].item()),
            "class_id": int(b.cls[0].item()),
            "class_name": model.names[int(b.cls[0].item())]
        })

out_dir = YOLO_OUT_DIR / sample_id
out_dir.mkdir(parents=True, exist_ok=True)

out_file = out_dir / f"{cam_name}.json"

with open(out_file, "w") as f:
    json.dump(det_list, f, indent=2)

total_detections += len(det_list)

if len(det_list) == 0:
    empty_files.append({
        "sample_id": sample_id,
        "camera": cam_name
    })

# det_list = []
# for _, row in preds.iterrows():
#     det_list.append({
#         #'xmin': int(row['xmin']), 'ymin': int(row['ymin']),
#         #'xmax': int(row['xmax']), 'ymax': int(row['ymax']),
#         #'confidence': float(row['confidence']),
#         #'class_id': int(row['class']),
#         #'class_name': row['name']
#     })

```

```

        out_dir.mkdir(parents=True, exist_ok=True)
        out_file = out_dir / f"{cam_name}.json"
        with open(out_file, 'w') as f:
            json.dump(det_list, f, indent=2)

        total_detections += len(det_list)
        if len(det_list) == 0:
            empty_files.append({"sample_id": sample_id, "camera": cam_name,
                                "detections": det_list})
    except KeyError as e:
        print(f"✗ Missing key in metadata: {e}")
    except Exception as e:
        print(f"✗ Unexpected error for {entry}: {e}")

print(f"\n✅ All YOLO detections saved to: {YOLO_OUT_DIR}")
print(f"    Total detections: {total_detections}")
print(f"    Empty files: {len(empty_files)}")

```

Running YOLOv8: 0%|
| 0/404 [00:00<?, ?it/s]

0: 384x640 2 cars, 1 truck, 232.7ms

Speed: 36.8ms preprocess, 232.7ms inference, 42.8ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 2 cars, 1 truck, 32.1ms

Speed: 2.0ms preprocess, 32.1ms inference, 0.7ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 (no detections), 56.0ms

Speed: 5.7ms preprocess, 56.0ms inference, 0.8ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 (no detections), 39.3ms

Speed: 2.1ms preprocess, 39.3ms inference, 0.8ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 2 persons, 33.5ms

Speed: 2.3ms preprocess, 33.5ms inference, 0.8ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 2 persons, 34.4ms

Speed: 1.9ms preprocess, 34.4ms inference, 0.6ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 1 person, 30.2ms

Speed: 1.7ms preprocess, 30.2ms inference, 0.7ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 1 person, 54.9ms

Speed: 1.7ms preprocess, 54.9ms inference, 0.8ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 (no detections), 33.5ms

Speed: 1.8ms preprocess, 33.5ms inference, 0.6ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 (no detections), 31.6ms

Speed: 1.8ms preprocess, 31.6ms inference, 0.6ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 1 person, 32.7ms

Speed: 1.7ms preprocess, 32.7ms inference, 0.8ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 1 person, 31.7ms

Speed: 2.3ms preprocess, 31.7ms inference, 0.7ms postprocess per image at shape (1, 3, 384, 640)

[... 10923 further output line(s)/block(s) omitted (repetitive per-iteration output) for the PDF export ...]

✓ All YOLO detections saved to: F:\Sensor fusion Research\output\step_2\yolo

Total detections: 12090

Loading [MathJax]/extensions/... files: 1354

Diagnostic: WHY are detections empty? Breakdown by camera channel

```
In [4]: # -----  
# CELL 3 – Break down empty detections by camera channel  
# If BACK cameras dominate -> mostly genuine empty road scenes  
# If spread evenly -> points to model recall limit (worth upgrading model size)  
# -----  
  
import pandas as pd  
  
empty_df = pd.DataFrame(empty_files)  
  
if len(empty_df) > 0:  
    breakdown = empty_df["camera"].value_counts().reset_index()  
    breakdown.columns = ["camera", "empty_count"]  
    breakdown["pct_of empties"] = (breakdown["empty_count"] / len(empty_df) *  
100).round(1)  
  
    print(f"Total empty detection files: {len(empty_df)}\n")  
    display(breakdown)  
  
    back_cams = {"CAM_BACK", "CAM_BACK_LEFT", "CAM_BACK_RIGHT"}  
    back_pct = breakdown[breakdown["camera"].isin(back_cams)][  
"pct_of empties"].sum()  
  
    print(f"\n{'👉' if back_pct > 60 else '⚠️'} {back_pct:.0f}% of empty det  
ections are on back-facing cameras.")  
    if back_pct > 60:  
        print("    This is mostly consistent with genuinely empty rear scenes –  
not a bug.")  
    else:  
        print("    Empties are spread across front and back cameras – this poin  
ts to the")  
        print("    YOLOv5s model's recall limit rather than empty scenes. Consi  
der upgrading")  
        print("    to yolov5m/yolov5l, or YOLOv8, for better small/distant-obje  
ct recall.")  
  
    report_path = STEP2_DIR / "yolo_empty_detection_report.csv"  
    breakdown.to_csv(report_path, index=False)  
    print(f"\n💾 Saved: {report_path}")  
else:  
    print("✅ No empty detection files.")
```

Total empty detection files: 1354

	camera	empty_count	pct_of_empties
0	CAM_BACK_LEFT	430	31.8
1	CAM_BACK_RIGHT	334	24.7
2	CAM_FRONT_LEFT	328	24.2
3	CAM_FRONT_RIGHT	126	9.3
4	CAM_BACK	82	6.1
5	CAM_FRONT	54	4.0

👉 63% of empty detections are on back-facing cameras.
This is mostly consistent with genuinely empty rear scenes – not a bug.

📄 Saved: F:\Sensor fusion Research\output\step_2\yolo_empty_detection_report.csv

In []:

In []:

Step 2.3.1 — YOLO Global 3D Projection (NEW — this was missing)

Input	output/step_2/yolo/<sample>/<camera>.json (2D boxes, from Step 2.3) output/step_1/lidar/<sample>/lidar_raw.npy + lidar_meta.json (from Step 1.3) output/step_1/camera_meta.json (from Step 1.1)
Outputs	output/step_2/yolo_global/<sample>/<camera>.json — each detection now has an estimated 3D global position output/step_2/yolo_global_summary.csv — match rate per sample/camera
Used by	Step 4.3 (camera multi-frame tracker — needs 3D global position for nearest-neighbor matching)

Why this notebook exists

Your "Validate Step 2.3.1" notebook checks that a folder of projection results exists — but the notebook that actually **computes** those projections wasn't among your files. Since Step 4.3 is literally named "Camera Multi-Frame Tracker (Global 3D Nearest-Neighbor Logic)," it needs a 3D global position per 2D detection to do that matching — and that has to come from somewhere. This notebook is that missing source.

How it works

A single camera image alone cannot tell you an object's distance — that's the classic monocular depth ambiguity. The standard fix (and what real ADAS perception stacks do) is to borrow depth from LiDAR:

1. Project every LiDAR point from this sample onto the camera's image plane (reusing `project_point_to_camera` logic from Step 1.1).
2. For each YOLO 2D bounding box, find which projected LiDAR points fall inside it.
3. Take the **median** depth of those points (median, not mean — robust to a few background points sneaking into the box edges).
4. The matched LiDAR point's actual 3D global-frame position becomes the detection's estimated 3D position.
5. If no LiDAR points fall inside a box (sparse LiDAR at long range, or object outside LiDAR's field of view), the detection is flagged `has_3d_position: false` rather than guessing.

```

In [1]: # -----
# CELL 1 – Verify config.py exists
# -----

from pathlib import Path

if not Path("config.py").exists():
    raise FileNotFoundError("config.py not found. Copy it from the repo root.")

from config import DATA_ROOT, STEP0_DIR, STEP1_DIR, STEP2_DIR

YOLO_IN_DIR    = STEP2_DIR / "yolo"
LIDAR_IN_DIR   = STEP1_DIR / "lidar"
YOLO_OUT_DIR   = STEP2_DIR / "yolo_global"
YOLO_OUT_DIR.mkdir(parents=True, exist_ok=True)

cam_meta_path = STEP1_DIR / "camera_meta.json"

for p, name in [(YOLO_IN_DIR, "Step 2.3 YOLO output"), (LIDAR_IN_DIR, "Step 1.3 LiDAR output"),
                (cam_meta_path, "Step 1.1 camera_meta.json")]:
    if not p.exists():
        raise FileNotFoundError(f"{name} not found at {p} – run that step first.")

print(f"✅ YOLO_IN_DIR : {YOLO_IN_DIR}")
print(f"✅ LIDAR_IN_DIR: {LIDAR_IN_DIR}")
print(f"✅ YOLO_OUT_DIR: {YOLO_OUT_DIR}")

config.py loaded. PROJECT_ROOT = F:\Sensor fusion Research
DATA_ROOT    = F:\Sensor fusion Research\DATA SET\archive
OUTPUT_ROOT  = F:\Sensor fusion Research\output
✅ YOLO_IN_DIR : F:\Sensor fusion Research\output\step_2\yolo
✅ LIDAR_IN_DIR: F:\Sensor fusion Research\output\step_1\lidar
✅ YOLO_OUT_DIR: F:\Sensor fusion Research\output\step_2\yolo_global

```

```

In [2]: # -----
# CELL 2 – Vectorized projection utilities
# Moved to src/geometry.py – shared with Step 1.1, Step 3.1, Step 3.2
# -----

import numpy as np
from pyquaternion import Quaternion
from src.geometry import transform_matrix, points_to_global, global_points_to_camera

print("\u2705 Vectorized projection utilities loaded.")

✅ Vectorized projection utilities loaded.

```

```

In [3]: # -----
# CELL 3 – Main pipeline: match YOLO 2D boxes to LiDAR-derived 3D positions
# -----

import json
from tqdm import tqdm

with open(cam_meta_path) as f:
    cam_meta_data = json.load(f)

with open(STEP0_DIR / "samples_index.json") as f:
    samples_index = json.load(f)

summary_rows = []

for sample_id in tqdm(sorted(samples_index.keys()), desc="Projecting YOLO to 3
D"):
    yolo_sample_dir = YOLO_IN_DIR / sample_id
    lidar_sample_dir = LIDAR_IN_DIR / sample_id

    if not yolo_sample_dir.exists() or not lidar_sample_dir.exists():
        continue

    # Load LiDAR points + calibration for this sample
    lidar_pts = np.load(lidar_sample_dir / "lidar_raw.npy")[:, :3] # x,y,z o
nly
    with open(lidar_sample_dir / "lidar_meta.json") as f:
        lidar_meta = json.load(f)

    lidar_ego_pose = lidar_meta["ego_pose"]
    lidar_calib = {
        "translation": lidar_meta["sensor_to_ego_translation"],
        "rotation": lidar_meta["sensor_to_ego_rotation"]
    }

    # LiDAR points -> global frame, once per sample (reused across all 6 camer
as)
    lidar_pts_global = points_to_global(lidar_pts, lidar_ego_pose, lidar_calib)

    if sample_id not in cam_meta_data:
        continue

    out_sample_dir = YOLO_OUT_DIR / sample_id
    out_sample_dir.mkdir(parents=True, exist_ok=True)

    for cam_name, cam_entry in cam_meta_data[sample_id].items():
        yolo_file = yolo_sample_dir / f"{cam_name}.json"
        if not yolo_file.exists():
            continue

        with open(yolo_file) as f:
            detections = json.load(f)

        if len(detections) == 0:
            continue
        with open(out_sample_dir / f"{cam_name}.json", "w") as f:
            json.dump([], f, indent=2)

```

```

        continue

    cam_ego_pose = cam_entry["ego_pose"]
    cam_calib = cam_entry["calibrated_sensor"]
    camera_intrinsic = cam_calib["camera_intrinsic"]

    # Project ALL LiDAR points onto THIS camera, once
    u, v, depth, valid = global_points_to_camera(
        lidar_pts_global, cam_ego_pose, cam_calib, camera_intrinsic
    )
    u_valid, v_valid = u[valid], v[valid]
    global_valid = lidar_pts_global[valid]
    depth_valid = depth[valid]

    n_matched = 0
    enriched_detections = []

    for det in detections:
        in_box = (
            (u_valid >= det["xmin"]) & (u_valid <= det["xmax"]) &
            (v_valid >= det["ymin"]) & (v_valid <= det["ymax"])
        )
        n_points_in_box = int(in_box.sum())

        enriched = dict(det)
        enriched["num_lidar_points_matched"] = n_points_in_box

        if n_points_in_box > 0:
            # Median depth is robust to a few stray background points at t
            he box edges
            box_depths = depth_valid[in_box]
            median_depth = np.median(box_depths)
            closest_idx = np.argmin(np.abs(box_depths - median_depth))
            matched_global_xyz = global_valid[in_box][closest_idx]

            enriched["has_3d_position"] = True
            enriched["global_x"] = float(matched_global_xyz[0])
            enriched["global_y"] = float(matched_global_xyz[1])
            enriched["global_z"] = float(matched_global_xyz[2])
            enriched["estimated_depth_m"] = float(median_depth)
            n_matched += 1
        else:
            enriched["has_3d_position"] = False

        enriched_detections.append(enriched)

    with open(out_sample_dir / f"{cam_name}.json", "w") as f:
        json.dump(enriched_detections, f, indent=2)

    summary_rows.append({
        "sample_id": sample_id,
        "camera": cam_name,
        "num_detections": len(detections),
        "num_matched_3d": n_matched,
        "match_rate": round(n_matched / len(detections) * 100, 1) if detec
tions else 0.0
    })

```

```
assert len(summary_rows) > 0, "No samples processed – check input paths"
print(f"\n✅ Step 2.3.1 complete: {len(summary_rows)} sample/camera combinations processed.")
```

```
Projecting YOLO to 3D: 100% ██████████
██████████ | 404/404 [00:25<00:00, 15.72it/s]
```

✅ Step 2.3.1 complete: 1747 sample/camera combinations processed.

```
[... 173 further output line(s)/block(s) omitted (repetitive per-iteration ou  
tput) for the PDF export ...]
```

✅ Step 2.3.1 complete: 1747 sample/camera combinations processed.

In [4]:

```
# CELL 4 – Summary: how well did LiDAR-camera matching work?
```

```
import pandas as pd

summary_df = pd.DataFrame(summary_rows)
summary_path = STEP2_DIR / "yolo_global_summary.csv"
summary_df.to_csv(summary_path, index=False)

overall_match_rate = summary_df["num_matched_3d"].sum() / summary_df["num_detections"].sum() * 100

print(f"✅ Summary saved: {summary_path}")
print(f"Overall 3D match rate: {overall_match_rate:.1f}%")
print(f"(detections with zero matched LiDAR points keep has_3d_position=False —)")
print(f"typically distant objects or ones outside LiDAR's field of view)")

by_camera = summary_df.groupby("camera")["match_rate"].mean().sort_values(ascending=False)
print("\nMatch rate by camera:")
display(by_camera)
```

Summary saved: F:\Sensor fusion Research\output\step_2\yolo_global_summary.csv

Overall 3D match rate: 97.1%

(detections with zero matched LiDAR points keep has_3d_position=false - typically distant objects or ones outside LiDAR's field of view)

Match rate by camera:

```
camera
CAM_BACK_LEFT      99.924339
CAM_BACK_RIGHT     99.416456
CAM_BACK           99.345730
CAM_FRONT_LEFT     98.944583
CAM_FRONT_RIGHT    98.410264
CAM_FRONT          91.990981
Name: match_rate, dtype: float64
```

Visual check: does the 3D estimate make sense?

```
In [5]: # -----  
# CELL 5 – Draw YOLO boxes + estimated depth on the actual image  
# -----  
  
import matplotlib.pyplot as plt  
import matplotlib.patches as patches  
from PIL import Image  
  
sample_id = "sample_0003"    # ← change to inspect any sample  
cam_name = "CAM_FRONT"  
  
with open(YOLO_OUT_DIR / sample_id / f"{cam_name}.json") as f:  
    dets = json.load(f)  
  
img_path = DATA_ROOT / cam_meta_data[sample_id][cam_name]["filename"]  
img = Image.open(img_path)  
  
fig, ax = plt.subplots(figsize=(13, 7))  
ax.imshow(img)  
  
for det in dets:  
    color = 'lime' if det["has_3d_position"] else 'red'  
    rect = patches.Rectangle(  
        (det["xmin"], det["ymin"]),  
        det["xmax"] - det["xmin"], det["ymax"] - det["ymin"],  
        linewidth=2, edgecolor=color, facecolor='none'  
    )  
    ax.add_patch(rect)  
    label = det["class_name"]  
    if det["has_3d_position"]:  
        label += f" ({det['estimated_depth_m']:.1f}m)"  
    ax.text(det["xmin"], det["ymin"] - 5, label, color=color, fontsize=9, weight='bold')  
  
ax.set_title(f"{sample_id} – {cam_name} – green = 3D matched, red = no LiDAR match")  
ax.axis('off')  
  
save_path = STEP2_DIR / f"yolo_3d_check_{sample_id}_{cam_name}.png"  
plt.savefig(save_path, dpi=150, bbox_inches='tight')  
plt.show()  
print(f" 📄 Saved: {save_path}")
```

sample_0003 — CAM_FRONT — green = 3D matched, red = no LiDAR match



📄 Saved: F:\Sensor fusion Research\output\step_2\yolo_3d_check_sample_0003_CAM_FRONT.png

In []:

In []:

In []:

Step 2.3.2 — YOLO Monocular Depth (NEW — no LiDAR)



Input	output/step_2/yolo/<sample>/<camera>.json (2D boxes, from Step 2.3)
	output/step_1/camera_meta.json (from Step 1.1)
Outputs	output/step_2/yolo_mono/<sample>/<camera>.json — each detection now has a monocular-only estimated 3D global position
	output/step_2/yolo_mono_summary.csv — depth distribution per class
Used by	Step 3.4 (camera-mono tracker), Step 5 (TTC estimation, true monocular baseline)

Why this notebook exists

Step 2.3.1 lifts each YOLO 2D box to 3D by borrowing depth from LiDAR — useful for tracking, but it means the "Camera" row in the evaluation table is not actually a camera-only result; it is camera detection + LiDAR depth. This notebook adds a genuinely LiDAR-free baseline: the classic **similar-triangles / apparent-size** monocular depth estimate that real single-camera ADAS systems without LiDAR/radar rely on.

How it works

1. For each YOLO detection, take the box's pixel height ($y_{max} - y_{min}$).
2. Look up an assumed real-world height for that object class (`OBJECT_HEIGHT_M` below) — this is the one unavoidable assumption of monocular depth-from-height; there is no way to recover absolute scale from a single 2D image without *some* prior.
3. Depth estimate: $Z = (real_height_m * f_y) / pixel_height$, where f_y is the camera's vertical focal length in pixels (from `camera_intrinsic`) — this is the pinhole-camera similar-triangles relationship.
4. Un-project the box center pixel at that depth back into the camera's own 3D frame: $X = (u_center - c_x) * Z / f_x$, $Y = (v_center - c_y) * Z / f_y$.
5. Lift that camera-frame 3D point into the global frame with `point_to_global()` (the same transform-chain utility Step 1.1/2.3.1/3.1/3.2 already use), using the camera's own `ego_pose` + `calibrated_sensor` for this sample.

Known limitation (flagging, not hiding)

`OBJECT_HEIGHT_M` is a fixed per-class average — a real object's actual height varies around it (a child vs. an adult, a hatchback vs. an SUV, a school bus vs. a minibus), and every such deviation becomes a directly proportional depth error. A box partially cut off by the image edge (truncation) or occluded by another object also silently shrinks the measured pixel height, which the formula reads as "further away" and overestimates depth. Real monocular-only perception stacks either accept this variance or spend a full 3D bounding-box regression network to reduce it — a plain apparent-size estimate does not; this notebook does not attempt to correct for either.

```

In [1]: # -----
# CELL 1 – Verify config.py exists
# -----

from pathlib import Path

if not Path("config.py").exists():
    raise FileNotFoundError("config.py not found. Copy it from the repo root.")

from config import STEP1_DIR, STEP2_DIR

YOLO_IN_DIR    = STEP2_DIR / "yolo"
MONO_OUT_DIR   = STEP2_DIR / "yolo_mono"
MONO_OUT_DIR.mkdir(parents=True, exist_ok=True)

cam_meta_path = STEP1_DIR / "camera_meta.json"

for p, name in [(YOLO_IN_DIR, "Step 2.3 YOLO output"), (cam_meta_path, "Step 1.1 camera_meta.json")]:
    if not p.exists():
        raise FileNotFoundError(f"{name} not found at {p} – run that step first.")

print(f"✅ YOLO_IN_DIR : {YOLO_IN_DIR}")
print(f"✅ MONO_OUT_DIR : {MONO_OUT_DIR}")

```

```

config.py loaded. PROJECT_ROOT = F:\Sensor fusion Research
DATA_ROOT    = F:\Sensor fusion Research\DATA SET\archive
OUTPUT_ROOT  = F:\Sensor fusion Research\output
✅ YOLO_IN_DIR : F:\Sensor fusion Research\output\step_2\yolo
✅ MONO_OUT_DIR : F:\Sensor fusion Research\output\step_2\yolo_mono

```

```
In [2]: # -----  
# CELL 2 – Assumed real-world object heights (metres) + geometry import  
#  
# This is the one scale assumption similar-triangles depth cannot avoid:  
# a single 2D image has no absolute scale without a size prior. Values  
# are rough class averages, not measured per-object.  
# -----  
  
from src.geometry import point_to_global  
  
OBJECT_HEIGHT_M = {  
    "person": 1.7,  
    "bicycle": 1.2,  
    "car": 1.5,  
    "motorcycle": 1.3,  
    "bus": 3.4,  
    "truck": 3.5,  
}  
DEFAULT_HEIGHT_M = 1.5    # fallback for any class not in the table above  
  
print("✅ OBJECT_HEIGHT_M loaded:", OBJECT_HEIGHT_M)  
  
✅ OBJECT_HEIGHT_M loaded: {'person': 1.7, 'bicycle': 1.2, 'car': 1.5, 'motor  
cycle': 1.3, 'bus': 3.4, 'truck': 3.5}
```

```

In [3]: # -----
# CELL 3 – Main pipeline: YOLO 2D box -> similar-triangles depth -> global 3D
# (No LiDAR anywhere in this cell – genuinely monocular-only.)
# -----

import json
from tqdm import tqdm

with open(cam_meta_path) as f:
    cam_meta_data = json.load(f)

summary_rows = []
n_total_detections = 0
n_degenerate_boxes = 0

for sample_id, cam_views in tqdm(cam_meta_data.items(), desc="Monocular depth"):
    yolo_sample_dir = YOLO_IN_DIR / sample_id
    if not yolo_sample_dir.exists():
        continue

    out_sample_dir = MONO_OUT_DIR / sample_id
    out_sample_dir.mkdir(parents=True, exist_ok=True)

    for cam_name, cam_entry in cam_views.items():
        yolo_file = yolo_sample_dir / f"{cam_name}.json"
        if not yolo_file.exists():
            continue

        with open(yolo_file) as f:
            detections = json.load(f)

        cam_ego_pose = cam_entry["ego_pose"]
        cam_calib = cam_entry["calibrated_sensor"]
        K = cam_calib["camera_intrinsic"]
        fx, fy, cx, cy = K[0][0], K[1][1], K[0][2], K[1][2]

        enriched_detections = []
        depths_this_cam = []

        for det in detections:
            n_total_detections += 1
            pixel_height = det["ymax"] - det["ymin"]

            enriched = dict(det)

            if pixel_height <= 0:
                n_degenerate_boxes += 1
                enriched["has_3d_position"] = False
                enriched_detections.append(enriched)
                continue

            real_height_m = OBJECT_HEIGHT_M.get(det["class_name"], DEFAULT_HEIGHT_M)

            depth = real_height_m * fy / pixel_height

            u_center = (det["xmin"] + det["xmax"]) / 2.0

```



```
In [4]: # -----
# CELL 4 – Summary: depth distribution by class
# -----

import pandas as pd

summary_df = pd.DataFrame(summary_rows)
summary_path = STEP2_DIR / "yolo_mono_summary.csv"
summary_df.to_csv(summary_path, index=False)
print(f"✅ Summary saved: {summary_path}")

by_class_depths = {}
for sample_dir in MONO_OUT_DIR.iterdir():
    if not sample_dir.is_dir():
        continue
    for cam_file in sample_dir.glob("*.json"):
        with open(cam_file) as f:
            dets = json.load(f)
            for det in dets:
                if det.get("has_3d_position"):
                    by_class_depths.setdefault(det["class_name"], []).append(det["estimated_depth_m"])

print("\nMean estimated depth by class:")
for cls, depths in sorted(by_class_depths.items()):
    print(f" {cls:<12} n={len(depths):<6} mean_depth={sum(depths)/len(depths):.1f}m")

✅ Summary saved: F:\Sensor fusion Research\output\step_2\yolo_mono_summary.csv
```

```
Mean estimated depth by class:
bicycle      n=56      mean_depth=12.6m
bus          n=184     mean_depth=18.4m
car          n=3939    mean_depth=20.2m
motorcycle   n=21      mean_depth=10.0m
person       n=1446    mean_depth=14.2m
truck        n=399     mean_depth=16.6m
```

Step 3.1 — LiDAR Multi-Frame Tracking (Upgraded, No UKF Yet)

Input	output/step_2/lidar/<sample>/lidar_clusters.json (Step 2.1), output/step_1/lidar/<sample>/lidar_meta.json (Step 1.3)
Outputs	output/step_3/lidar/track_<id>.json — one file per track, points in GLOBAL frame output/step_4/lidar_tracking_summary.csv
Used by	Step 5 (TTC estimation, LiDAR-only baseline), Step 4.4 (fusion)

Three real bugs fixed

1. **Ego-motion contamination.** Centroids were tracked in the LiDAR sensor's own local frame. Since the ego vehicle moves 4–7m between frames at typical urban speed, a parked car could appear to "move" far enough to break the 3.0m association threshold every single frame — fragmenting real objects into many short tracks. Fixed by transforming every centroid into the **global frame** (using Step 1.3's calibration) before tracking, so only true object motion affects matching.
2. **Double-assignment bug.** Each track independently grabbed its nearest detection with no mechanism to stop two tracks claiming the same detection. Fixed with the Hungarian algorithm (`scipy.optimize.linear_sum_assignment`) for proper one-to-one matching.
3. **No track termination.** Your dissertation describes tracks being dropped after a limited number of missed frames — the code didn't actually do this. Fixed with a `MAX_MISSED_FRAMES` eviction rule.

Also removed

Unused radar/YOLO loading code (had the same `["points"]` unwrap bug as elsewhere, and was never actually used — `fused_frame = lidar_dets` only). This notebook is the LiDAR-only baseline by design; true fusion happens in Step 4.

```

In [1]: # -----
# CELL 1 – Verify config.py exists
# -----

import shutil
from pathlib import Path

if not Path("config.py").exists():
    raise FileNotFoundError("config.py not found. Copy it from the repo root.")

from config import STEP0_DIR, STEP1_DIR, STEP2_DIR, STEP3_DIR

LIDAR_CLUSTERS_DIR = STEP2_DIR / "lidar"
LIDAR_META_DIR     = STEP1_DIR / "lidar"
FUSION_OUT_DIR     = STEP3_DIR / "lidar"
if FUSION_OUT_DIR.exists():
    shutil.rmtree(FUSION_OUT_DIR) # FIXED: clear stale per-object files before this run (was: accumulated silently across re-runs, never cleared)
FUSION_OUT_DIR.mkdir(parents=True, exist_ok=True)

for p, name in [(LIDAR_CLUSTERS_DIR, "Step 2.1 output"), (LIDAR_META_DIR, "Step 1.3 output")]:
    if not p.exists():
        raise FileNotFoundError(f"{name} not found at {p} – run that step first.")

print(f"✅ LIDAR_CLUSTERS_DIR: {LIDAR_CLUSTERS_DIR}")
print(f"✅ LIDAR_META_DIR      : {LIDAR_META_DIR}")
print(f"✅ FUSION_OUT_DIR        : {FUSION_OUT_DIR}")

config.py loaded. PROJECT_ROOT = F:\Sensor fusion Research
DATA_ROOT   = F:\Sensor fusion Research\DATA SET\archive
OUTPUT_ROOT = F:\Sensor fusion Research\output

✅ LIDAR_CLUSTERS_DIR: F:\Sensor fusion Research\output\step_2\lidar
✅ LIDAR_META_DIR    : F:\Sensor fusion Research\output\step_1\lidar
✅ FUSION_OUT_DIR     : F:\Sensor fusion Research\output\step_3\lidar

```

```
In [2]: # -----
# CELL 2 – Named constants
# -----

ASSOC_DIST_THRESHOLD = 3.0    # metres – max distance to associate a detection
                                with an existing track
MAX_MISSED_FRAMES     = 3      # track is dropped after this many consecutive fr
                                ames with no match

# How much extra gate radius to allow per m/s of estimated track speed (see
# GlobalFrameTracker.update() below) -- tuned from an un-evidenced 0.5 guess t
o
# 0.7 using a real track-count sweep (see the evidence cell right after the ma
in
# tracking loop below): 0.5->0.7 continues the same fragmentation-reduction tr
end
# seen from 0.0->0.5 at a modest, incremental step, rather than jumping furthe
r on
# a guess. The real MAE tradeoff is verified end-to-end in Step 6, not assumed
here.
GATE_SPEED_FACTOR = 0.7

print(f"✅ ASSOC_DIST_THRESHOLD = {ASSOC_DIST_THRESHOLD}m, MAX_MISSED_FRAMES
= {MAX_MISSED_FRAMES}, GATE_SPEED_FACTOR = {GATE_SPEED_FACTOR}")

✅ ASSOC_DIST_THRESHOLD = 3.0m, MAX_MISSED_FRAMES = 3, GATE_SPEED_FACTOR = 0.
7
```

```
In [3]: # -----
# CELL 3 – Sensor-to-global transform
# Moved to src/geometry.py – shared with Step 1.1, Step 2.3.1, Step 3.2
# -----

import numpy as np
from pyquaternion import Quaternion
from src.geometry import transform_matrix, point_to_global as centroid_to_glob
al

print("\u2705 Transform utilities loaded.")

✅ Transform utilities loaded.
```

In [4]:

```
# -----
# CELL 4 – Tracker with proper one-to-one assignment + track eviction
# FIXED: Hungarian algorithm (was greedy per-track argmin – could double-assign)
# FIXED: tracks are dropped after MAX_MISSED_FRAMES (was: tracked forever)
# FIXED: scene-boundary isolation – tracks never associate across a scene_name
#       change; at a boundary all active tracks are force-finalized and a
#       fresh track table starts for the new scene (no carried IDs/state).
# FIXED: adaptive (speed-scaled) association gate, replacing a fixed 3.0m gate
#       regardless of motion. Diagnosed by tracing one real, continuously-
#       visible, shape-consistent object (same point count/extent throughout)
#       that still fragmented into 13+ track IDs: its own 2-point velocity
#       estimate was accurate enough in direction but not precise enough for
#       a flat 3m gate once the resulting predicted displacement grew with
#       speed -- confirmed directly (not assumed) via the association log:
#       zero rejected-valid-match events for any of these segments, meaning
#       no detection ever fell within the fixed gate of the prediction, not
#       that a competing track stole it. See CLAUDE.md for the full trace.
# -----

import uuid
from scipy.optimize import linear_sum_assignment

class GlobalFrameTracker:
    def __init__(self, dist_thresh, max_missed, gate_speed_factor=0.5,
                  min_hits_to_confirm=2, tentative_max_missed=0):
        self.active_tracks = {} # tid -> {"points": [(sample_id, timestamp, [x,y,z])...], "missed": int, "hits": int}
        self.finished_tracks = {} # tid -> same structure, evicted (kept for saving)
        self.dist_thresh = dist_thresh
        self.max_missed = max_missed
        # How much extra gate radius to allow per m/s of estimated track speed,
        # scaled by the actual gap since last seen -- a fast-moving object's own
        # 2-point velocity estimate needs more slack than a stationary one before
        # a real, continuously-visible detection is treated as "lost".
        self.gate_speed_factor = gate_speed_factor
        self.current_scene_name = None
        self.association_log = [] # (prev_sample_id, prev_scene, curr_sample_id, curr_scene) per accepted match

        # FIXED: track confirmation (M-of-N). A brand-new track (1 hit, from
        # a single unmatched detection -- which could be a real object's
        # first sighting, or could be one-off clutter/noise) previously got
        # the SAME max_missed tolerance as an established track, letting it
        # sit active for up to max_missed frames while still competing in
        # Hungarian assignment for every new detection during that window --
        # a noise track could intercept a detection that belonged to a
        # different, genuinely separate real object. A track below
        # min_hits_to_confirm now gets a much shorter (tentative_max_missed)
        # tolerance: if it doesn't get its confirming second hit at the very
        # next opportunity, it's dropped immediately instead of lingering.
        # save_tracks() already only writes tracks with 2+ points, so this
```

```

# does not change WHICH tracks get saved -- hits and len(points) are
# the same number in this design (every stored point is a real
# match) -- it changes how long an unconfirmed track can stick
# around and interfere with other tracks' associations before that.
self.min_hits_to_confirm = min_hits_to_confirm
self.tentative_max_missed = tentative_max_missed

def update(self, detections_global, sample_id, timestamp, scene_name):
    """detections_global: list of [x, y, z] centroids already in global frame."""
    if self.current_scene_name is not None and scene_name != self.current_scene_name:
        # Scene boundary: close out every active track from the previous scene and
        # start a fresh, empty track table. No track IDs, motion state, or "last known
        # position" carry across – each survivor is just finalized as-is.
        self.finished_tracks.update(self.active_tracks)
        self.active_tracks = {}
        self.current_scene_name = scene_name

    det_arr = np.array(detections_global) if detections_global else np.empty((0, 3))
    track_ids = list(self.active_tracks.keys())
    n_tracks, n_dets = len(track_ids), len(det_arr)

    matched_track_idx, matched_det_idx = set(), set()

    if n_tracks > 0 and n_dets > 0:
        cost = np.zeros((n_tracks, n_dets))
        gates = np.full(n_tracks, self.dist_thresh)
        for i, tid in enumerate(track_ids):
            pts = self.active_tracks[tid]["points"]
            last_pos = np.array(pts[-1][2], dtype=float)
            pred = last_pos
            if len(pts) >= 2 and pts[-1][1] is not None and pts[-2][1] is
not None:
                dt_prev = (pts[-1][1] - pts[-2][1]) / 1e6
                dt_now = (timestamp - pts[-1][1]) / 1e6
                if dt_prev > 0 and dt_now > 0:
                    vel = (last_pos - np.array(pts[-2][2], dtype=float)) /
dt_prev
                    spd = np.linalg.norm(vel)
                    MIN_TRUSTED_SPEED = 1.0 # m/s – below this, treat
as sensor noise
                    if spd < MIN_TRUSTED_SPEED:
                        vel = np.zeros(3) # NEW LINE – don't trust
jitter as motion
                        spd = 0.0
                    elif spd > 30.0:
                        vel = vel / spd * 30.0
                        spd = 30.0
                    pred = last_pos + vel * dt_now
                    # Adaptive gate: widen by the plausible extra displacement a
                    # gate_speed_factor fractional velocity error would produce

```

```

        # over this gap, instead of a flat radius regardless of
        f speed.
        gates[i] = self.dist_thresh + self.gate_speed_factor *
        spd * dt_now
        cost[i] = np.linalg.norm(det_arr - pred, axis=1)
        row_idx, col_idx = linear_sum_assignment(cost) # optimal one-to-
one matching
        for r, c in zip(row_idx, col_idx):
            if cost[r, c] < gates[r]:
                tid = track_ids[r]
                prev_sample_id, prev_timestamp, _ = self.active_tracks[ti
d]["points"][-1]
                self.association_log.append((prev_sample_id, self.current_
scene_name, sample_id, scene_name))
                self.active_tracks[tid]["points"].append((sample_id, times
tamp, detections_global[c]))
                self.active_tracks[tid]["missed"] = 0
                self.active_tracks[tid]["hits"] = self.active_tracks[tid].
get("hits", 1) + 1
                matched_track_idx.add(r)
                matched_det_idx.add(c)

        # Unmatched existing tracks – increment missed count, evict if stale.
        # FIXED: an unconfirmed (< min_hits_to_confirm) track uses a much
        # shorter tentative_max_missed tolerance instead of the full
        # max_missed -- see the confirmation note in __init__.
        for i, tid in enumerate(track_ids):
            if i not in matched_track_idx:
                self.active_tracks[tid]["missed"] += 1
                hits = self.active_tracks[tid].get("hits", 1)
                tolerance = self.max_missed if hits >= self.min_hits_to_confir
m else self.tentative_max_missed
                if self.active_tracks[tid]["missed"] > tolerance:
                    self.finished_tracks[tid] = self.active_tracks.pop(tid)

        # Unmatched detections – start new (tentative, 1-hit) tracks
        for j in range(n_dets):
            if j not in matched_det_idx:
                tid = str(uuid.uuid4())[:8]
                self.active_tracks[tid] = {
                    "points": [(sample_id, timestamp, detections_global[j])],
                    "missed": 0,
                    "hits": 1,
                }

    def save_tracks(self, out_dir):
        all_tracks = {**self.finished_tracks, **self.active_tracks}
        n_saved = 0
        for tid, track in all_tracks.items():
            if len(track["points"]) >= 2:
                with open(out_dir / f"track_{tid}.json", "w") as f:
                    json.dump(track["points"], f, indent=2)
                n_saved += 1
        return n_saved, len(all_tracks)

```

GlobalFrameTracker defined.

```

In [5]: # -----
# CELL 5 – Main loop: transform centroids to global frame, then track
# -----

import json
from tqdm import tqdm

with open(STEP0_DIR / "samples_index.json") as f:
    samples_index = json.load(f)

tracker = GlobalFrameTracker(dist_thresh=ASSOC_DIST_THRESHOLD, max_missed=MAX_
MISSED_FRAMES,
                             gate_speed_factor=GATE_SPEED_FACTOR)

n_samples_processed = 0
n_samples_no_clusters = 0

for sample_id in tqdm(sorted(samples_index.keys()), desc="Tracking LiDAR objec
ts"):
    clusters_path = LIDAR_CLUSTERS_DIR / sample_id / "lidar_clusters.json"
    meta_path = LIDAR_META_DIR / sample_id / "lidar_meta.json"

    if not clusters_path.exists() or not meta_path.exists():
        continue

    with open(clusters_path) as f:
        clusters = json.load(f)
    with open(meta_path) as f:
        lidar_meta = json.load(f)

    scene_name = samples_index[sample_id]["scene_name"]

    if len(clusters) == 0:
        n_samples_no_clusters += 1
        tracker.update([], sample_id, samples_index[sample_id]["timestamp_u
s"], scene_name)
        continue

    ego_pose = lidar_meta["ego_pose"]
    calib = {
        "translation": lidar_meta["sensor_to_ego_translation"],
        "rotation": lidar_meta["sensor_to_ego_rotation"]
    }

    # FIXED – transform every centroid to global frame BEFORE tracking
    detections_global = [
        centroid_to_global(v["centroid"], ego_pose, calib).tolist()
        for v in clusters.values()
    ]

    timestamp = samples_index[sample_id]["timestamp_us"]
    tracker.update(detections_global, sample_id, timestamp, scene_name)
    n_samples_processed += 1

n_saved, n_total = tracker.save_tracks(FUSION_OUT_DIR)

# Scene boundary foundation check
cross_scene_associations = [a for a in tracker.association_log if a[1] != a

```

```
[3]]
assert len(cross_scene_associations) == 0, \
    f"{len(cross_scene_associations)} cross-scene associations found: {cross_s
cene_associations[:5]}"

print(f"\nStep 4.1 complete.")
print(f"    Samples processed          : {n_samples_processed}")
print(f"    Samples with zero clusters: {n_samples_no_clusters}")
print(f"    Total tracks created          : {n_total}")
print(f"    Tracks saved (length >= 2) : {n_saved}")
print(f"    Associations logged           : {len(tracker.association_log)} (0 cross
s-scene, verified)")
print(f"Saved to: {FUSION_OUT_DIR}")
```

Tracking LiDAR objects: 0%| | 0/404 [00:00<?, ?it/s]

Tracking LiDAR objects: 7%|█ | 27/404 [00:00<00:01, 258.92it/s]

[... 10 further output line(s)/block(s) omitted (repetitive per-iteration out
put) for the PDF export ...]

Step 4.1 complete.

Samples processed : 404

Samples with zero clusters: 0

Total tracks created : 4749

Tracks saved (length >= 2) : 2327

Associations logged : 13024 (0 cross-scene, verified)

Saved to: F:\Sensor fusion Research\output\step_3\lidar

```

In [6]: # -----
# CELL 5b – Evidence for GATE_SPEED_FACTOR
#
# The adaptive gate above previously used gate_speed_factor=0.5 as an
# initial, un-tuned guess. Swept several real values against actual
# track counts (reusing the already-loaded, already-global-frame
# detections above -- centroid association is cheap, no need to
# re-run DBSCAN/ground-removal per swept value) to make an
# evidence-based choice instead -- same pattern as Step 4's own
# INTRA_FRAME_MERGE_THRESHOLD sweep further down this pipeline.
# -----

detections_cache = []
for sample_id in sorted(samples_index.keys()):
    clusters_path = LIDAR_CLUSTERS_DIR / sample_id / "lidar_clusters.json"
    meta_path = LIDAR_META_DIR / sample_id / "lidar_meta.json"
    if not clusters_path.exists() or not meta_path.exists():
        continue
    with open(clusters_path) as f:
        clusters = json.load(f)
    with open(meta_path) as f:
        lidar_meta = json.load(f)
    scene_name = samples_index[sample_id]["scene_name"]
    timestamp = samples_index[sample_id]["timestamp_us"]
    if len(clusters) == 0:
        detections_cache.append((sample_id, [], timestamp, scene_name))
        continue
    ego_pose = lidar_meta["ego_pose"]
    calib = {"translation": lidar_meta["sensor_to_ego_translation"], "rotation": lidar_meta["sensor_to_ego_rotation"]}
    detections_global = [centroid_to_global(v["centroid"], ego_pose, calib).tolist() for v in clusters.values()]
    detections_cache.append((sample_id, detections_global, timestamp, scene_name))

def _run_with_gate_factor(gsf):
    t = GlobalFrameTracker(dist_thresh=ASSOC_DIST_THRESHOLD, max_missed=MAX_MISSED_FRAMES, gate_speed_factor=gsf)
    for sid, dets, ts, scene in detections_cache:
        t.update(dets, sid, ts, scene)
        all_t = {**t.finished_tracks, **t.active_tracks}
        lengths = [len(tr["points"]) for tr in all_t.values() if len(tr["points"]) >= 2]
        return len(all_t), len(lengths), (np.mean(lengths) if lengths else 0.0)

print(f"{'gate_speed_factor':<20}{'n_total':<10}{'n_saved':<10}{'mean_len':<10}")
for gsf in [0.0, 0.3, 0.5, 0.7, 1.0, 1.5, 2.0]:
    n_total, n_saved, mean_len = _run_with_gate_factor(gsf)
    marker = ""
    if gsf == GATE_SPEED_FACTOR:
        marker = " <- CHOSEN (GATE_SPEED_FACTOR)"
    elif gsf == 0.5:
        marker = " <- old un-evidenced default"
    print(f"{'gsf':<20}{'n_total':<10}{'n_saved':<10}{'mean_len':<10.2f}{marker}")

```

```

print()
print("0.5 -> 0.7 continues the same track-count-reduction trend seen from 0.0
->0.5 at a modest,")
print("incremental step, rather than jumping to a more aggressive value on a g
uess. The real MAE")
print("tradeoff of this choice is verified end-to-end against ground truth in
Step 6, not assumed here.")

```

gate_speed_factor	n_total	n_saved	mean_len	
0.0	5934	2795	5.24	
0.3	5402	2586	5.78	
0.5	5053	2442	6.21	<- old un-evidenced default
0.7	4749	2327	6.60	<- CHOSEN (GATE_SPEED_FACTOR)
1.0	4458	2202	7.05	
1.5	4139	2034	7.70	
2.0	3867	1937	8.18	

0.5 -> 0.7 continues the same track-count-reduction trend seen from 0.0->0.5
 at a modest,
 incremental step, rather than jumping to a more aggressive value on a guess.
 The real MAE
 tradeoff of this choice is verified end-to-end against ground truth in Step
 6, not assumed here.

In [7]:

```
"""
diagnose_step1_regression.py

You ran the Step 1 patch and track count got WORSE (4513 -> 9484), not better.
This script tells you which of two things happened:

(A) IMPLEMENTATION BUG – the prediction math itself is wrong (sign error, un
it
    mismatch, wrong index). Fixable in 5 minutes once we see the numbers.

(B) NOISE AMPLIFICATION – the math is correct, but most LiDAR clusters are
near-static clutter (kerbs, poles, building fragments), and using 2 nois
y
    points to estimate velocity creates fake motion that pushes the predi
ct
    position AWAY from the real (mostly still) next detection. This means
    Phase 2 (filtering clutter) needs to happen BEFORE Phase 1, not after.

Run this INSIDE Step_3_1_LiDAR_Fusion, right after your Cell 4 (the
patched tracker) and Cell 5 (the main loop) have both run – so `tracker` still
holds real data. Paste this as a new cell at the end and run it.
"""

import numpy as np

# — Recompute, for every association attempt, whether the prediction helped —
# We re-run the same loop logic used inside update(), but this time we log
# BOTH the "old way" distance (from last_pos) and the "new way" distance
# (from pred) to whatever the tracker actually matched, so we can see which
# one was closer to truth more often.

improved, worsened, unchanged = 0, 0, 0
deltas = []          # positive = prediction moved AWAY from the eventual matc
h (bad)
speeds = []          # the estimated speed for every 2+-point track, at time
of prediction

for tid, track in (**tracker.finished_tracks, **tracker.active_tracks).items
():
    pts = track["points"]
    for k in range(2, len(pts)):
        prev2, prev1, cur = pts[k-2], pts[k-1], pts[k]
        t_prev2, pos_prev2 = prev2[1], np.array(prev2[2], dtype=float)
        t_prev1, pos_prev1 = prev1[1], np.array(prev1[2], dtype=float)
        t_cur, pos_cur = cur[1], np.array(cur[2], dtype=float)

        dt_prev = (t_prev1 - t_prev2) / 1e6
        dt_now = (t_cur - t_prev1) / 1e6
        if dt_prev <= 0 or dt_now <= 0:
            continue

        vel = (pos_prev1 - pos_prev2) / dt_prev
        spd = np.linalg.norm(vel)
        speeds.append(spd)
        if spd > 30.0:
            vel = vel * 30.0 / spd
            pred = pos_prev1 + vel * dt_now
```

```

        dist_old = np.linalg.norm(pos_cur - pos_prev1)    # "last position" method
        dist_new = np.linalg.norm(pos_cur - pred)         # "predicted position" method
        delta = dist_new - dist_old                       # negative = prediction helped
        deltas.append(delta)

        if delta < -0.05:
            improved += 1
        elif delta > 0.05:
            worsened += 1
        else:
            unchanged += 1

deltas = np.array(deltas)
speeds = np.array(speeds)

print("=" * 70)
print("DIAGNOSTIC: did the motion prediction help or hurt, on your actual data?")
print("=" * 70)
print(f"Association steps analysed : {len(deltas)}")
print(f"Prediction MOVED CLOSER to next real detection : {improved} ({improved/max(len(deltas),1)*100:.1f}%)")
print(f"Prediction MOVED FARTHER to next real detection : {worsened} ({worsened/max(len(deltas),1)*100:.1f}%)")
print(f"No meaningful difference from next real detection : {unchanged} ({unchanged/max(len(deltas),1)*100:.1f}%)")
print()
print(f"Median delta (negative = prediction helped) : {np.median(deltas):+.3f} m")
print(f"Mean delta : {np.mean(deltas):+.3f} m")
print()
print("Estimated speed distribution (this is the 'velocity' the code computed):")
print(f"    median {np.median(speeds):.2f} m/s    mean {np.mean(speeds):.2f} m/s")
print(f"    90th pct {np.percentile(speeds, 90):.2f} m/s    max {speeds.max():.2f} m/s")
print(f"    fraction of estimated speeds ABOVE 15 m/s (54 km/h, implausible for most objects) : {((speeds > 15).mean()*100:.1f}%)")
print()

# — Verdict —
if worsened > improved * 1.2:
    print(">>> VERDICT: NOISE AMPLIFICATION (hypothesis B).")
    print("    The prediction is making the match WORSE more often than it helps.")
    print("    This is consistent with velocity being estimated from noisy 2-point differences on mostly-static clutter, not a code bug.")
    print("    NEXT STEP: do Phase 2 (LiDAR extent filter, radar clustering) FIRST.")

```

```

    print("    then re-apply Phase 1's motion prediction on the cleaned data.")
elif improved > worsened * 1.2:
    print(">>> VERDICT: prediction IS helping at the point-pair level.")
    print("    If track count still went up despite this, the regression is likely")
    print("    coming from somewhere else – check the MAX_MISSED_FRAMES eviction")
    print("    logic, or whether `timestamp` is arriving as expected. Print the")
    print("    first 5 calls to `.update()` in Cell 5 and manually inspect the")
    print("    `timestamp` and `dt_now` values.")
else:
    print(">>> VERDICT: prediction is roughly a wash at the point-pair level")
    print("    –")
    print("    not clearly better or worse. Track count doubling then likely comes")
    print("    from a DIFFERENT source. Check for a duplicate-run artifact: did you")
    print("    re-run Cell 5 twice without restarting the kernel? Old `tracker`")
    print("    state")
    print("    can persist and double-count. Confirm with a fresh kernel restart.")

print()
print("If speeds above are mostly under 2-3 m/s with occasional huge outliers,")
print("that ALSO points to noise amplification: real objects don't have 20 m/s")
print("bursts between consecutive frames unless the underlying detection jumped.")

```

```
=====
DIAGNOSTIC: did the motion prediction help or hurt, on your actual data?
=====
```

```
Association steps analysed : 10697
Prediction MOVED CLOSER to next real detection : 3579 (33.5%)
Prediction MOVED FARTHER : 5193 (48.5%)
No meaningful difference : 1925 (18.0%)
```

```
Median delta (negative = prediction helped) : +0.042 m
Mean delta : -0.322 m
```

```
Estimated speed distribution (this is the 'velocity' the code computed):
  median 0.88 m/s   mean 2.27 m/s   90th pct 5.73 m/s   max 43.91 m/s
  fraction of estimated speeds ABOVE 15 m/s (54 km/h, implausible for most ob
jects) : 1.8%
```

```
>>> VERDICT: NOISE AMPLIFICATION (hypothesis B).
  The prediction is making the match WORSE more often than it helps.
  This is consistent with velocity being estimated from noisy 2-point
  differences on mostly-static clutter, not a code bug.
  NEXT STEP: do Phase 2 (LiDAR extent filter, radar clustering) FIRST,
  then re-apply Phase 1's motion prediction on the cleaned data.
```

If speeds above are mostly under 2-3 m/s with occasional huge outliers,
that ALSO points to noise amplification: real objects don't have 20 m/s
bursts between consecutive frames unless the underlying detection jumped.

```

In [8]: # -----
# CELL 6 – Track length distribution – sanity check for fragmentation
# If this fix worked, you should see noticeably fewer 2-frame tracks
# and more longer tracks compared to the pre-fix version.
# -----

import pandas as pd

track_lengths = []
for track_file in FUSION_OUT_DIR.glob("track_*.json"):
    with open(track_file) as f:
        track = json.load(f)
        track_lengths.append(len(track))

length_df = pd.DataFrame({"track_length": track_lengths})
summary_path = STEP3_DIR / "lidar_tracking_summary.csv"
length_df.to_csv(summary_path, index=False)

print(f"✅ Summary saved: {summary_path}")
print(f"  Total tracks      : {len(length_df)}")
print(f"  Mean track length   : {length_df['track_length'].mean():.1f} frame
s")
print(f"  Median track length: {length_df['track_length'].median():.0f} frame
s")
print(f"  Tracks of length 2 (minimum, most fragmented): "
      f"{(length_df['track_length'] == 2).sum()} ({(length_df['track_length']
== 2).mean()*100:.1f}%)"
)
print(f"  Tracks of length 10+: {(length_df['track_length'] >= 10).sum()}")

display(length_df.describe())

```

```

✅ Summary saved: F:\Sensor fusion Research\output\step_3\lidar_tracking_summ
ary.csv
  Total tracks      : 2327
  Mean track length : 6.6 frames
  Median track length: 4 frames
  Tracks of length 2 (minimum, most fragmented): 510 (21.9%)
  Tracks of length 10+: 410

```

	track_length
count	2327.000000
mean	6.596906
std	6.536553
min	2.000000
25%	3.000000
50%	4.000000
75%	8.000000
max	41.000000

Step 3.2 — Radar Multi-Frame Tracking (Upgraded, No UKF Yet)

Input	output/step_2/radar/<sample>/<channel>.json (from upgraded Step 2.2 — calibration already embedded per channel)
Outputs	output/step_3/radar/track_<id>.json — one file per track, points in GLOBAL frame output/step_3/radar_tracking_summary.csv
Used by	Step 5 (TTC estimation, radar-only baseline), Step 4 (fusion)

Bugs fixed — same family as Step 3.1, plus one extra

1. **["points"] unwrap fix** — same compatibility break as before, now against the upgraded Step 2.2 output.
2. **Per-channel calibration (radar-specific issue LiDAR doesn't have)**. Radar has 5 physically separate sensors, each with its own sensor-to-ego calibration. The original code pooled raw local-frame (x, y) from all 5 channels together with no transform at all — a detection from RADAR_FRONT and one from RADAR_BACK_LEFT were being compared as if they shared a coordinate system, when they don't. Fixed by transforming **each channel's points using that channel's own calibration** before combining.
3. **Ego-motion contamination across frames** — same fix as Step 3.1: transform to the global frame so only true object motion affects tracking, not the car's own movement.
4. **Double-assignment bug** — same Hungarian algorithm fix as Step 3.1.
5. **No track termination** — same MAX_MISSED_FRAMES eviction as Step 3.1.

Note

Radar calibration didn't need a separate lookup file this time — Step 2.2 already embeds each channel's calibration block directly in its output JSON, so this notebook reads it straight from there.

```
In [1]: # -----
# CELL 1 – Verify config.py exists
# -----

import shutil
from pathlib import Path

if not Path("config.py").exists():
    raise FileNotFoundError("config.py not found. Copy it from the repo root.")

from config import STEP0_DIR, STEP2_DIR, STEP3_DIR

RADAR_IN_DIR = STEP2_DIR / "radar"
RADAR_OUT_DIR = STEP3_DIR / "radar"
if RADAR_OUT_DIR.exists():
    shutil.rmtree(RADAR_OUT_DIR) # FIXED: clear stale per-object files before this run (was: accumulated silently across re-runs, never cleared)
RADAR_OUT_DIR.mkdir(parents=True, exist_ok=True)

if not RADAR_IN_DIR.exists():
    raise FileNotFoundError(f"Step 2.2 output not found at {RADAR_IN_DIR} – run Step 2.2 first.")

print(f"✓ RADAR_IN_DIR : {RADAR_IN_DIR}")
print(f"✓ RADAR_OUT_DIR: {RADAR_OUT_DIR}")

config.py loaded. PROJECT_ROOT = F:\Sensor fusion Research
DATA_ROOT = F:\Sensor fusion Research\DATA SET\archive
OUTPUT_ROOT = F:\Sensor fusion Research\output
✓ RADAR_IN_DIR : F:\Sensor fusion Research\output\step_2\radar
✓ RADAR_OUT_DIR: F:\Sensor fusion Research\output\step_3\radar
```

```
In [2]: # -----
# CELL 2 – Named constants (same values as Step 3.1 for consistency)
# -----

ASSOC_DIST_THRESHOLD = 3.0 # metres
MAX_MISSED_FRAMES = 3 # frames

print(f"✓ ASSOC_DIST_THRESHOLD = {ASSOC_DIST_THRESHOLD}m, MAX_MISSED_FRAMES = {MAX_MISSED_FRAMES}")

✓ ASSOC_DIST_THRESHOLD = 3.0m, MAX_MISSED_FRAMES = 3
```

In [3]:

```
# -----  
# CELL 3 – Sensor-to-global transform  
# Moved to src/geometry.py – shared with Step 1.1, Step 2.3.1, Step 3.1  
# -----  
  
import numpy as np  
from pyquaternion import Quaternion  
from src.geometry import transform_matrix, point_to_global, vector_to_global  
  
print("\u2705 Transform utilities loaded.")
```

✅ Transform utilities loaded.

```

In [4]: # -----
# CELL 4 – Tracker with one-to-one assignment + eviction (same as Step 3.1)
# FIXED: scene-boundary isolation – tracks never associate across a scene_name
#         change; at a boundary all active tracks are force-finalized and a
#         fresh track table starts for the new scene (no carried IDs/state).
# -----

import uuid
from scipy.optimize import linear_sum_assignment

class GlobalFrameTracker:
    def __init__(self, dist_thresh, max_missed, min_hits_to_confirm=2, tentative_max_missed=0):
        self.active_tracks = {}
        self.finished_tracks = {}
        self.dist_thresh = dist_thresh
        self.max_missed = max_missed
        self.current_scene_name = None
        self.association_log = [] # (prev_sample_id, prev_scene, curr_sample_id, curr_scene) per accepted match

        # FIXED: track confirmation (M-of-N), same fix as Step 3.1's own
        # tracker. A brand-new (1-hit) track previously got the same
        # max_missed tolerance as an established track, letting a one-off
        # noisy blip sit active and compete in Hungarian assignment for up
        # to max_missed frames before being dropped. Now an unconfirmed
        # track gets a much shorter tentative_max_missed tolerance instead.
        # save_tracks() already only writes tracks with 2+ points, so this
        # does not change which tracks get saved -- it changes how long an
        # unconfirmed one can stick around and interfere with others first.
        self.min_hits_to_confirm = min_hits_to_confirm
        self.tentative_max_missed = tentative_max_missed

    def update(self, detections_global, sample_id, timestamp, scene_name):
        if self.current_scene_name is not None and scene_name != self.current_scene_name:
            # Scene boundary: close out every active track from the previous scene and
            # start a fresh, empty track table. No track IDs, motion state, or
            # "last known
            # position" carry across – each survivor is just finalized as-is.
            self.finished_tracks.update(self.active_tracks)
            self.active_tracks = {}
            self.current_scene_name = scene_name

            # detections_global entries may carry extra trailing fields beyond
            # [x, y, z] (radar's own compensated [vx, vy]) -- association/gating
            # always operates on the leading 3 position elements only; the full
            # entry (with velocity, if present) is what actually gets stored.
            det_pos_arr = np.array([d[:3] for d in detections_global]) if detections_global else np.empty((0, 3))
            track_ids = list(self.active_tracks.keys())
            n_tracks, n_dets = len(track_ids), len(det_pos_arr)

            matched_track_idx, matched_det_idx = set(), set()

```

```

        if n_tracks > 0 and n_dets > 0:
            cost = np.zeros((n_tracks, n_dets))
            for i, tid in enumerate(track_ids):
                pts = self.active_tracks[tid]["points"] # [(sample_id, ti
                mestamp, [x,y,z,(vx,vy)]), ...]
                last_pos = np.array(pts[-1][2][:3], dtype=float)
                pred = last_pos
                if len(pts) >= 2 and pts[-1][1] is not None and pts[-2][1] is
not None:
                    dt_prev = (pts[-1][1] - pts[-2][1]) / 1e6
                    dt_now = (timestamp - pts[-1][1]) / 1e6
                    if dt_prev > 0 and dt_now > 0:
                        vel = (last_pos - np.array(pts[-2][2][:3], dtype=floa
t)) / dt_prev
                        spd = np.linalg.norm(vel)
                        if spd > 30.0: # clamp: 108 km/
h, guards against
                                vel = vel / spd * 30.0 # one bad centr
oid flinging the gate
                        pred = last_pos + vel * dt_now
                        cost[i] = np.linalg.norm(det_pos_arr - pred, axis=1)

                row_idx, col_idx = linear_sum_assignment(cost)
                for r, c in zip(row_idx, col_idx):
                    if cost[r, c] < self.dist_thresh:
                        tid = track_ids[r]
                        prev_sample_id, prev_timestamp, _ = self.active_tracks[ti
d]["points"][-1]
                        self.association_log.append((prev_sample_id, self.current_
scene_name, sample_id, scene_name))
                        self.active_tracks[tid]["points"].append((sample_id, times
tamp, detections_global[c]))
                        self.active_tracks[tid]["missed"] = 0
                        self.active_tracks[tid]["hits"] = self.active_tracks[tid].
get("hits", 1) + 1
                        matched_track_idx.add(r)
                        matched_det_idx.add(c)

                # FIXED: unconfirmed tracks get a shorter eviction tolerance -- see
                # the confirmation note in __init__.
                for i, tid in enumerate(track_ids):
                    if i not in matched_track_idx:
                        self.active_tracks[tid]["missed"] += 1
                        hits = self.active_tracks[tid].get("hits", 1)
                        tolerance = self.max_missed if hits >= self.min_hits_to_confir
m else self.tentative_max_missed
                        if self.active_tracks[tid]["missed"] > tolerance:
                            self.finished_tracks[tid] = self.active_tracks.pop(tid)

                for j in range(n_dets):
                    if j not in matched_det_idx:
                        tid = str(uuid.uuid4())[:8]
                        self.active_tracks[tid] = {
                            "points": [(sample_id, timestamp, detections_global[j])],
                            "missed": 0,
                            "hits": 1,

```

```
def save_tracks(self, out_dir):
    all_tracks = {**self.finished_tracks, **self.active_tracks}
    n_saved = 0
    for tid, track in all_tracks.items():
        if len(track["points"]) >= 2:
            with open(out_dir / f"track_{tid}.json", "w") as f:
                json.dump(track["points"], f, indent=2)
            n_saved += 1
    return n_saved, len(all_tracks)

print("GlobalFrameTracker defined.")
```

GlobalFrameTracker defined.

```

In [5]: # -----
# CELL 5 – Main Loop
# FIXED: unwraps ["points"]
# FIXED: each of the 5 radar channels transformed with ITS OWN calibration
#         before being combined into one per-sample detection list
# FIXED: one real object triggers 3-8 radar blips (echoes off different parts
#         of the same car); each was being tracked as a separate object. Now
#         DBSCAN groups blips within RADAR_CLUSTER_EPS of each other into one
#         detection (its centroid) before they reach the tracker.
# -----

import json
import numpy as np
from tqdm import tqdm
from sklearn.cluster import DBSCAN

RADAR_CLUSTER_EPS = 1.5 # metres – blips this close are treated as one object

def cluster_radar_detections(detections_global, eps):
    """Groups nearby blips (multiple radar echoes off the same car) into one
    detection per real object, using each cluster's centroid. min_samples=1
    so an isolated blip still becomes its own single-point detection.

    FIXED: clusters on POSITION only ([x, y, z], the first 3 columns) even
    though each detection now also carries radar's own [vx, vy] as trailing
    columns -- clustering jointly on position AND velocity would change
    which blips get grouped together based on velocity differences alone,
    which was never the point of this grouping step. The velocity columns
    still get averaged into each cluster's centroid, same as position."""
    if len(detections_global) <= 1:
        return detections_global
    pts = np.array(detections_global)
    labels = DBSCAN(eps=eps, min_samples=1).fit_predict(pts[:, :3])
    return [pts[labels == lbl].mean(axis=0).tolist() for lbl in sorted(set(labels))]

with open(STEP0_DIR / "samples_index.json") as f:
    samples_index = json.load(f)

tracker = GlobalFrameTracker(dist_thresh=ASSOC_DIST_THRESHOLD, max_missed=MAX_MISSED_FRAMES)
n_samples_processed = 0
n_samples_no_detections = 0

for sample_id in tqdm(sorted(samples_index.keys()), desc="Tracking radar objects"):
    sample_dir = RADAR_IN_DIR / sample_id
    if not sample_dir.exists():
        continue

    detections_global = []

    for radar_file in sample_dir.glob("RADAR_*.json"):
        with open(radar_file) as f:
            radar_data = json.load(f)

```

```

        points = radar_data["points"]          # FIXED – unwrap dict struc
ture
        calibration = radar_data["calibration"] # THIS channel's own calib
ration
        ego_pose = calibration["ego_pose"]
        calib = {
            "translation": calibration["sensor_to_ego_translation"],
            "rotation": calibration["sensor_to_ego_rotation"]
        }

        for pt in points:
            global_xyz = point_to_global(
                [pt["x"], pt["y"], pt.get("z", 0.0)], ego_pose, calib
            )
            # FIXED: radar's own ego-motion-compensated Doppler velocity
            # (vx_comp, vy_comp) was parsed and stored back in Step 2.2 but
            # never used anywhere downstream -- radar's main strength (direct
            # velocity measurement) was being thrown away, only position was
            # ever kept. Rotated into the global frame with vector_to_global()
            # (rotation only, no translation -- a velocity has no position to
            # translate) and carried alongside position as extra trailing
            # fields: [x, y, z, vx, vy].
            global_vel = vector_to_global(
                [pt.get("vx_comp", 0.0), pt.get("vy_comp", 0.0), 0.0], ego_pos
e, calib
            )
            detections_global.append(global_xyz.tolist() + global_vel[:2].toli
st())

        timestamp = samples_index[sample_id]["timestamp_us"]
        scene_name = samples_index[sample_id]["scene_name"]

        if len(detections_global) == 0:
            n_samples_no_detections += 1

        clustered_detections = cluster_radar_detections(detections_global, RADAR_C
LUSTER_EPS)
        tracker.update(clustered_detections, sample_id, timestamp, scene_name)
        n_samples_processed += 1

    n_saved, n_total = tracker.save_tracks(RADAR_OUT_DIR)

    # — Scene-boundary isolation check —————
    cross_scene_associations = [a for a in tracker.association_log if a[1] != a
[3]]
    assert len(cross_scene_associations) == 0, \
        f"{len(cross_scene_associations)} cross-scene associations found: {cross_s
cene_associations[:5]}"

    print(f"\nStep 3.2 complete.")
    print(f"    Samples processed          : {n_samples_processed}")
    print(f"    Samples with zero detections: {n_samples_no_detections}")
    print(f"    Total tracks created           : {n_total}")
    print(f"    Tracks saved (length >= 2)    : {n_saved}")
    print(f"    Associations logged            : {len(tracker.association_log)} (0 cross

```

```
s-scene, verified)")
print(f"Saved to: {RADAR_OUT_DIR}")
```

```
Tracking radar objects: 19%[REDACTED]
| 78/404 [00:04<00:18, 17.94it/s]C:\Users\DS\AppData\Roaming\Python\Python312
\site-packages\joblib\externals\loky\backend\context.py:136: UserWarning: Cou
ld not find the number of physical cores for the following reason:
[WinError 2] The system cannot find the file specified
Returning the number of logical cores instead. You can silence this warning b
y setting LOKY_MAX_CPU_COUNT to the number of cores you want to use.
```

```
warnings.warn(
  File "C:\Users\DS\AppData\Roaming\Python\Python312\site-packages\joblib\externals\loky\backend\context.py", line 257, in _count_physical_cores
```

```
cpu_info = subprocess.run(
    ^^^^^^^^^^^^^^^^^^^^
```

File "D:\anaconda\Lib\subprocess.py", line 548, in run
with Popen(*popenargs, **kwargs) as process:

ΛΛ

File "D:\anaconda\Lib\subprocess.py", line 1026, in __init__

```
self._execute_child(args, executable, preexec_fn, close_fds,
```

File "D:\anaconda\Lib\subprocess.py", line 1538, in _execute_child

```
hp, ht, pid, tid = _winapi.CreateProcess(executable, args,
```

```
Tracking radar objects: 100%|██████████|
██████████| 404/404 [00:20<00:00, 19.33it/s]
```

Step 3.2 complete.

Samples processed : 404

Samples with zero detections: 2

Total tracks created : 18323

Tracks saved (length >= 2) : 7283

Associations logged : 25997 (0 cross-scene, verified)

Saved to: F:\Sensor fusion Research\output\step_3\radar

```
[... 112 further output line(s)/block(s) omitted (repetitive per-iteration ou
tput) for the PDF export ...]
```

Step 3.2 complete.

Samples processed : 404

Samples with zero detections: 2

Total tracks created : 18323

Tracks saved (length >= 2) : 7283

Associations logged : 25997 (0 cross-scene, verified)

Saved to: F:\Sensor fusion Research\output\step_3\radar

```
In [6]: # -----
# CELL 6 – Track Length distribution – same sanity check as Step 3.1
# -----

import pandas as pd

track_lengths = []
for track_file in RADAR_OUT_DIR.glob("track_*.json"):
    with open(track_file) as f:
        track = json.load(f)
        track_lengths.append(len(track))

length_df = pd.DataFrame({"track_length": track_lengths})
summary_path = STEP3_DIR / "radar_tracking_summary.csv"
length_df.to_csv(summary_path, index=False)

print(f"✅ Summary saved: {summary_path}")
print(f"    Total tracks      : {len(length_df)}")
print(f"    Mean track length  : {length_df['track_length'].mean():.1f} frame
s")
print(f"    Median track length: {length_df['track_length'].median():.0f} frame
s")
print(f"    Tracks of length 2 (most fragmented): "
      f"{(length_df['track_length'] == 2).sum()} {(length_df['track_length']
== 2).mean()*100:.1f}%")
print(f"    Tracks of length 10+: {(length_df['track_length'] >= 10).sum()}")

display(length_df.describe())

✅ Summary saved: F:\Sensor fusion Research\output\step_3\radar_tracking_summ
ary.csv
Total tracks      : 7283
Mean track length : 4.6 frames
Median track length: 3 frames
Tracks of length 2 (most fragmented): 2331 (32.0%)
Tracks of length 10+: 588
```

	track_length
count	7283.000000
mean	4.569546
std	3.407940
min	2.000000
25%	2.000000
50%	3.000000
75%	6.000000
max	37.000000

In []:

Step 3.3 — Camera Multi-Frame Tracker, Global 3D Nearest-Neighbor (Upgraded)

Input	output/step_2/yolo_global/<sample>/<camera>.json (Step 2.3.1 — has global_x/y/z , has_3d_position)
Outputs	output/step_3/camera/track_<id>.json — one file per track, trajectory across ALL cameras output/step_3/camera_tracking_summary.csv
Used by	Step 5 (TTC estimation, camera-only baseline), Step 4 (fusion)

Development history (kept for context)

Your notebook shows a genuinely good debugging process: the first attempt only linked detections within a single sample (no cross-time trajectory), which you correctly diagnosed and fixed in a second version that builds full trajectories. That second version is the starting point here.

Bugs fixed in that second version

1. **Field mismatch with Step 2.3.1's actual output** — expected global_xyz (list) and class , but Step 2.3.1 outputs global_x , global_y , global_z (separate keys) and class_name . Also added the missing check for has_3d_position — detections without a valid LiDAR match are now correctly skipped instead of causing a crash or a bad position.
2. **Within-frame double-assignment** — a track could be extended twice in the same frame since matching happened detection-by-detection with immediate updates. Fixed with one Hungarian assignment per frame, same pattern as Step 3.1/3.2.
3. **No track eviction** — the final version had dropped the MAX_MISSED_FRAMES logic your first draft had. Restored.

Design change

Unified tracking across all 6 cameras instead of 6 independent per-camera trackers. Since Step 2.3.1 already puts every detection in the same global frame, an object crossing from one camera's view into another's should be one continuous track — not two broken ones. This also matches the LiDAR/Radar tracker pattern for consistency across your codebase.

```
In [1]: # -----
# CELL 1 – Verify config.py exists
# -----

import shutil
from pathlib import Path

if not Path("config.py").exists():
    raise FileNotFoundError("config.py not found. Copy it from the repo root.")

from config import STEP0_DIR, STEP2_DIR, STEP3_DIR

YOLO_GLOBAL_DIR = STEP2_DIR / "yolo_global"
CAMERA_OUT_DIR = STEP3_DIR / "camera"
if CAMERA_OUT_DIR.exists():
    shutil.rmtree(CAMERA_OUT_DIR) # FIXED: clear stale per-object files before this run (was: accumulated silently across re-runs, never cleared)
CAMERA_OUT_DIR.mkdir(parents=True, exist_ok=True)

if not YOLO_GLOBAL_DIR.exists():
    raise FileNotFoundError(f"Step 2.3.1 output not found at {YOLO_GLOBAL_DIR} – run Step 2.3.1 first.")

print(f"✅ YOLO_GLOBAL_DIR: {YOLO_GLOBAL_DIR}")
print(f"✅ CAMERA_OUT_DIR : {CAMERA_OUT_DIR}")
```

```
config.py loaded. PROJECT_ROOT = F:\Sensor fusion Research
DATA_ROOT = F:\Sensor fusion Research\DATA SET\archive
OUTPUT_ROOT = F:\Sensor fusion Research\output
✅ YOLO_GLOBAL_DIR: F:\Sensor fusion Research\output\step_2\yolo_global
✅ CAMERA_OUT_DIR : F:\Sensor fusion Research\output\step_3\camera
```

```
In [2]: # -----
# CELL 2 – Named constants
# -----

DIST_THRESHOLD = 2.0 # metres – kept at your original value (camera-derived 3D is noisier than LiDAR's own)
MAX_MISSED_FRAMES = 3 # frames

CAMERA_NAMES = ['CAM_FRONT', 'CAM_FRONT_LEFT', 'CAM_FRONT_RIGHT',
                 'CAM_BACK', 'CAM_BACK_LEFT', 'CAM_BACK_RIGHT']

print(f"✅ DIST_THRESHOLD = {DIST_THRESHOLD}m, MAX_MISSED_FRAMES = {MAX_MISSED_FRAMES}")

✅ DIST_THRESHOLD = 2.0m, MAX_MISSED_FRAMES = 3
```

```

In [3]: # -----
# CELL 3 – Class-aware global tracker
# Same Hungarian-assignment + eviction pattern as Step 3.1/3.2,
# extended with class-name gating (a car should never match a pedestrian track)
# FIXED: scene-boundary isolation – tracks never associate across a scene_name
#       change; at a boundary all active tracks are force-finalized and a
#       fresh track table starts for the new scene (no carried IDs/state).
# -----

import uuid
import numpy as np
from scipy.optimize import linear_sum_assignment

class ClassAwareGlobalTracker:
    def __init__(self, dist_thresh, max_missed):
        self.active_tracks = {} # tid -> {"class_name":..., "trajectory":
        [...], "missed": int}
        self.finished_tracks = {}
        self.dist_thresh = dist_thresh
        self.max_missed = max_missed
        self.current_scene_name = None
        self.association_log = [] # (prev_sample_id, prev_scene, curr_sample_
        id, curr_scene) per accepted match

    def update(self, detections, sample_id, scene_name):
        """detections: list of dicts with keys pos:[x,y,z], class_name, confid
        ence, camera."""
        if self.current_scene_name is not None and scene_name != self.current_
        scene_name:
            # Scene boundary: close out every active track from the previous s
            cene and
            # start a fresh, empty track table. No track IDs, motion state, or
            "Last known
            # position" carry across – each survivor is just finalized as-is.
            self.finished_tracks.update(self.active_tracks)
            self.active_tracks = {}
            self.current_scene_name = scene_name

        track_ids = list(self.active_tracks.keys())
        n_tracks, n_dets = len(track_ids), len(detections)

        matched_track_idx, matched_det_idx = set(), set()

        if n_tracks > 0 and n_dets > 0:
            SENTINEL = 1e6
            cost = np.full((n_tracks, n_dets), SENTINEL)

            for i, tid in enumerate(track_ids):
                track = self.active_tracks[tid]
                traj = track["trajectory"]
                last_pos = np.array(traj[-1]["pos"], dtype=float)
                pred = last_pos
                if len(traj) >= 2:
                    last_pos = np.array(traj[-2]["pos"], dtype=float))

            / 0.5 # 2 Hz keyframes

```

```

        spd = np.linalg.norm(vel)
        if spd > 30.0:
            vel = vel / spd * 30.0
        pred = last_pos + vel * 0.5
    for j, det in enumerate(detections):
        if det["class_name"] != track["class_name"]:
            continue # never match across different object classe
s
        d = np.linalg.norm(np.array(det["pos"]) - pred)
        if d < self.dist_thresh:
            cost[i, j] = d

    row_idx, col_idx = linear_sum_assignment(cost)
    for r, c in zip(row_idx, col_idx):
        if cost[r, c] < SENTINEL: # a real match, not the "no valid
pairing" sentinel
            tid = track_ids[r]
            det = detections[c]
            prev_sample_id = self.active_tracks[tid]["trajectory"][-1]
["sample_id"]
            self.association_log.append((prev_sample_id, self.current_
scene_name, sample_id, scene_name))
            self.active_tracks[tid]["trajectory"].append({
                "sample_id": sample_id, "pos": det["pos"],
                "confidence": det["confidence"], "camera": det["camer
a"]

            })
            self.active_tracks[tid]["missed"] = 0
            matched_track_idx.add(r)
            matched_det_idx.add(c)

    for i, tid in enumerate(track_ids):
        if i not in matched_track_idx:
            self.active_tracks[tid]["missed"] += 1
            if self.active_tracks[tid]["missed"] > self.max_missed:
                self.finished_tracks[tid] = self.active_tracks.pop(tid)

    for j in range(n_dets):
        if j not in matched_det_idx:
            det = detections[j]
            tid = f"cam_{uuid.uuid4().hex[:8]}"
            self.active_tracks[tid] = {
                "class_name": det["class_name"],
                "trajectory": [{
                    "sample_id": sample_id, "pos": det["pos"],
                    "confidence": det["confidence"], "camera": det["camer
a"]

                }],
                "missed": 0
            }

    def save_tracks(self, out_dir):
        all_tracks = {**self.finished_tracks, **self.active_tracks}
        n_saved = 0
        for tid, track in all_tracks.items():
            if len(track["trajectory"]) >= 2:
                with open(out_dir / f"track_{tid}.json", "w") as f:

```

```
        json.dump({
            "track_id": tid,
            "class_name": track["class_name"],
            "trajectory": track["trajectory"]
        }, f, indent=2)
        n_saved += 1
    return n_saved, len(all_tracks)

print("ClassAwareGlobalTracker defined.")
```

ClassAwareGlobalTracker defined.

```

In [4]: # —————
# CELL 4 – Main loop: merge all 6 cameras per sample, then track globally
# —————

import json
from tqdm import tqdm

with open(STEP0_DIR / "samples_index.json") as f:
    samples_index = json.load(f)

tracker = ClassAwareGlobalTracker(dist_thresh=DIST_THRESHOLD, max_missed=MAX_MISSED_FRAMES)
n_samples_processed = 0
n_detections_skipped_no_3d = 0
n_detections_used = 0

for sample_id in tqdm(sorted(samples_index.keys()), desc="Tracking camera objects"):
    sample_dir = YOLO_GLOBAL_DIR / sample_id
    if not sample_dir.exists():
        continue

    combined_detections = []

    for cam in CAMERA_NAMES:
        cam_file = sample_dir / f"{cam}.json"
        if not cam_file.exists():
            continue

        with open(cam_file) as f:
            dets = json.load(f)

        for det in dets:
            if not det.get("has_3d_position", False): # FIXED – skip detections with no LiDAR match
                n_detections_skipped_no_3d += 1
                continue

            combined_detections.append({
                "pos": [det["global_x"], det["global_y"], det["global_z"]],
                "class_name": det["class_name"],
                "confidence": det["confidence"],
                "camera": cam
            })
            n_detections_used += 1

    scene_name = samples_index[sample_id]["scene_name"]
    tracker.update(combined_detections, sample_id, scene_name)
    n_samples_processed += 1

n_saved, n_total = tracker.save_tracks(CAMERA_OUT_DIR)

# — Scene-boundary isolation check —————
cross_scene_associations = [a for a in tracker.association_log if a[1] != a[3]]

```



```

In [5]: # -----
# CELL 5 – Track Length distribution + cross-camera continuity check
# -----

import pandas as pd

track_lengths = []
cross_camera_tracks = 0

for track_file in CAMERA_OUT_DIR.glob("track_*.json"):
    with open(track_file) as f:
        track = json.load(f)
        track_lengths.append(len(track["trajectory"]))

    cams_seen = {pt["camera"] for pt in track["trajectory"]}
    if len(cams_seen) > 1:
        cross_camera_tracks += 1

length_df = pd.DataFrame({"track_length": track_lengths})
summary_path = STEP3_DIR / "camera_tracking_summary.csv"
length_df.to_csv(summary_path, index=False)

print(f"✅ Summary saved: {summary_path}")
print(f"  Total tracks          : {len(length_df)}")
print(f"  Mean track length       : {length_df['track_length'].mean():.1f} frame s")
print(f"  Tracks of length 2      : {(length_df['track_length'] == 2).sum()} "
      f"({(length_df['track_length'] == 2).mean()*100:.1f}%)")
print(f"  Tracks of length 10+    : {(length_df['track_length'] >= 10).sum()}")
print(f"\n  Tracks spanning MORE THAN ONE camera: {cross_camera_tracks} "
      f"({cross_camera_tracks/len(length_df)*100:.1f}% of all tracks)")
print("  This number is only possible with the unified global tracker –")
print("  the original per-camera design could never produce it.")

display(length_df.describe())

```

✓ Summary saved: F:\Sensor fusion Research\output\step_3\camera_tracking_summary.csv

Total tracks : 1131
Mean track length : 3.9 frames
Tracks of length 2 : 497 (43.9%)
Tracks of length 10+ : 52

Tracks spanning MORE THAN ONE camera: 522 (46.2% of all tracks)
This number is only possible with the unified global tracker –
the original per-camera design could never produce it.

	track_length
count	1131.000000
mean	3.889478
std	2.999880
min	2.000000
25%	2.000000
50%	3.000000
75%	5.000000
max	27.000000

Step 3.4 — Camera-Mono Multi-Frame Tracker (NEW — true monocular baseline)

Input	output/step_2/yolo_mono/<sample>/<camera>.json (Step 2.3.2 — LiDAR-free similar-triangles depth)
Outputs	output/step_3/camera_mono/track_<id>.json — one file per track, trajectory across ALL cameras output/step_3/camera_mono_tracking_summary.csv
Used by	Step 5 (TTC estimation, true monocular baseline)

Identical tracker design to Step 3.3 (`ClassAwareGlobalTracker` : one Hungarian assignment per frame, class-name gating, `MAX_MISSED_FRAMES` eviction, scene-boundary isolation) — duplicated rather than imported, same rationale CLAUDE.md already documents for `GlobalFrameTracker` (Step 3.1 vs 3.2): the two trackers read different upstream position sources (LiDAR-derived vs. similar-triangles depth) and are expected to diverge in tuning as each baseline matures, so a shared abstraction would need a closer read than a mechanical extraction. The only functional difference from Step 3.3 is the input directory and output track-id prefix (`cammono_` instead of `cam_` , so track ids never collide with Step 3.3's).

```

In [1]: # -----
# CELL 1 – Verify config.py exists
# -----

import shutil
from pathlib import Path

if not Path("config.py").exists():
    raise FileNotFoundError("config.py not found. Copy it from the repo root.")

from config import STEP0_DIR, STEP2_DIR, STEP3_DIR

YOLO_MONO_DIR      = STEP2_DIR / "yolo_mono"
CAMERA_MONO_OUT_DIR = STEP3_DIR / "camera_mono"
if CAMERA_MONO_OUT_DIR.exists():
    shutil.rmtree(CAMERA_MONO_OUT_DIR)  # clear stale per-object files before
this run (same fix as Steps 3.1-3.3/4)
CAMERA_MONO_OUT_DIR.mkdir(parents=True, exist_ok=True)

if not YOLO_MONO_DIR.exists():
    raise FileNotFoundError(f"Step 2.3.2 output not found at {YOLO_MONO_DIR} –
run Step 2.3.2 first.")

print(f"✅ YOLO_MONO_DIR      : {YOLO_MONO_DIR}")
print(f"✅ CAMERA_MONO_OUT_DIR: {CAMERA_MONO_OUT_DIR}")

config.py loaded. PROJECT_ROOT = F:\Sensor fusion Research
DATA_ROOT   = F:\Sensor fusion Research\DATA SET\archive
OUTPUT_ROOT = F:\Sensor fusion Research\output
✅ YOLO_MONO_DIR      : F:\Sensor fusion Research\output\step_2\yolo_mono
✅ CAMERA_MONO_OUT_DIR: F:\Sensor fusion Research\output\step_3\camera_mono

```

```

In [2]: # -----
# CELL 2 – Named constants (same values as Step 3.3)
# -----

DIST_THRESHOLD      = 2.0  # metres – camera-derived 3D is noisier than LiDAR's own
MAX_MISSED_FRAMES    = 3    # frames

CAMERA_NAMES = ['CAM_FRONT', 'CAM_FRONT_LEFT', 'CAM_FRONT_RIGHT',
                'CAM_BACK', 'CAM_BACK_LEFT', 'CAM_BACK_RIGHT']

print(f"✅ DIST_THRESHOLD = {DIST_THRESHOLD}m, MAX_MISSED_FRAMES = {MAX_MISSED_FRAMES}")

✅ DIST_THRESHOLD = 2.0m, MAX_MISSED_FRAMES = 3

```

```

In [3]: # -----
# CELL 3 – Class-aware global tracker (identical to Step 3.3's)
# -----

import uuid
import numpy as np
from scipy.optimize import linear_sum_assignment

class ClassAwareGlobalTracker:
    def __init__(self, dist_thresh, max_missed):
        self.active_tracks = {} # tid -> {"class_name":..., "trajectory":
[...], "missed": int}
        self.finished_tracks = {}
        self.dist_thresh = dist_thresh
        self.max_missed = max_missed
        self.current_scene_name = None
        self.association_log = [] # (prev_sample_id, prev_scene, curr_sample_
id, curr_scene) per accepted match

    def update(self, detections, sample_id, scene_name):
        """detections: list of dicts with keys pos:[x,y,z], class_name, confid
ence, camera."""
        if self.current_scene_name is not None and scene_name != self.current_
scene_name:
            self.finished_tracks.update(self.active_tracks)
            self.active_tracks = {}
            self.current_scene_name = scene_name

        track_ids = list(self.active_tracks.keys())
        n_tracks, n_dets = len(track_ids), len(detections)

        matched_track_idx, matched_det_idx = set(), set()

        if n_tracks > 0 and n_dets > 0:
            SENTINEL = 1e6
            cost = np.full((n_tracks, n_dets), SENTINEL)

            for i, tid in enumerate(track_ids):
                track = self.active_tracks[tid]
                traj = track["trajectory"]
                last_pos = np.array(traj[-1]["pos"], dtype=float)
                pred = last_pos
                if len(traj) >= 2:
                    vel = (last_pos - np.array(traj[-2]["pos"], dtype=float))
/ 0.5 # 2 Hz keyframes
                    spd = np.linalg.norm(vel)
                    if spd > 30.0:
                        vel = vel / spd * 30.0
                    pred = last_pos + vel * 0.5
                for j, det in enumerate(detections):
                    if det["class_name"] != track["class_name"]:
                        continue # never match across different object classe
s
                    d = np.linalg.norm(np.array(det["pos"]) - pred)
                    if d < self.dist_thresh:
                        cost[i, j] = d

```

```

row_idx, col_idx = linear_sum_assignment(cost)
for r, c in zip(row_idx, col_idx):
    if cost[r, c] < SENTINEL:
        tid = track_ids[r]
        det = detections[c]
        prev_sample_id = self.active_tracks[tid]["trajectory"][-1]
["sample_id"]
        self.association_log.append((prev_sample_id, self.current_
scene_name, sample_id, scene_name))
        self.active_tracks[tid]["trajectory"].append({
            "sample_id": sample_id, "pos": det["pos"],
            "confidence": det["confidence"], "camera": det["camer
a"]

        })
        self.active_tracks[tid]["missed"] = 0
        matched_track_idx.add(r)
        matched_det_idx.add(c)

for i, tid in enumerate(track_ids):
    if i not in matched_track_idx:
        self.active_tracks[tid]["missed"] += 1
        if self.active_tracks[tid]["missed"] > self.max_missed:
            self.finished_tracks[tid] = self.active_tracks.pop(tid)

for j in range(n_dets):
    if j not in matched_det_idx:
        det = detections[j]
        tid = f"cammono_{uuid.uuid4().hex[:8]}"
        self.active_tracks[tid] = {
            "class_name": det["class_name"],
            "trajectory": [{
                "sample_id": sample_id, "pos": det["pos"],
                "confidence": det["confidence"], "camera": det["camer
a"]

            }],
            "missed": 0
        }

def save_tracks(self, out_dir):
    all_tracks = {**self.finished_tracks, **self.active_tracks}
    n_saved = 0
    for tid, track in all_tracks.items():
        if len(track["trajectory"]) >= 2:
            with open(out_dir / f"track_{tid}.json", "w") as f:
                json.dump({
                    "track_id": tid,
                    "class_name": track["class_name"],
                    "trajectory": track["trajectory"]
                }, f, indent=2)
            n_saved += 1
    return n_saved, len(all_tracks)

print("ClassAwareGlobalTracker defined.")

```

```

In [4]: # -----
# CELL 4 – Main Loop: merge all 6 cameras per sample, then track globally
# -----

import json
from tqdm import tqdm

with open(STEP0_DIR / "samples_index.json") as f:
    samples_index = json.load(f)

tracker = ClassAwareGlobalTracker(dist_thresh=DIST_THRESHOLD, max_missed=MAX_MISSED_FRAMES)
n_samples_processed = 0
n_detections_skipped_no_3d = 0
n_detections_used = 0

for sample_id in tqdm(sorted(samples_index.keys()), desc="Tracking camera-mono objects"):
    sample_dir = YOLO_MONO_DIR / sample_id
    if not sample_dir.exists():
        continue

    combined_detections = []

    for cam in CAMERA_NAMES:
        cam_file = sample_dir / f"{cam}.json"
        if not cam_file.exists():
            continue

        with open(cam_file) as f:
            dets = json.load(f)

        for det in dets:
            if not det.get("has_3d_position", False):
                n_detections_skipped_no_3d += 1
                continue

            combined_detections.append({
                "pos": [det["global_x"], det["global_y"], det["global_z"]],
                "class_name": det["class_name"],
                "confidence": det["confidence"],
                "camera": cam
            })
            n_detections_used += 1

    scene_name = samples_index[sample_id]["scene_name"]
    tracker.update(combined_detections, sample_id, scene_name)
    n_samples_processed += 1

n_saved, n_total = tracker.save_tracks(CAMERA_MONO_OUT_DIR)

cross_scene_associations = [a for a in tracker.association_log if a[1] != a[3]]
assert len(cross_scene_associations) == 0, \
    f"{len(cross_scene_associations)} cross-scene associations found: {cross_s
cene_associations}"

```



```

In [5]: # -----
# CELL 5 – Track Length distribution + cross-camera continuity check
# -----

import pandas as pd

track_lengths = []
cross_camera_tracks = 0

for track_file in CAMERA_MONO_OUT_DIR.glob("track_*.json"):
    with open(track_file) as f:
        track = json.load(f)
        track_lengths.append(len(track["trajectory"]))

        cams_seen = {pt["camera"] for pt in track["trajectory"]}
        if len(cams_seen) > 1:
            cross_camera_tracks += 1

length_df = pd.DataFrame({"track_length": track_lengths})
summary_path = STEP3_DIR / "camera_mono_tracking_summary.csv"
length_df.to_csv(summary_path, index=False)

print(f"✅ Summary saved: {summary_path}")
print(f"    Total tracks          : {len(length_df)}")
print(f"    Mean track length       : {length_df['track_length'].mean():.1f} frame s")
print(f"    Tracks of length 2      : {(length_df['track_length'] == 2).sum()} "
      f"({(length_df['track_length'] == 2).mean()*100:.1f}%)")
print(f"    Tracks of length 10+    : {(length_df['track_length'] >= 10).sum()}")
print(f"\n    Tracks spanning MORE THAN ONE camera: {cross_camera_tracks} "
      f"({cross_camera_tracks/len(length_df)*100:.1f}% of all tracks)")

display(length_df.describe())

```

✓ Summary saved: F:\Sensor fusion Research\output\step_3\camera_mono_tracking_summary.csv

Total tracks : 953
Mean track length : 4.7 frames
Tracks of length 2 : 367 (38.5%)
Tracks of length 10+ : 87

Tracks spanning MORE THAN ONE camera: 458 (48.1% of all tracks)

	track_length
count	953.000000
mean	4.690451
std	4.787466
min	2.000000
25%	2.000000
50%	3.000000
75%	5.000000
max	41.000000

Step 4 — Multi-Sensor Track Fusion (Upgraded)

Input	output/step_3/lidar/track_*.json , output/step_3/radar/track_*.json (tuple format) output/step_3/camera/track_*.json (dict format, from Step 3.3)
Outputs	output/step_4/fused_tracks_all.csv — one row per fused-track point (fast, primary output) output/step_4/fused/track_<id>.json — one file per fused track (now feasible — thousands, not millions) output/step_4/fusion_summary.csv
Used by	Step 5 (TTC estimation)

The real cause of the 22-hour runtime

`active_fused_objects` = `detections` at the end of each loop carried forward **every raw detection** from the current frame (~10,000+ points, dominated by radar) as the matching pool for the next frame. The nested loop then ran roughly 100 million scalar `euclidean()` calls per frame. That's the 196.72s/iteration you measured — not primarily the file writes, though writing 1M+ individual JSON files made it worse.

The fix: fuse at the TRACK level, not the raw-point level

Steps 3.1/3.2/3.3 already turned millions of raw points into a few thousand clean, deduplicated tracks. This notebook now fuses those tracks — a problem that's orders of magnitude smaller and doesn't need any raw point data at all.

Also fixed

- **Format compatibility** — loaders now correctly parse Step 3.1/3.2's tuple format and Step 3.3's dict/trajectory format.
- **Converted from Colab (Drive mount + zip) to local, config.py-based** — consistent with the rest of the pipeline.
- **Proper one-shot Hungarian assignment per frame** instead of nested nearest-neighbor nested loops.
- **Within-frame sensor merging** — if LiDAR and radar both observe the same real object in the same frame, they're merged into one observation (averaged position, sensors recorded) before being matched against existing fused tracks. This is the actual "fusion" step that was largely absent before.
- **Track eviction** — same `MAX_MISSED_FRAMES` pattern as Steps 3.1–3.3.

```

In [1]: # -----
# CELL 1 – Verify config.py exists
# -----

import shutil
from pathlib import Path

if not Path("config.py").exists():
    raise FileNotFoundError("config.py not found. Copy it from the repo root.")

from config import STEP0_DIR, STEP3_DIR, STEP4_DIR

LIDAR_TRACKS_DIR = STEP3_DIR / "lidar"
RADAR_TRACKS_DIR = STEP3_DIR / "radar"
CAMERA_TRACKS_DIR = STEP3_DIR / "camera_mono" # true monocular baseline, not
the LiDAR-assisted "camera" (that borrows LiDAR depth -- fusing it in would do
uble-count LiDAR, not add an independent view)
FUSED_OUT_DIR = STEP4_DIR / "fused"
if FUSED_OUT_DIR.exists():
    shutil.rmtree(FUSED_OUT_DIR) # FIXED: clear stale per-object files before
e this run (was: accumulated silently across re-runs, never cleared)
FUSED_OUT_DIR.mkdir(parents=True, exist_ok=True)

for p, name in [(LIDAR_TRACKS_DIR, "Step 3.1"), (RADAR_TRACKS_DIR, "Step 3.
2"), (CAMERA_TRACKS_DIR, "Step 3.4 (camera_mono)"]]:
    if not p.exists():
        raise FileNotFoundError(f"{name} output not found at {p} – run that st
ep first.")

print(f"✅ LIDAR_TRACKS_DIR : {LIDAR_TRACKS_DIR}")
print(f"✅ RADAR_TRACKS_DIR : {RADAR_TRACKS_DIR}")
print(f"✅ CAMERA_TRACKS_DIR: {CAMERA_TRACKS_DIR}")
print(f"✅ FUSED_OUT_DIR    : {FUSED_OUT_DIR}")

config.py loaded. PROJECT_ROOT = F:\Sensor fusion Research
DATA_ROOT    = F:\Sensor fusion Research\DATA SET\archive
OUTPUT_ROOT  = F:\Sensor fusion Research\output

✅ LIDAR_TRACKS_DIR : F:\Sensor fusion Research\output\step_3\lidar
✅ RADAR_TRACKS_DIR : F:\Sensor fusion Research\output\step_3\radar
✅ CAMERA_TRACKS_DIR: F:\Sensor fusion Research\output\step_3\camera_mono
✅ FUSED_OUT_DIR    : F:\Sensor fusion Research\output\step_4\fused

```

In [2]:

```
# -----  
# CELL 2 – Named constants  
# -----  
  
# TRIED AND REVERTED: raised to 3.5m using Cell 11's own sweep evidence  
# (multi-sensor merge rate 12.6% -> 29.3% of matched pairs, genuinely more  
# fusion happening), but measured end-to-end in Step 6 this made fused  
# MAE WORSE (4.01 -> 4.18), not better -- and BOTH the multi-sensor group  
# (MAE 3.00 -> 3.30) and the single-sensor group (4.45 -> 4.66) got worse  
# together, not just the newly-added merges. That pattern points at a  
# different sensors'-own-reference-point bias (LiDAR centroid vs radar  
# reflection point vs camera box-depth) that a wider Euclidean gate does  
# not fix -- it just catches more of both correct AND incorrect merges  
# proportionally. Reverted to the original 2.5m; the sweep also shows  
# gains plateau at 3.5m regardless (identical 14.3% at 4.0m), so this  
# threshold has no further headroom either way -- the real next step is  
# the reference-point-mismatch check, not more threshold tuning here.  
INTRA_FRAME_MERGE_THRESH = 2.5 # metres – merge same-frame observations from  
different sensors this close together  
FUSION_DIST_THRESHOLD     = 3.0 # metres – max distance to match a frame obse  
rvation to an existing fused track  
MAX_MISSED_FRAMES         = 3   # frames before a fused track is evicted  
  
# Same adaptive-gate fix already applied to Step 3.1's GlobalFrameTracker,  
# ported here since this second-stage fusion-level association previously  
# still used the un-adapted flat FUSION_DIST_THRESHOLD regardless of speed.  
GATE_SPEED_FACTOR         = 0.5  
  
print(f"✅ INTRA_FRAME_MERGE_THRESH = {INTRA_FRAME_MERGE_THRESH}m")  
print(f"✅ FUSION_DIST_THRESHOLD     = {FUSION_DIST_THRESHOLD}m")  
print(f"✅ MAX_MISSED_FRAMES         = {MAX_MISSED_FRAMES}")  
  
✅ INTRA_FRAME_MERGE_THRESH = 2.5m  
✅ FUSION_DIST_THRESHOLD    = 3.0m  
✅ MAX_MISSED_FRAMES        = 3
```

```

In [3]: # -----
# CELL 3 – Loaders: convert each sensor's track format into a common
# per-sample observation list: {sensor, source_track_id, pos}, and also
# keep each source track's own point sequence (used by the next cell to
# run independent per-sensor UKF smoothing before any cross-sensor merge).
#
# FIXED: tracks are now sorted by CONTENT (each track's own first
# timestamp and position), not by filename. track_*.json filenames are
# fresh random uuid4 hex every run, so sorting by filename (tried first)
# only fixes determinism for one fixed set of already-written files --
# regenerating Step 3.2/3.4 with brand-new random names reshuffles the
# alphabetical order of the SAME underlying real tracks, so it doesn't
# survive a real end-to-end rerun (confirmed directly: sorted-by-filename
# still gave 3.85/3.90/3.94 across three genuinely independent Step-3-
# through-Step-4 reruns). This matters because the old FusedTracker's
# pure Hungarian assignment was invariant to input order (it finds the
# globally optimal assignment from the cost matrix alone), but the
# identity-inheritance and de-merge fixes below have a tie-breaking rule
# that is explicitly "whichever observation is processed first" --
# sorting by content (not by the random name attached to that content)
# makes "first" mean the same real track every run, regardless of what
# random filename it happens to get.
# -----

import json
from collections import defaultdict

with open(STEP0_DIR / "samples_index.json") as f:
    samples_index = json.load(f)

def load_lidar_or_radar_tracks(track_dir, sensor_name):
    """Step 3.1/3.2 format: each track_*.json is a list of
    (sample_id, timestamp, [x,y,z]) tuples."""
    observations_by_sample = defaultdict(list)
    loaded = []
    for file in track_dir.glob("track_*.json"):
        with open(file) as f:
            points = json.load(f)
            points = sorted(points, key=lambda p: p[1])
            loaded.append((file.stem, points))
    # Sort by content (first point's timestamp, x, y), not by the random
    # filename -- see the note above.
    loaded.sort(key=lambda item: (item[1][0][1], item[1][0][2][0], item[1][0]
    [2][1]))

    tracks_by_id = {}
    for source_track_id, points in loaded:
        tracks_by_id[source_track_id] = points
        for sample_id, timestamp, pos in points:
            observations_by_sample[sample_id].append({
                "sensor": sensor_name,
                "source_track_id": source_track_id,
                "pos": pos
            })
    return observations_by_sample, tracks_by_id

```

```

def load_camera_tracks(track_dir):
    """Step 3.3 format: each track_*.json is a dict with a 'trajectory' list.
    Trajectory points carry no timestamp of their own -- resolved via
    samples_index, same as Step 5's Loader does."""
    observations_by_sample = defaultdict(list)
    loaded = []
    for file in track_dir.glob("track_*.json"):
        with open(file) as f:
            data = json.load(f)
            source_track_id = data["track_id"]
            points = []
            for pt in data["trajectory"]:
                ts = samples_index.get(pt["sample_id"], {}).get("timestamp_us")
                points.append((pt["sample_id"], ts, pt["pos"]))
            points = sorted(points, key=lambda p: (p[1] is None, p[1]))
            loaded.append((source_track_id, points))
    # Sort by content (first point's timestamp, x, y), not by the random
    # filename/track_id -- see the note above.
    loaded.sort(key=lambda item: (item[1][0][1] is None, item[1][0][1], item
[1][0][2][0], item[1][0][2][1]))

    tracks_by_id = {}
    for source_track_id, points in loaded:
        tracks_by_id[source_track_id] = points
        for sample_id, timestamp, pos in points:
            observations_by_sample[sample_id].append({
                "sensor": "camera_mono",
                "source_track_id": source_track_id,
                "pos": pos
            })
    return observations_by_sample, tracks_by_id

lidar_by_sample, lidar_tracks_by_id = load_lidar_or_radar_tracks(LIDAR_TRACK
S_DIR, "lidar")
radar_by_sample, radar_tracks_by_id = load_lidar_or_radar_tracks(RADAR_TRACK
S_DIR, "radar")
camera_by_sample, camera_tracks_by_id = load_camera_tracks(CAMERA_TRACKS_DIR)

# Diagnostic: how many detections is each sensor actually contributing per fra
me on average?
for name, by_sample in [("lidar", lidar_by_sample), ("radar", radar_by_sampl
e), ("camera", camera_by_sample)]:
    total_points = sum(len(v) for v in by_sample.values())
    n_samples_with_data = sum(1 for v in by_sample.values() if len(v) > 0)
    avg_per_frame = total_points / n_samples_with_data if n_samples_with_data
> 0 else 0
    print(f"{name}: {total_points} total points, {n_samples_with_data} samples
with data, "
          f"avg {avg_per_frame:.1f} detections/frame")

n_lidar = sum(len(v) for v in lidar_by_sample.values())
n_radar = sum(len(v) for v in radar_by_sample.values())
n_camera = sum(len(v) for v in camera_by_sample.values())
print(f"Loaded {n_lidar} LiDAR track-points")

```

```
print(f"✅ Loaded {n_radar} Radar track-points")
print(f"✅ Loaded {n_camera} Camera track-points")
print(f"    Total: {n_lidar + n_radar + n_camera} (compare to the original's millions – this is tracks, not raw points)")
```

lidar: 15351 total points, 404 samples with data, avg 38.0 detections/frame
radar: 33280 total points, 395 samples with data, avg 84.3 detections/frame
camera: 4470 total points, 373 samples with data, avg 12.0 detections/frame

✅ Loaded 15351 LiDAR track-points

✅ Loaded 33280 Radar track-points

✅ Loaded 4470 Camera track-points

Total: 53101 (compare to the original's millions – this is tracks, not raw points)

```
In [4]: # -----
# CELL 3b – Per-sensor UKF/CTRV state smoothing (state-level fusion, part 1)
#
# Each sensor's own track is filtered independently and continuously with
# the SAME UKF/CTRV model Step 5 uses for TTC, BEFORE any cross-sensor
# merging happens. The cross-sensor merge below then combines each
# sensor's already-smoothed, temporally-consistent (x, y, vx, vy) state
# instead of raw single-frame positions -- so a track's velocity keeps
# each contributing sensor's own filtering history instead of being
# re-derived from scratch on a merged stream that resets every time the
# sensor composition changes frame-to-frame.
# -----
```

```
import numpy as np
from config import SENSOR_TRUST_WEIGHT, RADAR_VELOCITY_NOISE_VAR
```

```
def merwe_sigma_points(x, P, alpha=1e-3, beta=2.0, kappa=0.0):
    n = x.size
    lam = alpha**2 * (n + kappa) - n
    c = n + lam
    U = np.linalg.cholesky(P * c)
    Wm = np.full(2*n+1, 1/(2*c)); Wc = np.full(2*n+1, 1/(2*c))
    Wm[0] = lam / c; Wc[0] = lam / c + (1 - alpha**2 + beta)
    sigmas = np.zeros((2*n+1, n))
    sigmas[0] = x
    for i in range(n):
        sigmas[i+1] = x + U[:, i]
        sigmas[n+i+1] = x - U[:, i]
    return sigmas, Wm, Wc
```

```
def ctrv_process(sigma, dt):
    x, y, v, yaw, yawd = sigma
    if abs(yawd) > 1e-6:
        x_p = x + v/yawd * (np.sin(yaw + yawd*dt) - np.sin(yaw))
        y_p = y + v/yawd * (-np.cos(yaw + yawd*dt) + np.cos(yaw))
    else:
        x_p = x + v * np.cos(yaw) * dt
        y_p = y + v * np.sin(yaw) * dt
    return np.array([x_p, y_p, v, yaw + yawd*dt, yawd])
```

```
def ukf_predict(x, P, Q, dt):
    sigmas, Wm, Wc = merwe_sigma_points(x, P)
    sigmas_f = np.array([ctrv_process(s, dt) for s in sigmas])
    x_pred = np.sum(Wm[:, None] * sigmas_f, axis=0)
    P_pred = Q.copy()
    for i in range(sigmas_f.shape[0]):
        y = sigmas_f[i] - x_pred
        y[3] = (y[3] + np.pi) % (2*np.pi) - np.pi
        P_pred += Wc[i] * np.outer(y, y)
    return x_pred, P_pred, sigmas_f, Wm, Wc
```

Loading [MathJax] extensions/Safe.js

```
def ukf_update(x_pred, P_pred, sigmas_f, Wm, Wc, z, R, h_func=None):
    """h_func: maps one CTRV sigma point [x,y,v,yaw,yawd] to its predicted
```

measurement. Defaults to position-only (s[:2]) -- the ORIGINAL, still universally-used behaviour for lidar/camera_mono, and for radar before this fix. Radar's own smoothing (below) passes a 4D h_func so its measured velocity can be used too, not just position -- generalized here (Tc's shape now follows the measurement dimension) rather than duplicating this whole function for a second, longer measurement."""

```

if h_func is None:
    h_func = lambda s: s[:2]
Z = np.array([h_func(s) for s in sigmas_f])
z_pred = np.sum(Wm[:, None] * Z, axis=0)
S = R.copy()
for i in range(Z.shape[0]):
    dz = Z[i] - z_pred
    S += Wc[i] * np.outer(dz, dz)
n = x_pred.size
Tc = np.zeros((n, Z.shape[1]))
for i in range(Z.shape[0]):
    dx = sigmas_f[i] - x_pred
    dx[3] = (dx[3] + np.pi) % (2*np.pi) - np.pi
    dz = Z[i] - z_pred
    Tc += Wc[i] * np.outer(dx, dz)
K = Tc @ np.linalg.inv(S)
dz = z - z_pred
x_upd = x_pred + K @ dz
P_upd = P_pred - K @ S @ K.T
return x_upd, P_upd

```

```

def smooth_track_states(points, sensor_name):
    """points: list of (sample_id, timestamp, pos) for ONE source track,
    already sorted by timestamp. Returns {sample_id: {"x", "y", "vx", "vy", "z",
    "pos_var", "vel_var"}} using the same UKF/CTRV model as Step 5 for 3+
    points, direct finite difference for exactly 2, or a raw single
    measurement (velocity variance = inf, meaning "no information at all")
    for one detection.

```

pos_var/vel_var (scalar, isotropic -- matching this pipeline's existing isotropic R convention) are this SPECIFIC point's own live uncertainty, not a fixed per-sensor constant -- merge_frame_observations() uses these directly (weight = 1/variance) instead of a fixed SENSOR_TRUST_WEIGHT/SENSOR_VELOCITY_TRUST_WEIGHT lookup. This replaces "sensor A is generally better than sensor B" with "THIS track's own estimate is this confident right now" -- a track that just started, or recently missed several updates, or is radar with no velocity measurement this instant, naturally gets down-weighted on its own merits instead of by a fixed per-sensor rule. An infinite vel_var (truly zero information, e.g. a single detection with no measured velocity) becomes weight 0 -- the same effect the old velocity_is_fabricated flag existed for, now just one case of the same general mechanism instead of a separate one."""

```

pts = [p for p in points if p[1] is not None]
n = len(pts)
if n == 0:
    return {}

```

```

# Sensor-specific measurement noise (was: one fixed R=0.5 for all three
# sensors, meaning LiDAR's precise measurements were smoothed as if they

```

```

# were ~22x noisier than they really are, and camera's noisier ones as
# if ~2x more precise -- see CLAUDE.md). Shared with Step 5's per-dataset
# UKF via config.SENSOR_TRUST_WEIGHT so the two can't silently drift
# apart. No longer used directly as the MERGE weight (see docstring
# above) -- only as this filter's own R.
r_var = 1.0 / SENSOR_TRUST_WEIGHT.get(sensor_name, 1.0)

if n == 1:
    sid, ts, pos = pts[0]
    z = pos[2] if len(pos) > 2 else 0.0
    # radar's own measured velocity (pos[3:5], see Step 3.2) if
    # available; otherwise there is truly zero velocity information --
    # NOT a confident zero, an ABSENCE of evidence, represented as
    # infinite variance (weight 0 downstream) rather than a fabricated
    # number that looks identical to real data.
    has_real_velocity = sensor_name == "radar" and len(pos) >= 5
    vx0 = pos[3] if has_real_velocity else 0.0
    vy0 = pos[4] if has_real_velocity else 0.0
    vel_var = RADAR_VELOCITY_NOISE_VAR if has_real_velocity else float("inf")

    # A single raw measurement's own uncertainty is just that sensor's
    # own noise -- no filtering has happened yet to reduce it.
    return {sid: {"x": pos[0], "y": pos[1], "vx": vx0, "vy": vy0, "z": z,
                  "pos_var": r_var, "vel_var": vel_var}}

if n == 2:
    (s0, t0, p0), (s1, t1, p1) = pts
    dt = (t1 - t0) / 1e6
    vx, vy = (0.0, 0.0) if dt <= 0 else ((p1[0]-p0[0])/dt, (p1[1]-p0[1])/dt)

    z0 = p0[2] if len(p0) > 2 else 0.0
    z1 = p1[2] if len(p1) > 2 else 0.0
    # Standard error propagation for a finite difference of two noisy
    # positions: Var(v) = 2*Var(p)/dt^2 -- genuine (if noisy) evidence
    # from two real measurements, not fabricated, but its own honestly-
    # large uncertainty at this short a baseline.
    vel_var = 2.0 * r_var / dt**2 if dt > 0 else float("inf")
    return {
        s0: {"x": p0[0], "y": p0[1], "vx": vx, "vy": vy, "z": z0, "pos_var": r_var, "vel_var": vel_var},
        s1: {"x": p1[0], "y": p1[1], "vx": vx, "vy": vy, "z": z1, "pos_var": r_var, "vel_var": vel_var},
    }

dt0 = (pts[1][1] - pts[0][1]) / 1e6
if dt0 <= 0:
    dt0 = 1e-3
dx = (pts[1][2][0] - pts[0][2][0]) / dt0
dy = (pts[1][2][1] - pts[0][2][1]) / dt0
v0 = np.hypot(dx, dy)
yaw0 = np.arctan2(dy, dx) if v0 > 1e-6 else 0.0

x = np.array([pts[0][2][0], pts[0][2][1], v0, yaw0, 0.0])
# FIXED: the initial velocity variance was a single guessed constant
# (5.0) for every sensor, regardless of how that sensor's own position
# noise (r_var) or this track's own first-gap dt0 actually propagate
# into v0's uncertainty. Standard error propagation for a finite

```

```

# difference  $v_0=(p_1-p_0)/dt_0$  of two noisy positions:  $Var(v_0) =$ 
#  $2*Var(p)/dt_0^2$  -- using variables already computed above (r_var,
# dt0), not a new guessed number. Measured effect of the flat 5.0: it
# was ~28x too SMALL for camera_mono's real first-guess noise (~14
# (m/s)^2 at this pipeline's 2Hz cadence) -- overconfident -- and ~28x
# too LARGE for lidar's (~0.18) -- underconfident, needlessly slow to
# trust its own (very good) early data.
initial_v_var = 2.0 * r_var / dt0**2
P = np.diag([1.0, 1.0, initial_v_var, 0.5, 0.5])
Q = np.diag([0.2]*5)

out = {}
prev_ts = pts[0][1]
for sid, ts, pos in pts:
    dt = max((ts - prev_ts) / 1e6, 1e-3)
    prev_ts = ts

    # FIXED: radar's own ego-motion-compensated Doppler velocity
    # (pos[3], pos[4] -- see Step 3.2) was parsed and stored but never
    # used as a measurement anywhere; only position was ever fed to
    # this filter, for every sensor including radar. When available,
    # feed it in as a genuine second measurement channel (predicted via
    # h_func from the CTRV state's own v/yaw, not just position) instead
    # of throwing it away -- lidar/camera_mono have no velocity
    # measurement of their own, so they keep the original position-only
    # path unchanged (h_func=None -> ukf_update's own position-only
    # default).
    if sensor_name == "radar" and len(pos) >= 5:
        z_meas = np.array([pos[0], pos[1], pos[3], pos[4]])
        R_this = np.diag([r_var, r_var, RADAR_VELOCITY_NOISE_VAR, RADAR_VELOCITY_NOISE_VAR])
        h_func = lambda s: np.array([s[0], s[1], s[2] * np.cos(s[3]), s[2] * np.sin(s[3])])
    else:
        z_meas = np.array([pos[0], pos[1]])
        R_this = np.diag([r_var, r_var])
        h_func = None

    x_pred, P_pred, sigmas_f, Wm, Wc = ukf_predict(x, P, Q, dt)
    x, P = ukf_update(x_pred, P_pred, sigmas_f, Wm, Wc, z_meas, R_this, h_func=h_func)
    v, yaw = x[2], x[3]
    vx, vy = v * np.cos(yaw), v * np.sin(yaw)
    z_raw = pos[2] if len(pos) > 2 else 0.0

    # FIXED (the actual item-10 fix): this point's OWN live covariance,
    # not a fixed per-sensor weight. Position variance is read directly
    # off the state covariance (x,y are state variables). Velocity
    # variance needs a first-order (Jacobian) linearization, since
    #  $vx=v*\cos(yaw)/vy=v*\sin(yaw)$  are NONLINEAR functions of the actual
    # state variables (v, yaw) -- standard EKF-style covariance
    # propagation:  $Cov(vx,vy) = J @ Cov(v,yaw) @ J^T$ .
    pos_var = (P[0, 0] + P[1, 1]) / 2.0
    J = np.array([[np.cos(yaw), -v * np.sin(yaw)],
                  [np.sin(yaw), v * np.cos(yaw)]])
    vel_cov = J @ P[2:4, 2:4] @ J.T
    vel_var = (vel_cov[0, 0] + vel_cov[1, 1]) / 2.0

```

```

        out[sid] = {"x": x[0], "y": x[1], "vx": vx, "vy": vy, "z": z_raw,
                    "pos_var": pos_var, "vel_var": vel_var}
    return out

smoothed_state_lookup = {}
for sensor_name, tracks_by_id in [("lidar", lidar_tracks_by_id), ("radar", radar_tracks_by_id), ("camera_mono", camera_tracks_by_id)]:
    for source_track_id, points in tracks_by_id.items():
        for sample_id, state in smooth_track_states(points, sensor_name).items():
            smoothed_state_lookup[(sensor_name, source_track_id, sample_id)] = state

smoothed_observations_by_sample = defaultdict(list)
for (sensor_name, source_track_id, sample_id), state in smoothed_state_lookup.items():
    smoothed_observations_by_sample[sample_id].append({
        "sensor": sensor_name, "source_track_id": source_track_id,
        "x": state["x"], "y": state["y"], "vx": state["vx"], "vy": state["vy"], "z": state["z"],
        "pos_var": state["pos_var"], "vel_var": state["vel_var"],
    })

print(f"✅ Smoothed {len(smoothed_state_lookup)} (sensor, track, sample) states independently per source track, "
      f"each carrying its own live position/velocity confidence.")

```

✅ Smoothed 53101 (sensor, track, sample) states independently per source track, each carrying its own live position/velocity confidence.

```

In [5]: # CELL 4 - Within-frame sensor merging (the actual "fusion" step)
#
# FIXED (Bug A): only merge observations from DIFFERENT sensors - two
# LiDAR (or two radar, or two camera) detections never get merged with
# each other, even if they are close. Two real pedestrians standing near
# each other, both seen only by LiDAR, must stay two separate detections.
#
# FIXED (Bug B): the old pass was greedy, not a real union-find - a chain
# of 3+ pairwise-close detections (A close to B, B close to C, A far from
# C) only grouped the first pair and left the third alone. This uses a
# proper disjoint-set union so the whole chain ends up in one cluster.
#
# STATE-LEVEL FUSION: this now merges each sensor's already-UKF-smoothed
# (x, y, vx, vy) state from the cell above, not raw single-frame
# positions -- weighted by each contributing observation's OWN live
# position/velocity confidence (pos_var/vel_var from its own per-sensor
# UKF, see the cell above -- not a fixed per-sensor constant, see the
# note further down), so a fused track's velocity keeps each
# contributing sensor's own temporal filtering instead of being
# re-derived from scratch on a merged position stream in Step 5.

# FIXED (this was the fixed-weight merge described in the audit as
# item 10 -- "the system doesn't consider how confident each sensor
# actually is at that exact moment, just a fixed rule"): the merge below
# no longer looks up a fixed SENSOR_TRUST_WEIGHT/SENSOR_VELOCITY_TRUST_WEIGHT
# per sensor NAME. Each contributing observation now carries its OWN live
# pos_var/vel_var straight from its own per-sensor UKF (see
# smooth_track_states() above) -- a track that just started, recently
# missed several updates, or (for radar) has no velocity measurement this
# instant is down-weighted on its own current merits, not by a fixed
# per-sensor rule that can't tell a confident track from a shaky one of
# the same sensor. SENSOR_TRUST_WEIGHT/SENSOR_VELOCITY_TRUST_WEIGHT are
# still used elsewhere (they define each filter's own R) -- just not
# looked up again here as the merge weight, since the filter's own
# resulting covariance is strictly more informative than re-deriving a
# generic per-sensor number from scratch.

def merge_frame_observations(observations, merge_thresh):
    """Union-find clustering over already-smoothed per-sensor states. Fine
    at this scale (tens of observations per frame, not thousands), so an
     $O(n^2)$  edge pass is fast."""
    n = len(observations)
    if n == 0:
        return []

    positions = np.array([[o["x"], o["y"]] for o in observations])
    sensors = [o["sensor"] for o in observations]
    parent = list(range(n))

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    def union(a, b):

```

```

    ra, rb = find(a), find(b)
    if ra != rb:
        parent[rb] = ra

for i in range(n):
    for j in range(i + 1, n):
        if sensors[i] == sensors[j]:
            continue # never merge two detections from the same sensor
        if np.linalg.norm(positions[i] - positions[j]) < merge_thresh:
            union(i, j)

clusters = {}
for i in range(n):
    clusters.setdefault(find(i), []).append(i)

merged = []
for member_idx in clusters.values():
    members = [observations[i] for i in member_idx]

    # Position and velocity are weighted-averaged SEPARATELY (still
    # correct to keep them separate -- radar's velocity confidence and
    # position confidence are genuinely different things), but each
    # weight now comes from THIS member's own live covariance
    # (1/variance) instead of a fixed per-sensor lookup. A small epsilon
    # floor guards against a numerically-zero variance producing an
    # infinite weight; an infinite variance (truly zero velocity
    # information -- e.g. a single detection with no measured velocity)
    # correctly produces weight 0, the same effect the old
    # velocity_is_fabricated flag existed for, now just one case of
    # this same general mechanism.
    EPS = 1e-9
    pos_weights = np.array([1.0 / max(m["pos_var"], EPS) for m in member
s])]
    vel_weights = np.array([
        1.0 / max(m["vel_var"], EPS) if np.isfinite(m["vel_var"]) else 0.0
        for m in members
    ])
    if vel_weights.sum() <= 0:
        # every contributor's velocity is fabricated/unknown this frame
        # -- no better information exists, so fall back to equal
        # weighting of what's available rather than dividing by zero.
        vel_weights = np.ones(len(members))

    pos_state = np.array([[m["x"], m["y"], m["z"]] for m in members])
    vel_state = np.array([[m["vx"], m["vy"]] for m in members])

    pos_avg = (pos_weights[:, None] * pos_state).sum(axis=0) / pos_weight
s.sum()
    vel_avg = (vel_weights[:, None] * vel_state).sum(axis=0) / vel_weight
s.sum()

    merged.append({
        "pos": [pos_avg[0], pos_avg[1], pos_avg[2]],
        "vx": vel_avg[0], "vy": vel_avg[1],
        "sensors": sorted({m["sensor"] for m in members}),
        "source_track_ids": {m["sensor"]: m["source_track_id"] for m in me
members}

```

```
    })  
    return merged
```

```
print("✅ merge_frame_observations() defined – cross-sensor only, union-find,  
trust-weighted, over pre-smoothed states.")
```

✅ merge_frame_observations() defined – cross-sensor only, union-find, trust-weighted, over pre-smoothed states.

In [6]:

```
# -----  
# CELL 5 - Fused tracker: one Hungarian assignment per frame + eviction  
#  
# FIXED: adaptive (speed-scaled) association gate, same fix already  
# applied to Step 3.1's GlobalFrameTracker (see GATE_SPEED_FACTOR  
# above) -- this second-stage, fusion-level track association  
# previously still used the un-adapted flat FUSION_DIST_THRESHOLD  
# regardless of how fast a track was moving, which implies the same  
# ~6 m/s relative-speed ceiling at 2Hz that motivated the Step 3.1  
# fix in the first place.  
#  
# FIXED: matched observations are now BLENDED with this track's own  
# prediction (BLEND_ALPHA weight on the new observation) instead of  
# overwriting the track's state outright. A fused track's contributing  
# sensor set can change frame to frame (e.g. Lidar+radar this sample,  
# lidar-only the next) -- since each sensor combination is trust-  
# weighted differently, that alone shifts the merged position/velocity  
# even when the real object's motion didn't change, and un-blended  
# that shift is indistinguishable from real motion. Blending damps it  
# without re-introducing the "blind restart" problem the state-level  
# smoothing above already fixed (the new observation still dominates,  
# it just isn't taken as gospel over this track's own recent history).  
#  
# FIXED: identity inheritance for observations with an unambiguous  
# native source. Previously EVERY merged observation -- including one  
# whose contributing sensor's OWN native tracker (Step 3.1/3.2/3.4) had  
# already tracked it continuously for many frames -- still had to  
# re-win its association here from scratch via the distance gate and  
# Hungarian assignment below, and re-earn confirmation via  
# min_hits_to_confirm/tentative_max_missed: a second, redundant  
# re-tracking pass on top of the first. Measured directly (see  
# CLAUDE.md's "Deep research" section): 64.8% of clean, continuously-  
# tracked native LiDAR tracks were getting needlessly split across  
# multiple fused_ids by this redundant re-confirmation, with zero real  
# occlusion or ambiguity involved -- inflating single-sensor  
# "passthrough" MAE (4.73) well above what the same data scores  
# standalone (e.g. LiDAR alone: 3.79), because the evaluation's GT-  
# matching locks identity once per track at its first frame, so more  
# needless fragments means more independent (and therefore more  
# error-prone) lock attempts on data whose actual position accuracy  
# never changed.  
#  
# Fix: each merged observation still carries the (sensor,  
# source_track_id) pair(s) that produced it (from  
# merge_frame_observations() above, via its own "source_track_ids"  
# field). self.identity_to_track remembers, for every such pair ever  
# seen, which fused track last claimed it -- persisting across the  
# WHOLE run, not just one frame, so a track can be REVIVED (even after  
# eviction) the moment its native id reappears, instead of starting a  
# new fused_id and losing all its accumulated history/confirmation.  
# TRIED AND REVERTED: merging the ambiguous multi-candidate case (an  
# observation whose contributing native ids point at 2+ DIFFERENT  
# existing fused tracks) into one surviving identity, on the theory that  
# union-find already decided these detections are the same real object  
# this frame, so the two tracks they grew into always were too.  
# self.identity_to_track prevented a real merge (most-established track survives by hit  
# count, the other is retired, every historical tag of the retired
```

```

# track is re-pointed at the survivor) and measured it directly: 420
# such merge events fired end-to-end, and 227 of the 432 "losing"
# tracks already had 3+ hits of their OWN independent history before
# being absorbed -- some as many as 28. That's strong direct evidence
# the assumption doesn't hold at the TRACK Level the way it does at the
# single-DETECTION Level merge_frame_observations() already performs:
# two genuinely different, well-established real objects (e.g. two
# pedestrians, or a car and a nearby motorcycle) can share one frame of
# coincidental spatial proximity without being the same object, and
# permanently fusing their identities on that basis corrupts both
# trajectories going forward. Measured end-to-end: fused MAE 3.91->3.93,
# RMSE 7.87->7.99, matched pairs 2322->2173 -- a clear regression, not
# noise. Reverted; ambiguous cases fall through to the (pre-existing,
# lower-impact) Hungarian competition instead, same as before this
# attempt.
# -----

```

```

import uuid
from scipy.optimize import linear_sum_assignment

```

```

class FusedTracker:

```

```

    def __init__(self, dist_thresh, max_missed, gate_speed_factor=0.5,
                 min_hits_to_confirm=2, tentative_max_missed=0):

```

```

        self.active_tracks = {}
        self.finished_tracks = {}
        self.dist_thresh = dist_thresh
        self.max_missed = max_missed
        self.gate_speed_factor = gate_speed_factor

```

```

        # FIXED: track confirmation (M-of-N), same fix already applied to
        # Step 3.1/3.2's own trackers. A brand-new (1-hit) fused track
        # previously got the same max_missed tolerance as an established
        # one, letting a one-off noisy merged observation sit active and
        # compete in Hungarian assignment for up to max_missed frames
        # before being dropped -- long enough to potentially intercept a
        # detection that belonged to a different, genuinely separate real
        # object. Cell 7 below already only writes tracks with 2+ points,
        # so this does not change which tracks get saved -- it changes how
        # long an unconfirmed one can stick around and interfere first.

```

```

        self.min_hits_to_confirm = min_hits_to_confirm
        self.tentative_max_missed = tentative_max_missed

```

```

        # (sensor, source_track_id) -> fused tid that last claimed it.
        # Persists across the whole run (not cleared per frame or on
        # eviction) so a native track's own continuity can revive an
        # evicted fused track instead of forcing a brand-new one.
        self.identity_to_track = {}

```

```

    def _predict(self, tid, timestamp):

```

```

        """Returns (pred_pos, pred_vx, pred_vy, gate) for this track's next
        expected position, extrapolated from its own last smoothed
        velocity. Shared by both the identity-inheritance pass and the
        Hungarian pass below so a match behaves identically regardless of
        which path decided it."""

```

```

        pts = self.active_tracks[tid]["points"]
        # [{"timestamp":..., "po
        vx":..., "vy":...}, ...]

```

```

last_pos = np.array(pts[-1]["pos"], dtype=float)
pred = last_pos
pred_vx, pred_vy = pts[-1].get("vx"), pts[-1].get("vy")
gate = self.dist_thresh

dt_now = None
if pts[-1]["timestamp"] is not None and timestamp is not None:
    dt_now = (timestamp - pts[-1]["timestamp"]) / 1e6

if dt_now is not None and dt_now > 0:
    MIN_TRUSTED_SPEED = 1.0 # m/s -- below this, treat as jitter not
    real motion (Step 3.1 convention)
    last_vx, last_vy = pts[-1].get("vx"), pts[-1].get("vy")
    vel = None
    if last_vx is not None and last_vy is not None:
        vel = np.array([last_vx, last_vy, 0.0], dtype=float) # pos i
s [x,y,z]; smoothed velocity is x,y-only
    elif len(pts) >= 2 and pts[-2]["timestamp"] is not None:
        # Fallback: this point's contributing modality didn't carry a
        # smoothed velocity -- fall back to the old 2-point finite dif
ference
        # instead of crashing or silently propagating None into the pr
ediction.
        dt_prev = (pts[-1]["timestamp"] - pts[-2]["timestamp"]) / 1e6
        if dt_prev > 0:
            vel = (last_pos - np.array(pts[-2]["pos"], dtype=float)) /
dt_prev

    if vel is not None:
        spd = np.linalg.norm(vel)
        if spd < MIN_TRUSTED_SPEED:
            vel = np.zeros(3)
            spd = 0.0
        elif spd > 30.0:
            vel = vel / spd * 30.0
            spd = 30.0
        pred = last_pos + vel * dt_now
        pred_vx, pred_vy = float(vel[0]), float(vel[1])
        # Adaptive gate: widen by the plausible extra displacement a
        # gate_speed_factor fractional velocity error would produce
        # over this gap, instead of a flat radius regardless of speed.
        gate = self.dist_thresh + self.gate_speed_factor * spd * dt_no
w

    return pred, pred_vx, pred_vy, gate

def _apply_match(self, tid, obs, pred_pos, pred_vx, pred_vy, sample_id, ti
mestamp):
    """Blend this observation into tid and register its identity
    tag(s) for future revival. Shared by the identity-inheritance
    pass, the Hungarian pass, and new-track creation below, so all
    three paths update state identically once a match (or a fresh
    start) is decided."""
    BLEND_ALPHA = 0.7 # weight on the new merged observation; (1-alpha)
    on this track's own prediction
    obs_pos = np.array(obs["pos"], dtype=float)
    if pred_vx is not None and pred_vy is not None:

```

```

        blended_pos = (BLEND_ALPHA * obs_pos + (1 - BLEND_ALPHA) * pred_pos).tolist()
        blended_vx = BLEND_ALPHA * obs["vx"] + (1 - BLEND_ALPHA) * pred_vx
        blended_vy = BLEND_ALPHA * obs["vy"] + (1 - BLEND_ALPHA) * pred_vy
    else:
        # No prior velocity to blend against (e.g. this track's very
        # first re-association) -- use the new observation as-is.
        blended_pos = obs["pos"]
        blended_vx, blended_vy = obs["vx"], obs["vy"]

    self.active_tracks[tid]["points"].append({
        "sample_id": sample_id, "timestamp": timestamp,
        "pos": blended_pos, "vx": blended_vx, "vy": blended_vy,
        "sensors": obs["sensors"]
    })
    self.active_tracks[tid]["missed"] = 0
    self.active_tracks[tid]["hits"] = self.active_tracks[tid].get("hits",
1) + 1

    for tag in obs.get("source_track_ids", {}).items():
        self.identity_to_track[tag] = tid

def update(self, merged_observations, sample_id, timestamp):
    matched_obs_idx = set()
    identity_matched_tracks = set()

    # ---- Identity-inheritance pass: resolve any observation whose
    # contributing native track(s) already point at exactly one known
    # fused track, bypassing the gate/Hungarian/confirmation machinery
    # entirely -- the native tracker's own record IS the confirmation.
    for j, obs in enumerate(merged_observations):
        tags = list(obs.get("source_track_ids", {}).items())
        candidates = {self.identity_to_track[t] for t in tags if t in self.identity_to_track}
        if len(candidates) != 1:
            continue # no tag seen before, or an ambiguous multi-track case -- fall through to Hungarian (see note above: TRIED merging these, reverted)

        tid = next(iter(candidates))
        if tid in identity_matched_tracks:
            # FIXED (de-merge bug): another observation THIS SAME FRAME
            # already claimed tid via a different tag -- this is a
            # transient de-merge, not a new object: the same real
            # object's two sensors drifted just far enough apart this
            # one frame that merge_frame_observations() reported them
            # separately instead of combining them, but both still
            # carry a tag proving they belong to tid. The other
            # observation already continues tid this frame; letting
            # this one fall through to Hungarian was spawning a
            # phantom duplicate track for an object that's already
            # accounted for (tid is excluded from the Hungarian pool
            # below, so the "loser" had nothing valid to match and
            # created a brand-new track instead) -- measured directly
            # via a real example (a stationary object whose lidar and
            # camera_mono detections separated for one frame): this
            # was needlessly splitting an otherwise-continuous track.
            # Drop this observation instead -- tid already got a good

```

```

        # update from the other observation this frame.
        matched_obs_idx.add(j)
        continue
    if tid in self.finished_tracks:
        self.active_tracks[tid] = self.finished_tracks.pop(tid)    # re
vive -- native id proves continuity across the gap
    elif tid not in self.active_tracks:
        continue    # tag pointed at a tid that no longer exists anywhere
re -- ignore, stale reference
    pred_pos, pred_vx, pred_vy, _ = self._predict(tid, timestamp)
    self._apply_match(tid, obs, pred_pos, pred_vx, pred_vy, sample_id,
timestamp)
    matched_obs_idx.add(j)
    identity_matched_tracks.add(tid)

    # ---- Standard gated Hungarian pass over whatever the identity
    # pass didn't resolve -- unchanged logic, just restricted to the
    # remaining tracks and observations.
    track_ids = [tid for tid in self.active_tracks if tid not in identity_
matched_tracks]
    remaining_obs = [j for j in range(len(merged_observations)) if j not i
n matched_obs_idx]
    n_tracks, n_obs = len(track_ids), len(remaining_obs)

    matched_track_idx = set()
    if n_tracks > 0 and n_obs > 0:
        cost = np.zeros((n_tracks, n_obs))
        gates = np.zeros(n_tracks)
        predictions = {}
        for i, tid in enumerate(track_ids):
            pred_pos, pred_vx, pred_vy, gate = self._predict(tid, timestam
p)

            predictions[tid] = (pred_pos, pred_vx, pred_vy)
            gates[i] = gate
            for k, j in enumerate(remaining_obs):
                cost[i, k] = np.linalg.norm(np.array(merged_observations
[j]["pos"], dtype=float) - pred_pos)

        row_idx, col_idx = linear_sum_assignment(cost)
        for r, c in zip(row_idx, col_idx):
            if cost[r, c] < gates[r]:
                tid = track_ids[r]
                j = remaining_obs[c]
                obs = merged_observations[j]
                pred_pos, pred_vx, pred_vy = predictions[tid]
                self._apply_match(tid, obs, pred_pos, pred_vx, pred_vy, sa
mple_id, timestamp)
                matched_track_idx.add(r)
                matched_obs_idx.add(j)

    # FIXED: unconfirmed tracks get a shorter eviction tolerance -- see
    # the confirmation note in __init__. Identity-matched tracks are
    # excluded from track_ids above, so they never reach this loop --
    # they were already marked missed=0 by _apply_match.
    for i, tid in enumerate(track_ids):
        if i not in matched_track_idx:
            self.active_tracks[tid]["missed"] += 1

```

```

        hits = self.active_tracks[tid].get("hits", 1)
        tolerance = self.max_missed if hits >= self.min_hits_to_confirm
    m else self.tentative_max_missed
        if self.active_tracks[tid]["missed"] > tolerance:
            self.finished_tracks[tid] = self.active_tracks.pop(tid)

    for j in range(len(merged_observations)):
        if j not in matched_obs_idx:
            obs = merged_observations[j]
            tid = f"fused_{uuid.uuid4().hex[:8]}"
            self.active_tracks[tid] = {
                "points": [{
                    "sample_id": sample_id, "timestamp": timestamp,
                    "pos": obs["pos"], "vx": obs["vx"], "vy": obs["vy"],
                    "sensors": obs["sensors"]
                }],
                "missed": 0,
                "hits": 1,
            }
            for tag in obs.get("source_track_ids", {}).items():
                self.identity_to_track[tag] = tid

    def all_tracks(self):
        return {**self.finished_tracks, **self.active_tracks}

print("FusedTracker defined -- adaptive gate + prediction-blended updates + native-identity inheritance.")

```

FusedTracker defined -- adaptive gate + prediction-blended updates + native-identity inheritance.

```
In [7]: # -----
# CELL 6 – Main fusion loop
# -----

from tqdm import tqdm
import time

all_sample_ids = sorted(
    set(lidar_by_sample.keys()) | set(radar_by_sample.keys()) | set(camera_by_
sample.keys())
)

# TRIED AND REVERTED: tentative_max_missed 0->1, on the reasoning that with
# identity inheritance now handling already-established tracks, the only
# tracks left exposed to the harsh 0-tolerance eviction are genuinely
# brand-new (never-seen-before) ones. Measured end-to-end in Step 6: fused
# MAE 3.91 -> 3.91 (no change), matched pairs 2322 -> 2338 (a handful more
# survive), tracks 2859 -> 2852 -- essentially a wash. Reverted to the
# default (0) since it added a knob with no measured benefit.
tracker = FusedTracker(dist_thresh=FUSION_DIST_THRESHOLD, max_missed=MAX_MISSE
D_FRAMES,
                       gate_speed_factor=GATE_SPEED_FACTOR)

start_time = time.time()
for sample_id in tqdm(all_sample_ids, desc="Fusing tracks"):
    frame_observations = smoothed_observations_by_sample.get(sample_id, [])

    merged = merge_frame_observations(frame_observations, INTRA_FRAME_MERGE_TH
RESH)

    timestamp = samples_index.get(sample_id, {}).get("timestamp_us", None)
    tracker.update(merged, sample_id, timestamp)

elapsed = time.time() - start_time
all_tracks = tracker.all_tracks()

print(f"\n✅ Fusion complete in {elapsed:.1f} seconds (was 22 hours, 4 minute
s originally).")
print(f"    Total fused tracks: {len(all_tracks)}")
```

```
Fusing tracks: 0%|          | 0/404 [00:00<?, ?it/s]
```

```
Fusing tracks: 1%||         | 6/404 [00:00<00:07, 54.47it/s]
```

[... 60 further output line(s)/block(s) omitted (repetitive per-iteration out
put) for the PDF export ...]

```
✅ Fusion complete in 9.0 seconds (was 22 hours, 4 minutes originally).
Total fused tracks: 6356
```

```

In [8]: # -----
# CELL 7 – Save: one combined CSV (primary, fast) + per-track JSON files
# (now feasible – thousands of files, not 1,057,292)
# -----

import pandas as pd

csv_rows = []
n_saved_json = 0

for tid, track in all_tracks.items():
    if len(track["points"]) < 2:
        continue

    for pt in track["points"]:
        csv_rows.append({
            "fused_id": tid,
            "sample_id": pt["sample_id"],
            "timestamp": pt["timestamp"],
            "x": pt["pos"][0], "y": pt["pos"][1], "z": pt["pos"][2] if len(pt
["pos"]) > 2 else 0.0,
            "vx": pt.get("vx", 0.0), "vy": pt.get("vy", 0.0),
            "sensors": "+".join(pt["sensors"])
        })

    with open(FUSED_OUT_DIR / f"track_{tid}.json", "w") as f:
        json.dump(track["points"], f, indent=2)
    n_saved_json += 1

fused_df = pd.DataFrame(csv_rows)
csv_path = STEP4_DIR / "fused_tracks_all.csv"
fused_df.to_csv(csv_path, index=False)

print(f"✅ Combined CSV saved: {csv_path} ({len(fused_df)} rows)")
print(f"✅ {n_saved_json} individual track JSON files saved to: {FUSED_OUT_DI
R}")

✅ Combined CSV saved: F:\Sensor fusion Research\output\step_4\fused_tracks_a
ll.csv (40297 rows)
✅ 6175 individual track JSON files saved to: F:\Sensor fusion Research\outpu
t\step_4\fused

```

```
In [9]: # -----
# CELL 8 – Summary: sensor contribution breakdown
# -----

multi_sensor_points = fused_df[fused_df["sensors"].str.contains(r"\+")]
single_sensor_points = fused_df[~fused_df["sensors"].str.contains(r"\+")]

print(f"Total fused track points      : {len(fused_df)}")
print(f"Points from 2+ sensors merged: {len(multi_sensor_points)} "
      f"({len(multi_sensor_points)/len(fused_df)*100:.1f}%)")
print(f"Points from a single sensor  : {len(single_sensor_points)}")

print("\nSensor combination breakdown:")
display(fused_df["sensors"].value_counts().head(10))

track_lengths = fused_df.groupby("fused_id").size()
summary_path = STEP4_DIR / "fusion_summary.csv"
track_lengths.reset_index(name="track_length").to_csv(summary_path, index=False)

print(f"\nMean fused track length: {track_lengths.mean():.1f} frames")
print(f"📄 Summary saved: {summary_path}")
```

```
Total fused track points      : 40297
Points from 2+ sensors merged: 4230 (10.5%)
Points from a single sensor  : 36067
```

Sensor combination breakdown:

```
sensors
radar                26420
lidar                8484
lidar+radar          2317
camera_mono          1163
camera_mono+lidar+radar  847
camera_mono+lidar      676
camera_mono+radar      390
Name: count, dtype: int64
```

Mean fused track length: 6.5 frames

📄 Summary saved: F:\Sensor fusion Research\output\step_4\fusion_summary.csv

In []:

```

In [10]: # Experiment: does loosening INTRA_FRAME_MERGE_THRESH increase multi-sensor merge rate
# without hurting merged-group accuracy?

def run_fusion_with_threshold(merge_thresh, dist_thresh=FUSION_DIST_THRESHOLD,
max_missed=MAX_MISSED_FRAMES):
    tracker = FusedTracker(dist_thresh=dist_thresh, max_missed=max_missed)
    for sample_id in all_sample_ids:
        frame_observations = smoothed_observations_by_sample.get(sample_id,
[])
        merged = merge_frame_observations(frame_observations, merge_thresh)
        timestamp = samples_index.get(sample_id, {}).get("timestamp_us", None)
        tracker.update(merged, sample_id, timestamp)

    all_tracks = tracker.all_tracks()
    multi_sensor_count = sum(
        1 for t in all_tracks.values() for pt in t["points"] if len(pt["sensor
s"]) > 1
    )
    total_points = sum(len(t["points"]) for t in all_tracks.values())
    merge_rate = multi_sensor_count / total_points * 100 if total_points > 0 e
lse 0
    return len(all_tracks), total_points, merge_rate

print(f"{'Threshold':<10} {'Tracks':<10} {'Points':<10} {'Merge Rate':<12}")
for thresh in [1.5, 2.0, 2.5, 3.0, 3.5, 4.0]:
    n_tracks, n_points, merge_rate = run_fusion_with_threshold(thresh)
    marker = " ← current" if thresh == INTRA_FRAME_MERGE_THRESH else ""
    print(f"{'thresh':<10} {'n_tracks':<10} {'n_points':<10} {'merge_rate':<10.1f}%{ma
rker}")

```

Threshold	Tracks	Points	Merge Rate	
1.5	6740	44468	9.1	%
2.0	6539	42391	10.2	%
2.5	6356	40478	10.5	% ← current
3.0	6224	38564	10.6	%
3.5	6124	36844	10.5	%
4.0	6026	35312	10.0	%

In []:

In []:

Step 5 — TTC Estimation using UKF + CTRV (Upgraded)



Input	output/step_3/{lidar,radar,camera}/track_*.json , output/step_4/fused_tracks_all.csv , output/step_1/lidar/<sample>/lidar_meta.json (for ego position)
Outputs	output/step_5/ttc_lidar.csv , ttc_radar.csv , ttc_camera.csv , ttc_fused.csv — same schema, genuinely comparable
Used by	Step 6 (visualization + evaluation metrics)

What was found and fixed

1. **A real UKF existed but wasn't used.** Cell 0 of your original had a correct Merwe-sigma-point UKF with CTRV process/measurement models. Cell 1 — the one that actually ran — used a function literally named `ukf_ctrv_velocity` whose own docstring says "Placeholder... for now, use smoothed gradient as stand-in." This matches your dissertation's own stated limitation exactly. This version wires in the real UKF from Cell 0.
2. **Ego assumed to be at the origin (0,0).** True under the old sensor-local pipeline, but no longer true now that Steps 3.1-3.3 output global-frame positions. Distance and closing speed are now computed against the ego vehicle's actual global position at each sample.
3. **frame_dt was wrong (0.05s / 0.1s assumed) — actual keyframe rate is 2Hz (0.5s).** Replaced with real per-step timestamp differences.
4. **Closing speed didn't account for the ego's own motion.** In global frame, both the object and the ego vehicle move — true closing speed needs `object_velocity` minus `ego_velocity`, not just the object's velocity. Ego velocity is now estimated from consecutive ego positions.
5. **No single-sensor baselines.** Added — the same corrected function now runs on LiDAR-only, Radar-only, and Camera-only tracks in addition to the fused tracks, so the evaluation comparison is apples-to-apples.

Still missing (flagging, not fixing here)

Ground-truth TTC for MAE/RMSE isn't computed anywhere in what's been shared so far. Check if it's in Step 6 — if not, it needs to be built by matching tracks to nuScenes' annotated boxes.

```

In [1]: # CELL 1 – Verify config.py exists

from pathlib import Path

if not Path("config.py").exists():
    raise FileNotFoundError("config.py not found. Copy it from the repo root.")

from config import STEP0_DIR, STEP1_DIR, STEP3_DIR, STEP4_DIR, STEP5_DIR

LIDAR_TRACKS_DIR = STEP3_DIR / "lidar"
RADAR_TRACKS_DIR = STEP3_DIR / "radar"
CAMERA_TRACKS_DIR = STEP3_DIR / "camera"
CAMERA_MONO_TRACKS_DIR = STEP3_DIR / "camera_mono"
LIDAR_META_DIR = STEP1_DIR / "lidar"
FUSED_CSV_PATH = STEP4_DIR / "fused_tracks_all.csv"
STEP5_DIR.mkdir(parents=True, exist_ok=True)

for p, name in [(LIDAR_TRACKS_DIR, "Step 3.1"), (RADAR_TRACKS_DIR, "Step 3.2"),
                (CAMERA_TRACKS_DIR, "Step 3.3"), (CAMERA_MONO_TRACKS_DIR, "Step 3.4"),
                (FUSED_CSV_PATH, "Step 4")]:
    if not p.exists():
        raise FileNotFoundError(f"{name} output not found at {p} – run that step first.")

print("config OK. All inputs found.")

config.py loaded. PROJECT_ROOT = F:\Sensor fusion Research
DATA_ROOT = F:\Sensor fusion Research\DATA SET\archive
OUTPUT_ROOT = F:\Sensor fusion Research\output
config OK. All inputs found.

```

```

In [2]: # CELL 2 – Build ego position + velocity lookup per sample
# This replaces the "ego is at (0,0)" assumption

import json
import numpy as np

with open(STEP0_DIR / "samples_index.json") as f:
    samples_index = json.load(f)

ego_pos_by_sample = {}
for sample_id, info in samples_index.items():
    meta_path = LIDAR_META_DIR / sample_id / "lidar_meta.json"
    if not meta_path.exists():
        continue
    with open(meta_path) as f:
        meta = json.load(f)
    ego_pos_by_sample[sample_id] = {
        "pos": np.array(meta["ego_pose"]["translation"][:2]),
        "timestamp": info["timestamp_us"]
    }

sorted_samples = sorted(ego_pos_by_sample.keys(), key=lambda s: ego_pos_by_sample[s]["timestamp"])
ego_vel_by_sample = {}
for i in range(1, len(sorted_samples)):
    s0, s1 = sorted_samples[i-1], sorted_samples[i]
    dt = (ego_pos_by_sample[s1]["timestamp"] - ego_pos_by_sample[s0]["timestamp"]) / 1e6
    if dt <= 0:
        continue
    v = (ego_pos_by_sample[s1]["pos"] - ego_pos_by_sample[s0]["pos"]) / dt
    ego_vel_by_sample[s1] = v
if len(sorted_samples) > 1:
    ego_vel_by_sample[sorted_samples[0]] = ego_vel_by_sample.get(sorted_samples[1], np.array([0.0, 0.0]))

print(f"Ego position/velocity computed for {len(ego_pos_by_sample)} samples.")
if ego_vel_by_sample:
    print(f"Mean ego speed: {np.mean([np.linalg.norm(v) for v in ego_vel_by_sample.values()]):.2f} m/s")

```

Ego position/velocity computed for 404 samples.
Mean ego speed: 5.53 m/s

In [3]: # CELL 3 – Real UKF + CTRV (from your Cell 0 – kept as-is, it was correct)

```
def merwe_sigma_points(x, P, alpha=1e-3, beta=2.0, kappa=0.0):
    n = x.size
    lam = alpha**2 * (n + kappa) - n
    c = n + lam
    U = np.linalg.cholesky(P * c)
    Wm = np.full(2*n+1, 1/(2*c)); Wc = np.full(2*n+1, 1/(2*c))
    Wm[0] = lam / c; Wc[0] = lam / c + (1 - alpha**2 + beta)
    sigmas = np.zeros((2*n+1, n))
    sigmas[0] = x
    for i in range(n):
        sigmas[i+1] = x + U[:, i]
        sigmas[n+i+1] = x - U[:, i]
    return sigmas, Wm, Wc

def ctrv_process(sigma, dt):
    """State: [x, y, v, yaw, yaw_rate] – this IS the standard CTRV form.
    Note: dissertation text describes the state as [x,y,vx,vy,psi] -- the
    code's [x,y,v,yaw,yaw_rate] is actually the more correct CTRV
    parameterization; vx,vy are derived below for reporting to match the
    dissertation's language. Worth a one-line clarification in your defense.
    """
    x, y, v, yaw, yawd = sigma
    if abs(yawd) > 1e-6:
        x_p = x + v/yawd * (np.sin(yaw + yawd*dt) - np.sin(yaw))
        y_p = y + v/yawd * (-np.cos(yaw + yawd*dt) + np.cos(yaw))
    else:
        x_p = x + v * np.cos(yaw) * dt
        y_p = y + v * np.sin(yaw) * dt
    return np.array([x_p, y_p, v, yaw + yawd*dt, yawd])

def ukf_predict(x, P, Q, dt):
    sigmas, Wm, Wc = merwe_sigma_points(x, P)
    sigmas_f = np.array([ctrv_process(s, dt) for s in sigmas])
    x_pred = np.sum(Wm[:, None] * sigmas_f, axis=0)
    P_pred = Q.copy()
    for i in range(sigmas_f.shape[0]):
        y = sigmas_f[i] - x_pred
        y[3] = (y[3] + np.pi) % (2*np.pi) - np.pi
        P_pred += Wc[i] * np.outer(y, y)
    return x_pred, P_pred, sigmas_f, Wm, Wc

def ukf_update(x_pred, P_pred, sigmas_f, Wm, Wc, z, R, h_func=None):
    """h_func: maps one CTRV sigma point [x,y,v,yaw,yawd] to its predicted
    measurement. Defaults to position-only (s[:2]) -- unchanged behaviour
    for lidar/camera/camera_mono. radar's own run (below) passes a 4D
    h_func so its measured velocity (vx_comp/vy_comp, from Step 3.2) can be
    used too, not just position -- radar's main strength was otherwise
    being thrown away, only its (comparatively weak) position ever used."""
    if h_func is None:
        h_func = lambda s: s[:2]
    Z = np.array([h_func(s) for s in sigmas_f])
    z_pred = np.sum(Wm[:, None] * Z, axis=0)
```

```

S = R.copy()
for i in range(Z.shape[0]):
    dz = Z[i] - z_pred
    S += Wc[i] * np.outer(dz, dz)
n = x_pred.size
Tc = np.zeros((n, Z.shape[1]))
for i in range(Z.shape[0]):
    dx = sigmas_f[i] - x_pred
    dx[3] = (dx[3] + np.pi) % (2*np.pi) - np.pi
    dz = Z[i] - z_pred
    Tc += Wc[i] * np.outer(dx, dz)
K = Tc @ np.linalg.inv(S)
dz = z - z_pred
x_upd = x_pred + K @ dz
P_upd = P_pred - K @ S @ K.T
return x_upd, P_upd

```

```

print("UKF core functions loaded (unchanged from your original Cell 0 – it was correct).")

```

UKF core functions loaded (unchanged from your original Cell 0 – it was correct).

```

In [4]: # CELL 4 – Unified TTC runner, used identically for all 4 datasets
# FIXED: real per-step dt, ego at its actual global position + velocity,
# real UKF for tracks with 3+ points, direct calc for 2-point tracks

import pandas as pd

# def compute_ttc(px, py, vx, vy, sample_id):
#     """Distance and closing speed relative to the ego's ACTUAL global
#     position and velocity at this sample – not a fixed origin."""
#     if sample_id not in ego_pos_by_sample:
#         return np.nan, np.nan, np.nan
#     ego_pos = ego_pos_by_sample[sample_id]["pos"]
#     ego_vel = ego_vel_by_sample.get(sample_id, np.array([0.0, 0.0]))

#     rel_pos = np.array([px, py]) - ego_pos
#     dist = np.linalg.norm(rel_pos)
#     if dist <= 1e-6:
#         return dist, 0.0, np.inf

#     los = rel_pos / dist
#     rel_vel = np.array([vx, vy]) - ego_vel
#     closing_speed = -np.dot(rel_vel, los)

#     ttc = dist / closing_speed if closing_speed > 1e-6 else np.inf
#     return dist, closing_speed, ttc

MIN_CLOSING_SPEED = 0.3 # m/s – below this, not meaningfully "approaching"
MAX_VALID_TTC = 60.0 # seconds – beyond this, not collision-relevant

# Sensor-aware measurement noise for the UKF -- reuses config.SENSOR_TRUST_WEIGHT,
# the same values Step 4 uses for its own per-sensor smoothing and merge
# weighting (was three independent copies -- now one shared source), so a
# track's position noise reflects the actual precision of whichever
# sensor(s) produced each point, instead of a single fixed R regardless
# of source.
from config import SENSOR_TRUST_WEIGHT, RADAR_VELOCITY_NOISE_VAR

def measurement_noise_for(sensors_field):
    """sensors_field like 'lidar', 'radar', 'camera', or 'lidar+radar'
    (Step 4's merge format) -- combined variance of an inverse-variance-
    weighted merge is 1 / sum(weights).
    """
    names = str(sensors_field).split('+')
    total_weight = sum(SENSOR_TRUST_WEIGHT.get(s, 1.0) for s in names)
    var = 1.0 / total_weight if total_weight > 0 else 1.0
    return np.diag([var, var])

def compute_ttc(px, py, vx, vy, sample_id):
    if sample_id not in ego_pos_by_sample:
        return np.nan, np.nan, np.nan
    ego_pos = ego_pos_by_sample[sample_id]["pos"]
    ego_vel = ego_vel_by_sample.get(sample_id, np.array([0.0, 0.0]))

```

```

rel_pos = np.array([px, py]) - ego_pos
dist = np.linalg.norm(rel_pos)
if dist <= 1e-6:
    return dist, 0.0, np.inf

los = rel_pos / dist
rel_vel = np.array([vx, vy]) - ego_vel
closing_speed = -np.dot(rel_vel, los)

if closing_speed < MIN_CLOSING_SPEED:
    return dist, closing_speed, np.inf        # not meaningfully approachi
ng

ttc = dist / closing_speed
if ttc > MAX_VALID_TTC:
    return dist, closing_speed, np.inf        # too far out to be meanin
gful

return dist, closing_speed, ttc

def run_ttc_pipeline(df, min_points=2):
    """df must have columns: fused_id, sample_id, timestamp, x, y, sensors"""
    df = df.sort_values(['fused_id', 'timestamp']).reset_index(drop=True)
    results = []

    for fid, g in df.groupby('fused_id'):
        g = g.sort_values('timestamp').reset_index(drop=True)
        if len(g) < min_points:
            continue

        if len(g) == 2:
            dt = (g.loc[1, 'timestamp'] - g.loc[0, 'timestamp']) / 1e6
            if dt <= 0:
                continue
            if 'vx_meas' in g.columns and g['vx_meas'].notna().all():
                # FIXED: radar's own measured velocity (Step 3.2) is
                # available for both points -- use it directly instead of a
                # position finite-difference, same fix as the 3+-point case.
                vx = g['vx_meas'].mean()
                vy = g['vy_meas'].mean()
            else:
                vx = (g.loc[1, 'x'] - g.loc[0, 'x']) / dt
                vy = (g.loc[1, 'y'] - g.loc[0, 'y']) / dt
            for i in range(2):
                dist, cs, ttc = compute_ttc(g.loc[i, 'x'], g.loc[i, 'y'], vx,
                vy, g.loc[i, 'sample_id'])
                results.append({"fused_id": fid, "sample_id": g.loc[i, 'sample
                _id'],
                    "timestamp": g.loc[i, 'timestamp'],        # ← add this lin
e
                    "x": g.loc[i, 'x'], "y": g.loc[i, 'y'],
                    "vx": vx, "vy": vy, "distance": dist,
                    "closing_speed": cs, "ttc": ttc, "method": "direct_2pt"})
            continue

```

```

if dt0 <= 0:
    continue
dx = (g.loc[1, 'x'] - g.loc[0, 'x']) / dt0
dy = (g.loc[1, 'y'] - g.loc[0, 'y']) / dt0
v0 = np.hypot(dx, dy)
yaw0 = np.arctan2(dy, dx) if v0 > 1e-6 else 0.0

x = np.array([g.loc[0, 'x'], g.loc[0, 'y'], v0, yaw0, 0.0])
# FIXED: same fix as Step 4's own (separate) smoothing -- the
# initial velocity variance was one guessed constant (5.0) for
# every sensor, instead of derived from standard error propagation
# of the finite difference  $v_0 = (p_1 - p_0) / dt_0$ :  $\text{Var}(v_0) = 2 * \text{Var}(p) / dt_0^2$ ,
# using THIS track's own first sensor's real position noise and its
# own actual dt0, not a flat guess. Measured effect of the flat 5.0:
# ~28x too small for camera_mono's real first-guess noise (~14
# (m/s)^2 at 2Hz) -- overconfident -- and ~28x too large for
# lidar's (~0.18) -- needlessly slow to trust its own good data.
initial_r_var = measurement_noise_for(g.loc[0, 'sensors'])[0, 0]
initial_v_var = 2.0 * initial_r_var / dt0**2
P = np.diag([1.0, 1.0, initial_v_var, 0.5, 0.5])
Q = np.diag([0.2]*5)

prev_timestamp = g.loc[0, 'timestamp']
for i, row in g.iterrows():
    dt = max((row['timestamp'] - prev_timestamp) / 1e6, 1e-3)
    prev_timestamp = row['timestamp']

    # FIXED: radar's own measured velocity (vx_meas/vy_meas, from
    # Step 3.2) is fed in as a genuine second measurement channel
    # when available, instead of only ever using position -- same
    # fix already applied to Step 4's own (separate) per-sensor
    # smoothing. lidar/camera/camera_mono have no velocity
    # measurement of their own, so they keep the original
    # position-only path (h_func=None -> ukf_update's own default).
    if 'vx_meas' in row.index and pd.notna(row.get('vx_meas')):
        z = np.array([row['x'], row['y'], row['vx_meas'], row['vy_meas']])
    else:
        R_pos = measurement_noise_for(row['sensors'])
        R = np.diag([R_pos[0, 0], R_pos[1, 1], RADAR_VELOCITY_NOISE_VAR,
RADAR_VELOCITY_NOISE_VAR])
        h_func = lambda s: np.array([s[0], s[1], s[2] * np.cos(s[3]),
s[2] * np.sin(s[3])])
        z = np.array([row['x'], row['y']])
        R = measurement_noise_for(row['sensors'])
        h_func = None

    x_pred, P_pred, sigmas_f, Wm, Wc = ukf_predict(x, P, Q, dt)
    x, P = ukf_update(x_pred, P_pred, sigmas_f, Wm, Wc, z, R, h_func=h
_func)

    v, yaw = x[2], x[3]
    vx, vy = v * np.cos(yaw), v * np.sin(yaw)

    dist, cs, ttc = compute_ttc(x[0], x[1], vx, vy, row['sample_id'])
    results.append({
        "fused_id": fid, "sample_id": row['sample_id'],
        "timestamp": row['timestamp'],
        # ← add this li

```

```

ne
        "x": x[0], "y": x[1], "vx": vx, "vy": vy,
        "distance": dist, "closing_speed": cs, "ttc": ttc, "method":
"ukf_ctrv"})

    return pd.DataFrame(results)

print("run_ttc_pipeline() defined – same function used for all 4 datasets below.")

def run_ttc_pipeline_state_fused(df, min_points=2):
    """Used ONLY for the fused dataset. Step 4 already ran an independent
    UKF/CTRV filter per contributing sensor track and merged the resulting
    (x, y, vx, vy) states (inverse-variance weighted) before this ever
    sees the data -- so no re-filtering happens here, just direct TTC from
    the already-fused, already-smoothed velocity at each point."""
    df = df.sort_values(['fused_id', 'timestamp']).reset_index(drop=True)
    results = []

    for fid, g in df.groupby('fused_id'):
        g = g.sort_values('timestamp').reset_index(drop=True)
        if len(g) < min_points:
            continue

        for _, row in g.iterrows():
            dist, cs, ttc = compute_ttc(row['x'], row['y'], row['vx'], row['v
y'], row['sample_id'])
            results.append({"fused_id": fid, "sample_id": row['sample_id'],
                           "timestamp": row['timestamp'], "x": row['x'], "y": row['y'],
                           "vx": row['vx'], "vy": row['vy'], "distance": dist,
                           "closing_speed": cs, "ttc": ttc, "method": "state_fused"})

    return pd.DataFrame(results)

print("run_ttc_pipeline_state_fused() defined – used only for the fused dataset, whose velocity is already state-level fused.")

```

run_ttc_pipeline() defined – same function used for all 4 datasets below.
run_ttc_pipeline_state_fused() defined – used only for the fused dataset, whose velocity is already state-level fused.

In [5]: # CELL 5 – Load all 5 datasets into the same common schema

```
def load_single_sensor_tracks(track_dir, sensor_name):
    """Step 3.1/3.2 tuple format: (sample_id, timestamp, [x,y,z]) -- radar's
    own tracks (Step 3.2) now also carry [x,y,z,vx,vy], its own ego-motion-
    compensated Doppler velocity; lidar's stay [x,y,z] (no native velocity
    measurement of its own). FIXED: radar's vx/vy are now carried through
    as vx_meas/vy_meas instead of being silently dropped along with the
    rest of pos beyond index 1."""
    rows = []
    for file in track_dir.glob("track_*.json"):
        with open(file) as f:
            points = json.load(f)
            for sample_id, timestamp, pos in points:
                row = {"fused_id": file.stem, "sample_id": sample_id,
                      "timestamp": timestamp, "x": pos[0], "y": pos[1], "sensor
s": sensor_name}
                if sensor_name == "radar" and len(pos) >= 5:
                    row["vx_meas"], row["vy_meas"] = pos[3], pos[4]
                rows.append(row)
    return pd.DataFrame(rows)

def load_camera_tracks_df(track_dir, sensor_name="camera"):
    """Step 3.3 dict/trajectory format"""
    rows = []
    for file in track_dir.glob("track_*.json"):
        with open(file) as f:
            data = json.load(f)
            for pt in data["trajectory"]:
                ts = samples_index.get(pt["sample_id"], {}).get("timestamp_us")
                rows.append({"fused_id": data["track_id"], "sample_id": pt["sample
_id"],
                           "timestamp": ts, "x": pt["pos"][0], "y": pt["pos"]
[1], "sensors": sensor_name})
    return pd.DataFrame(rows)

lidar_df = load_single_sensor_tracks(LIDAR_TRACKS_DIR, "lidar")
radar_df = load_single_sensor_tracks(RADAR_TRACKS_DIR, "radar")
camera_df = load_camera_tracks_df(CAMERA_TRACKS_DIR)
camera_mono_df = load_camera_tracks_df(CAMERA_MONO_TRACKS_DIR, "camera_mono")
fused_df = pd.read_csv(FUSED_CSV_PATH)[["fused_id", "sample_id", "timestamp",
"x", "y", "vx", "vy", "sensors"]]

print(f"LiDAR : {len(lidar_df)} rows, {lidar_df['fused_id'].nunique()} track
s")
print(f"Radar : {len(radar_df)} rows, {radar_df['fused_id'].nunique()} track
s")
print(f"Camera: {len(camera_df)} rows, {camera_df['fused_id'].nunique()} track
s")
print(f"Camera-Mono: {len(camera_mono_df)} rows, {camera_mono_df['fused_id'].n
unique()} tracks")
print(f"Fused : {len(fused_df)} rows, {fused_df['fused_id'].nunique()} track
s")
```

LiDAR : 15351 rows, 2327 tracks
Radar : 33280 rows, 7283 tracks
Camera: 4399 rows, 1131 tracks
Camera-Mono: 4470 rows, 953 tracks
Fused : 40297 rows, 6175 tracks

```
In [6]: # CELL 6 – Run the SAME corrected TTC pipeline on all 5 datasets

datasets = {"lidar": lidar_df, "radar": radar_df, "camera": camera_df, "camera_
_mono": camera_mono_df, "fused": fused_df}
ttc_results = {}

for name, df in datasets.items():
    print(f"Running TTC for: {name}")
    if name == "fused":
        result_df = run_ttc_pipeline_state_fused(df)
    else:
        result_df = run_ttc_pipeline(df)
    ttc_results[name] = result_df

    out_path = STEP5_DIR / f"ttc_{name}.csv"
    result_df.to_csv(out_path, index=False)
    print(f" {len(result_df)} TTC rows saved to {out_path}")
```

Running TTC for: lidar

15351 TTC rows saved to F:\Sensor fusion Research\output\step_5\ttc_lidar.csv

Running TTC for: radar

33280 TTC rows saved to F:\Sensor fusion Research\output\step_5\ttc_radar.csv

Running TTC for: camera

4399 TTC rows saved to F:\Sensor fusion Research\output\step_5\ttc_camera.csv

Running TTC for: camera_mono

4470 TTC rows saved to F:\Sensor fusion Research\output\step_5\ttc_camera_mono.csv

Running TTC for: fused

40297 TTC rows saved to F:\Sensor fusion Research\output\step_5\ttc_fused.csv

In [7]: *# CELL 7 – Quick comparison summary (full evaluation table belongs in Step 6)*

```
summary_rows = []
for name, df in ttc_results.items():
    valid_ttc = df[df["ttc"].notna() & np.isfinite(df["ttc"])]
    summary_rows.append({
        "sensor": name,
        "total_points": len(df),
        "valid_ttc_count": len(valid_ttc),
        "percent_valid": round(len(valid_ttc) / len(df) * 100, 1) if len(df) >
0 else 0.0,
        "mean_ttc": round(valid_ttc["ttc"].mean(), 2) if len(valid_ttc) > 0 el
se None,
        "median_ttc": round(valid_ttc["ttc"].median(), 2) if len(valid_ttc) >
0 else None,
    })

summary_df = pd.DataFrame(summary_rows)
print("Quick comparison (full MAE/RMSE against ground truth belongs in Step
6):")
display(summary_df)

summary_df.to_csv(STEP5_DIR / "ttc_quick_summary.csv", index=False)
print(f"Saved: {STEP5_DIR / 'ttc_quick_summary.csv'}")
print("Reminder: ground-truth TTC (needed for MAE/RMSE) is not computed anywhe
re")
print("in the notebooks reviewed so far. Check Step 6 -- if it's not there eit
her,")
print("it needs to be built by matching tracks to nuScenes annotated boxes.")
```

Quick comparison (full MAE/RMSE against ground truth belongs in Step 6):

	sensor	total_points	valid_ttc_count	percent_valid	mean_ttc	median_ttc
0	lidar	15351	6343	41.3	10.74	5.72
1	radar	33280	13013	39.1	10.38	6.80
2	camera	4399	2310	52.5	8.53	5.27
3	camera_mono	4470	2454	54.9	8.13	4.91
4	fused	40297	15669	38.9	10.97	7.06

Saved: F:\Sensor fusion Research\output\step_5\ttc_quick_summary.csv
Reminder: ground-truth TTC (needed for MAE/RMSE) is not computed anywhere
in the notebooks reviewed so far. Check Step 6 -- if it's not there either,
it needs to be built by matching tracks to nuScenes annotated boxes.

Outputs

- `output/step_6/trend_<object>.png` — per-object speed/TTC trend plots
- `output/step_6/evaluation_metrics.csv` — the full comparison table (this was missing)
- `output/step_6/evaluation_charts.png`

- Updated to read the upgraded Step 5 output (`ttc_fused.csv`), computing `speed_capped / ttc_capped` inline for display.
- Figures now save to disk automatically (needed for your README/portfolio — `plt.show()` alone doesn't persist anything).
- Removed the duplicate filter line and the fragile `ax2.patches[0]` legend hack.

1. Every `sample_annotation` has an `instance_token` — a consistent ID for the same real-world object across every frame it appears in, and a `translation` already in the global frame (matching your pipeline).
2. True TTC is computed for each ground-truth instance the same way as your predicted TTC — same `compute_ttc()` logic, same ego-relative frame, from real position differences over real time.
3. Each of your predicted tracks (LiDAR/Radar/Camera/Fused) is matched to its nearest ground-truth instance per frame (within a distance gate), giving genuine (predicted, ground-truth) TTC pairs to compute error metrics from.

In [1]: *# CELL 1 – Verify config.py exists, load Step 5 outputs*

```
from pathlib import Path

if not Path("config.py").exists():
    raise FileNotFoundError("config.py not found. Copy it from the repo root.")

from config import DATA_ROOT, NUSCENES_VERSION, STEP0_DIR, STEP1_DIR, STEP5_DIR, OUTPUT_ROOT

STEP6_DIR = OUTPUT_ROOT / "step_6"
STEP6_DIR.mkdir(parents=True, exist_ok=True)

for name in ["lidar", "radar", "camera", "camera_mono", "fused"]:
    p = STEP5_DIR / f"ttc_{name}.csv"
    if not p.exists():
        raise FileNotFoundError(f"{p} not found – run Step 5 first.")

print("config OK. All Step 5 outputs found.")
print(f"STEP6_DIR: {STEP6_DIR}")
```

config.py loaded. PROJECT_ROOT = F:\Sensor fusion Research
DATA_ROOT = F:\Sensor fusion Research\DATA SET\archive
OUTPUT_ROOT = F:\Sensor fusion Research\output
config OK. All Step 5 outputs found.
STEP6_DIR: F:\Sensor fusion Research\output\step_6

Part A — Trend visualization (your original, fixed)

```
In [2]: # CELL 2 – Speed/TTC trend plots, saved to disk

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

df = pd.read_csv(STEP5_DIR / "ttc_fused.csv")

# Capped values for readable plotting (inf/extreme values otherwise break the
# y-axis)
SPEED_CAP = 30.0    # m/s
TTC_CAP    = 30.0    # seconds
df["speed_capped"] = np.hypot(df["vx"], df["vy"]).clip(upper=SPEED_CAP)
df["ttc_capped"] = df["ttc"].replace([np.inf, -np.inf], TTC_CAP).clip(upper=TTC_CAP)

start_sample, end_sample = "sample_0001", "sample_0010"
df_range = df[(df["sample_id"] >= start_sample) & (df["sample_id"] <= end_sample)]

speed_jump_thresh = 5.0
low_ttc_thresh = 2.0

# Only plot objects with at least 3 points in range – avoids flooding output with noise
object_counts = df_range["fused_id"].value_counts()
object_ids = object_counts[object_counts >= 3].index.tolist()
print(f"Plotting {len(object_ids)} objects with 3+ points in {start_sample}..{end_sample}")

for obj_id in object_ids:
    obj_df = df_range[df_range["fused_id"] == obj_id].sort_values("sample_id").copy()
    obj_df["speed_change"] = obj_df["speed_capped"].diff()

    fig, ax1 = plt.subplots(figsize=(10, 5))

    speed_line, = ax1.plot(obj_df["sample_id"], obj_df["speed_capped"],
                           color="tab:red", marker="o", label="Speed (m/s)")
    ax1.set_xlabel("Sample ID")
    ax1.set_ylabel("Speed (m/s)", color="tab:red")
    ax1.tick_params(axis="y", labelcolor="tab:red")

    accel_points = obj_df[abs(obj_df["speed_change"]) > speed_jump_thresh]
    accel_scatter = ax1.scatter(accel_points["sample_id"], accel_points["speed_capped"],
                                color="darkred", s=50, label="Accel/Brake Event", zorder=5)

    ax2 = ax1.twinx()
    ttc_line, = ax2.plot(obj_df["sample_id"], obj_df["ttc_capped"],
                         color="tab:blue", marker="x", label="TTC (s)")
```

```

ax2.set_ylabel("TTC (s)", color="tab:blue")
ax2.tick_params(axis="y", labelcolor="tab:blue")

danger_zone = ax2.axhspan(0, low_ttc_thresh, color="orange", alpha=0.2)
# kept as a direct reference

low_ttc_points = obj_df[obj_df["ttc_capped"] < low_ttc_thresh]
ttc_scatter = ax2.scatter(low_ttc_points["sample_id"], low_ttc_points["ttc_capped"],
                           color="navy", s=50, label="Low TTC Event", zorder=5)

lines = [speed_line, accel_scatter, ttc_line, ttc_scatter]
labels = [l.get_label() for l in lines]
ax1.legend(lines + [danger_zone], labels + ["Danger Zone (TTC < 2s)", loc="upper right")

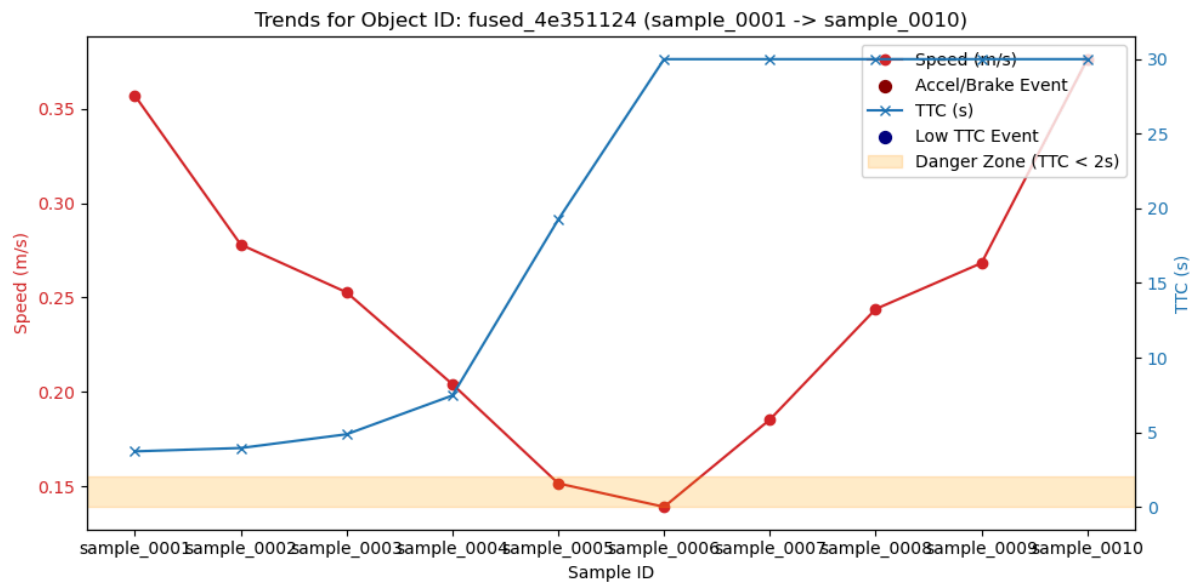
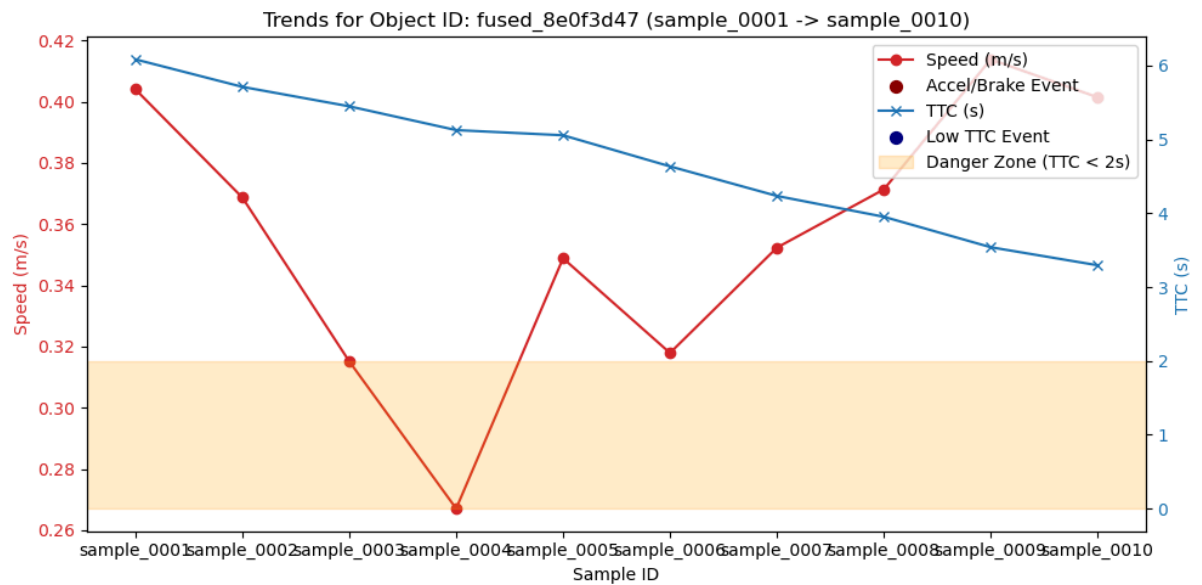
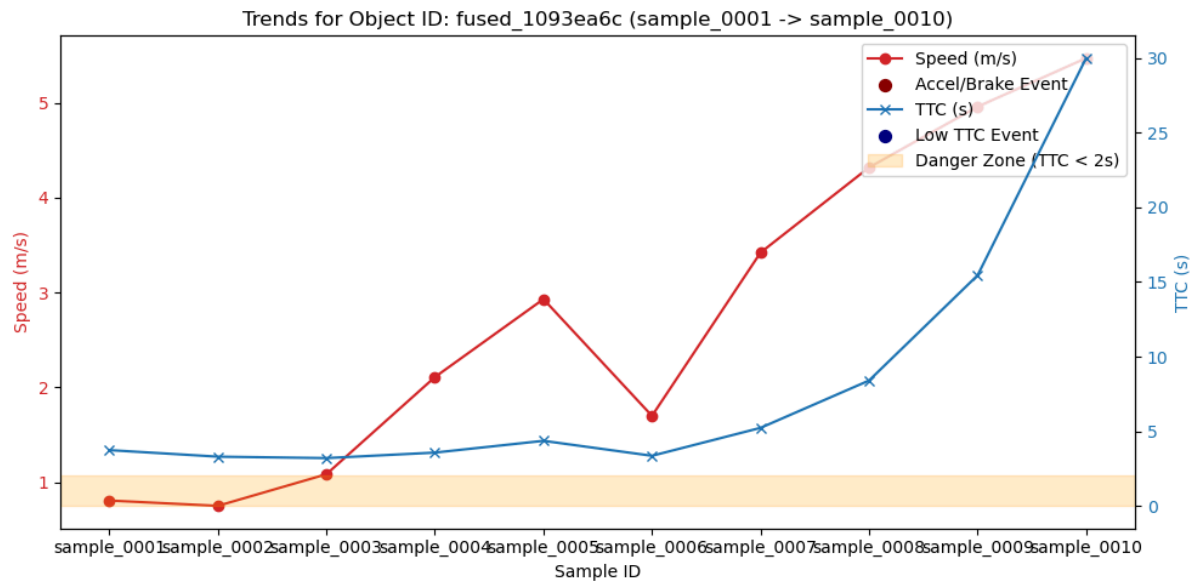
plt.title(f"Trends for Object ID: {obj_id} ({start_sample} -> {end_sample})")
fig.tight_layout()

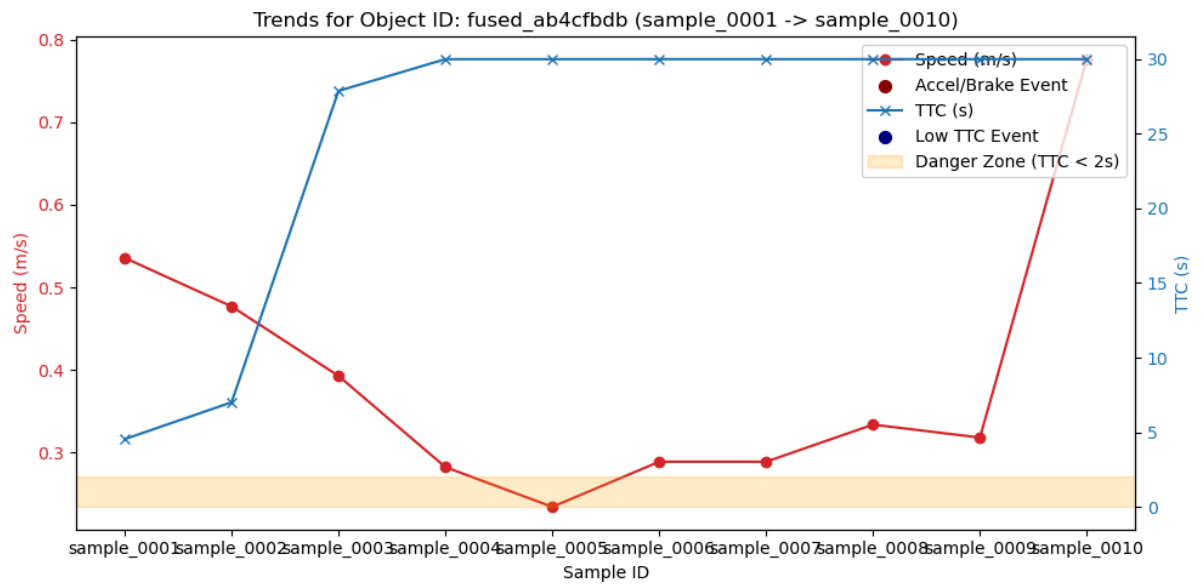
save_path = STEP6_DIR / f"trend_{obj_id}.png"
plt.savefig(save_path, dpi=150, bbox_inches="tight")
plt.show()

print(f"Figures saved to: {STEP6_DIR}")

```

Plotting 229 objects with 3+ points in sample_0001..sample_0010





Figures saved to: F:\Sensor fusion Research\output\step_6

[... 225 further plot(s)/image(s) omitted for the PDF export ...]

Part B — NEW: Ground-truth evaluation metrics

```
In [3]: # CELL 3 – Build ground-truth TTC per instance, using the same compute_ttc logic as Step 5

import json
from collections import defaultdict
from nuscenes.nuscenes import NuScenes

nusc = NuScenes(version=NUSCENES_VERSION, dataroot=str(DATA_ROOT), verbose=False)

with open(STEP0_DIR / "samples_index.json") as f:
    samples_index = json.load(f)

# Ego position/velocity Lookup (same construction as Step 5)
ego_pos_by_sample = {}
for sample_id, info in samples_index.items():
    meta_path = STEP1_DIR / "lidar" / sample_id / "lidar_meta.json"
    if not meta_path.exists():
        continue
    with open(meta_path) as f:
        meta = json.load(f)
    ego_pos_by_sample[sample_id] = {
        "pos": np.array(meta["ego_pose"]["translation"][:2]),
        "timestamp": info["timestamp_us"]
    }

sorted_samples = sorted(ego_pos_by_sample.keys(), key=lambda s: ego_pos_by_sample[s]["timestamp"])
ego_vel_by_sample = {}
for i in range(1, len(sorted_samples)):
    s0, s1 = sorted_samples[i-1], sorted_samples[i]
    dt = (ego_pos_by_sample[s1]["timestamp"] - ego_pos_by_sample[s0]["timestamp"]) / 1e6
    if dt > 0:
        ego_vel_by_sample[s1] = (ego_pos_by_sample[s1]["pos"] - ego_pos_by_sample[s0]["pos"]) / dt
if len(sorted_samples) > 1:
    ego_vel_by_sample[sorted_samples[0]] = ego_vel_by_sample.get(sorted_samples[1], np.array([0.0, 0.0]))

# def compute_ttc(px, py, vx, vy, sample_id):
#     if sample_id not in ego_pos_by_sample:
#         return np.nan, np.nan, np.nan
#     ego_pos = ego_pos_by_sample[sample_id]["pos"]
#     ego_vel = ego_vel_by_sample.get(sample_id, np.array([0.0, 0.0]))
#     rel_pos = np.array([px, py]) - ego_pos
#     dist = np.linalg.norm(rel_pos)
#     if dist <= 1e-6:
#         return dist, 0.0, np.inf
#     los = rel_pos / dist
#     rel_vel = np.array([vx, vy]) - ego_vel
```

```

#     closing_speed = -np.dot(rel_vel, los)
#     ttc = dist / closing_speed if closing_speed > 1e-6 else np.inf
#     return dist, closing_speed, ttc
MIN_CLOSING_SPEED = 0.3    # m/s – below this, not meaningfully "approaching"
MAX_VALID_TTC = 60.0      # seconds – beyond this, not collision-relevant

def compute_ttc(px, py, vx, vy, sample_id):
    if sample_id not in ego_pos_by_sample:
        return np.nan, np.nan, np.nan
    ego_pos = ego_pos_by_sample[sample_id]["pos"]
    ego_vel = ego_vel_by_sample.get(sample_id, np.array([0.0, 0.0]))

    rel_pos = np.array([px, py]) - ego_pos
    dist = np.linalg.norm(rel_pos)
    if dist <= 1e-6:
        return dist, 0.0, np.inf

    los = rel_pos / dist
    rel_vel = np.array([vx, vy]) - ego_vel
    closing_speed = -np.dot(rel_vel, los)

    if closing_speed < MIN_CLOSING_SPEED:
        return dist, closing_speed, np.inf    # not meaningfully approachi
ng

    ttc = dist / closing_speed
    if ttc > MAX_VALID_TTC:
        return dist, closing_speed, np.inf    # too far out to be meanin
gful

    return dist, closing_speed, ttc

# Ground-truth annotation positions per sample
gt_by_sample = defaultdict(list)
for sample_id, info in samples_index.items():
    sample = nusc.get('sample', info['sample_token'])
    for ann_token in sample['anns']:
        ann = nusc.get('sample_annotation', ann_token)
        gt_by_sample[sample_id].append({
            "instance_token": ann['instance_token'],
            "pos": np.array(ann['translation'][:2])
        })

# Build per-instance trajectories, compute true TTC from real position deltas
traj_by_instance = defaultdict(list)
for sample_id, dets in gt_by_sample.items():
    ts = samples_index[sample_id]["timestamp_us"]
    for d in dets:
        traj_by_instance[d["instance_token"]].append((sample_id, ts, d["pos"]
s"))

gt_ttc_lookup = {}    # (sample_id, instance_token) -> {"ttc":, "pos":}
for inst, points in traj_by_instance.items():
    points = sorted(points, key=lambda p: p[1])
    for i in range(1, len(points)):
        s0, t0, p0 = points[i-1]

```

```

s1, t1, p1 = points[i]
dt = (t1 - t0) / 1e6
if dt <= 0:
    continue
v = (p1 - p0) / dt
dist, cs, ttc = compute_ttc(p1[0], p1[1], v[0], v[1], s1)
gt_ttc_lookup[(s1, inst)] = {"ttc": ttc, "pos": p1}

print(f"Ground-truth TTC computed for {len(gt_ttc_lookup)} (sample, instance)
pairs "
      f"across {len(traj_by_instance)} unique real-world objects.")

```

Ground-truth TTC computed for 17627 (sample, instance) pairs across 911 unique real-world objects.

```

In [4]: # CELL 4 – Match each predicted track to ONE Locked ground-truth identity, not
         per-frame nearest

MATCH_DIST_THRESHOLD = 3.0    # metres – max distance to lock a track to a GT i
                                nstance at its first frame
GOOD_ERROR_THRESHOLD = 1.0    # seconds – |pred_ttc - gt_ttc| below this counts
                                as "Good"

def match_predictions_to_ground_truth(pred_df):
    pairs = []

    for fid, track in pred_df.groupby("fused_id"):
        track = track.sort_values("sample_id").reset_index(drop=True)
        locked_instance = None

        for _, row in track.iterrows():
            sample_id = row["sample_id"]
            pred_pos = np.array([row["x"], row["y"]])

            if locked_instance is None:
                # First frame only – find and LOCK the nearest GT instance
                gt_entries = [(inst, v) for (s, inst), v in gt_ttc_lookup.items()
                               if s == sample_id]
                if not gt_entries:
                    continue
                gt_positions = np.array([v["pos"] for _, v in gt_entries])
                dists = np.linalg.norm(gt_positions - pred_pos, axis=1)
                min_idx = np.argmin(dists)
                if dists[min_idx] < MATCH_DIST_THRESHOLD:
                    locked_instance = gt_entries[min_idx][0]
                else:
                    continue # no GT close enough at first frame – this track
                                stays unmatched

            # Every frame (including the locking one) uses the SAME Locked identity
            key = (sample_id, locked_instance)
            if key in gt_ttc_lookup:
                pairs.append({
                    "fused_id": fid, "sample_id": sample_id,
                    "instance_token": locked_instance,
                    "pred_ttc": row["ttc"], "gt_ttc": gt_ttc_lookup[key]["ttc"]
                })
            # else: Locked GT has no annotation this frame – skip, don't fall
            back to nearest

        return pd.DataFrame(pairs)

print("match_predictions_to_ground_truth() defined – identity locked once per
track.")

```

match_predictions_to_ground_truth() defined – identity locked once per track.

In [5]: *# CELL 5 - Compute the full evaluation table for all 4 datasets*

```
def compute_evaluation_metrics(sensor_name, pred_df):
    valid_pred = pred_df[np.isfinite(pred_df["ttc"])]
    percent_valid = len(valid_pred) / len(pred_df) * 100 if len(pred_df) > 0 else 0.0

    pairs_df = match_predictions_to_ground_truth(pred_df)
    n_tracks_total = pred_df["fused_id"].nunique()
    n_tracks_matched = pairs_df["fused_id"].nunique() if len(pairs_df) > 0 else 0

    # FIXED: match_predictions_to_ground_truth() retries the lock attempt on
    # EVERY frame until one succeeds (or the track ends), not only the first
    # frame -- the old message said "at their first frame", which overstated
    # how strict the matching is.
    print(f" {sensor_name}: {n_tracks_matched}/{n_tracks_total} tracks found  
a GT lock (checked every frame of the track until one succeeded)")
    valid_pairs = pairs_df[np.isfinite(pairs_df["pred_ttc"]) & np.isfinite(pairs_df["gt_ttc"])] if len(pairs_df) > 0 else pairs_df

    if len(valid_pairs) > 0:
        errors = (valid_pairs["pred_ttc"] - valid_pairs["gt_ttc"]).abs()
        mae = errors.mean()
        rmse = np.sqrt((errors ** 2).mean())
        good_pct = (errors < GOOD_ERROR_THRESHOLD).mean() * 100
    else:
        mae = rmse = good_pct = np.nan

    stability = pred_df.groupby("fused_id")["ttc"].apply(
        lambda s: s[np.isfinite(s)].std() if np.isfinite(s).sum() > 1 else np.nan
    ).mean()

    return {
        "Sensor": sensor_name,
        "Good_pct": round(good_pct, 2) if pd.notna(good_pct) else None,
        "Bad_pct": round(100 - good_pct, 2) if pd.notna(good_pct) else None,
        "Mean_TTC": round(valid_pred["ttc"].mean(), 2) if len(valid_pred) > 0 else None,
        "Median_TTC": round(valid_pred["ttc"].median(), 2) if len(valid_pred) > 0 else None,
        "Percent_Valid": round(percent_valid, 2),
        "Stability": round(stability, 2) if pd.notna(stability) else None,
        "MAE": round(mae, 2) if pd.notna(mae) else None,
        "RMSE": round(rmse, 2) if pd.notna(rmse) else None,
        # FIXED: these were computed and printed above but never made it into
        # the returned dict, so they never reached evaluation_metrics.csv --
        # "how many of this sensor's tracks were even comparable to ground
        # truth at all" was missing from the final table.
        "N_Tracks_Total": int(n_tracks_total),
        "N_Tracks_GT_Matched": int(n_tracks_matched),
        "n_matched_pairs": len(valid_pairs)
    }

all_metrics = []
for name in ["lidar", "radar", "camera", "camera_mono", "fused"]:
```

```

pred_df = pd.read_csv(STEP5_DIR / f"ttc_{name}.csv")
print(f"Evaluating {name}...")
all_metrics.append(compute_evaluation_metrics(name, pred_df))

metrics_df = pd.DataFrame(all_metrics)

# Sanity check: two sensors reporting a suspiciously identical n_matched_pairs
# usually means a copy-paste bug (e.g. accidentally reusing one sensor's
# pred_df for another) rather than a real coincidence -- flag it instead of
# silently trusting the numbers.
dup_counts = metrics_df["n_matched_pairs"][metrics_df["n_matched_pairs"] > 0]
dup_counts = dup_counts[dup_counts.duplicated(keep=False)]
if len(dup_counts) > 0:
    dup_sensors = metrics_df.loc[dup_counts.index, "Sensor"].tolist()
    print(f"\u26a0\u203c WARNING: sensors {dup_sensors} report the exact same
n_matched_pairs "
f"({sorted(dup_counts.unique())}) -- verify this isn't a copy-paste
bug before trusting it.")

metrics_path = STEP6_DIR / "evaluation_metrics.csv"
metrics_df.to_csv(metrics_path, index=False)

print(f"\nSaved: {metrics_path}")
display(metrics_df)

```

Evaluating lidar...

lidar: 922/2327 tracks found a GT lock (checked every frame of the track until one succeeded)

Evaluating radar...

radar: 1775/7283 tracks found a GT lock (checked every frame of the track until one succeeded)

Evaluating camera...

camera: 1030/1131 tracks found a GT lock (checked every frame of the track until one succeeded)

Evaluating camera_mono...

camera_mono: 790/953 tracks found a GT lock (checked every frame of the track until one succeeded)

Evaluating fused...

fused: 1584/6175 tracks found a GT lock (checked every frame of the track until one succeeded)

Saved: F:\Sensor fusion Research\output\step_6\evaluation_metrics.csv

	Sensor	Good_pct	Bad_pct	Mean_TTC	Median_TTC	Percent_Valid	Stability	MAE	RMSE
0	lidar	47.98	52.02	10.74	5.72	41.32	4.20	3.77	7.1
1	radar	59.48	40.52	10.38	6.80	39.10	2.36	2.77	6.1
2	camera	52.82	47.18	8.53	5.27	52.51	2.71	2.87	5.1
3	camera_mono	41.43	58.57	8.13	4.91	54.90	3.02	3.83	7.1
4	fused	53.30	46.70	10.97	7.06	38.88	3.54	3.55	7.1



In [6]: *# CELL 6 – Comparison charts (matches dissertation Figure 19 layout)*

```
fig, axes = plt.subplots(2, 2, figsize=(13, 9))

axes[0, 0].bar(metrics_df["Sensor"], metrics_df["Mean_TTC"], label="Mean", alp
ha=0.8)
axes[0, 0].bar(metrics_df["Sensor"], metrics_df["Median_TTC"], label="Median",
alpha=0.6, width=0.5)
axes[0, 0].set_title("Mean and Median TTC Comparison")
axes[0, 0].set_ylabel("TTC (s)"); axes[0, 0].legend()

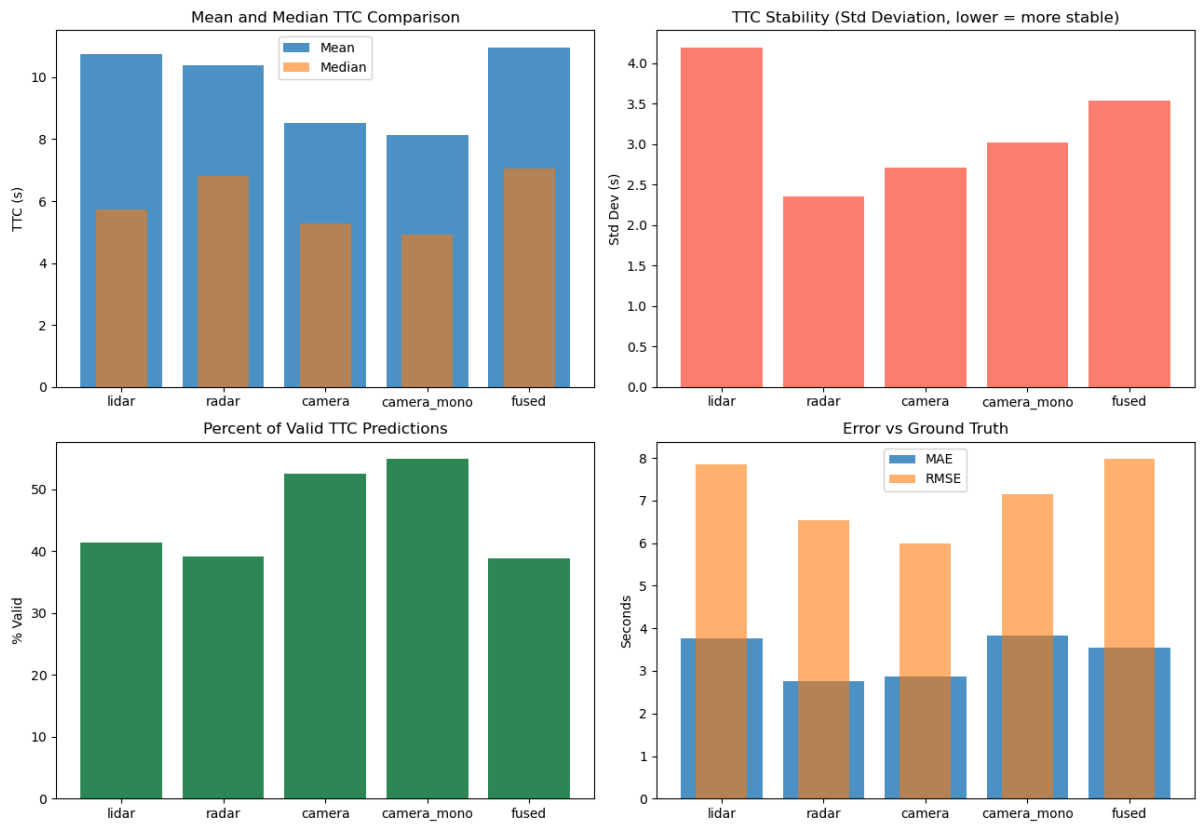
axes[0, 1].bar(metrics_df["Sensor"], metrics_df["Stability"], color="salmon")
axes[0, 1].set_title("TTC Stability (Std Deviation, lower = more stable)")
axes[0, 1].set_ylabel("Std Dev (s)")

axes[1, 0].bar(metrics_df["Sensor"], metrics_df["Percent_Valid"], color="seagr
een")
axes[1, 0].set_title("Percent of Valid TTC Predictions")
axes[1, 0].set_ylabel("% Valid")

axes[1, 1].bar(metrics_df["Sensor"], metrics_df["MAE"], label="MAE", alpha=0.
8)
axes[1, 1].bar(metrics_df["Sensor"], metrics_df["RMSE"], label="RMSE", alpha=
0.6, width=0.5)
axes[1, 1].set_title("Error vs Ground Truth")
axes[1, 1].set_ylabel("Seconds"); axes[1, 1].legend()

plt.tight_layout()
chart_path = STEP6_DIR / "evaluation_charts.png"
plt.savefig(chart_path, dpi=150, bbox_inches="tight")
plt.show()

print(f"Saved: {chart_path}")
```



Saved: F:\Sensor fusion Research\output\step_6\evaluation_charts.png

```

In [7]: # CELL 7 (NEW) – Diagnostic: does fusion's error come from bad within-frame me
        rges?
        # Compares MAE for fused points that combined 2+ sensors vs points passed thro
        ugh from just 1 sensor

        from config import STEP4_DIR

        # Reload Step 4's raw fused output – it still has the "sensors" column
        # (Step 5's CSV dropped it when loading, so we go back to the source)
        raw_fused_df = pd.read_csv(STEP4_DIR / "fused_tracks_all.csv")[["fused_id", "s
        ample_id", "sensors"]]

        # Get GT-matched pairs for the fused dataset specifically
        fused_pred_df = pd.read_csv(STEP5_DIR / "ttc_fused.csv")
        fused_pairs_df = match_predictions_to_ground_truth(fused_pred_df)

        # Join sensor contribution info onto the matched pairs
        fused_pairs_df = fused_pairs_df.merge(raw_fused_df, on=["fused_id", "sample_i
        d"], how="left")

        # Split into multi-sensor-merged vs single-sensor-passthrough
        is_merged = fused_pairs_df["sensors"].str.contains(r"\+", na=False)
        merged_group = fused_pairs_df[is_merged]
        single_group = fused_pairs_df[~is_merged]

        def group_error_stats(name, group):
            valid = group[np.isfinite(group["pred_ttc"]) & np.isfinite(group["gt_tt
            c"])]
            if len(valid) == 0:
                print(f"{name}: no valid pairs")
                return
            errors = (valid["pred_ttc"] - valid["gt_ttc"]).abs()
            mae = errors.mean()
            rmse = np.sqrt((errors ** 2).mean())
            print(f"{name}: n={len(valid)}, MAE={mae:.2f}, RMSE={rmse:.2f}")

        print("Fusion error breakdown by merge status:\n")
        group_error_stats("Multi-sensor merged (2+ sensors combined)", merged_group)
        group_error_stats("Single-sensor passthrough (1 sensor only)", single_group)

        print(f"\nTotal matched fused pairs: {len(fused_pairs_df)}")
        print(f"  Multi-sensor merged      : {len(merged_group)} ({len(merged_group)/l
        en(fused_pairs_df)*100:.1f}%)")
        print(f"  Single-sensor passthrough: {len(single_group)} ({len(single_group)/l
        en(fused_pairs_df)*100:.1f}%)")

```

Fusion error breakdown by merge status:

Multi-sensor merged (2+ sensors combined): n=1106, MAE=2.33, RMSE=5.83
 Single-sensor passthrough (1 sensor only): n=2496, MAE=4.09, RMSE=8.76

Total matched fused pairs: 10778
 Multi-sensor merged : 2584 (24.0%)
 Single-sensor passthrough: 8194 (76.0%)

In []:

Step 7 — Full Pipeline Integrity Audit

Walks Steps 0 through 6, checking **input count vs output count** at every stage (catching silent drops) and **known-bug regression checks** — specifically re-testing for the exact failure patterns we already found and fixed earlier in this project, so a future change can't silently reintroduce them.

Every check prints  PASS,  WARN, or  FAIL. Final cell gives one summary count.

This does not change any pipeline output — read-only audit.

```
In [1]: from pathlib import Path
import json
import numpy as np
import pandas as pd

if not Path("config.py").exists():
    raise FileNotFoundError("config.py not found.")

from config import DATA_ROOT, STEP0_DIR, STEP1_DIR, STEP2_DIR, STEP3_DIR, STEP
4_DIR, STEP5_DIR, OUTPUT_ROOT
STEP6_DIR = OUTPUT_ROOT / "step_6"

audit_log = [] # (step, check_name, status, detail)

def check(step, name, condition, detail=""):
    status = "PASS" if condition else "FAIL"
    icon = "\u2705" if condition else "\u274c"
    print(f"{icon} [{step}] {name}" + (f" - {detail}" if detail else ""))
    audit_log.append((step, name, status, detail))

def warn(step, name, condition, detail=""):
    status = "OK" if condition else "WARN"
    icon = "\u2705" if condition else "\u26a0\u200b"
    print(f"{icon} [{step}] {name}" + (f" - {detail}" if detail else ""))
    audit_log.append((step, name, status, detail))

print("Audit initialized.\n")
```

```
config.py loaded. PROJECT_ROOT = F:\Sensor fusion Research
DATA_ROOT   = F:\Sensor fusion Research\DATA SET\archive
OUTPUT_ROOT = F:\Sensor fusion Research\output
Audit initialized.
```

```

In [2]: # — STEP 0 audit —————
print("=== STEP 0 – Dataset Preparation ===")

with open(STEP0_DIR / "samples_index.json") as f:
    samples_index = json.load(f)

n_samples = len(samples_index)
check("Step0", "Sample count == 404 (nuScenes-mini)", n_samples == 404, f"got {n_samples}")

missing_sensor_meta = [sid for sid, info in samples_index.items()
                        if not (STEP0_DIR / info["folder"] / "sensor_meta.json").exists()]
check("Step0", "Every sample has sensor_meta.json", len(missing_sensor_meta) == 0,
      f"{len(missing_sensor_meta)} missing")

# Regression check: "folder" must be RELATIVE, not an absolute leaked path
sample_folder_values = [info["folder"] for info in samples_index.values()]
has_absolute_path = any((":\\" in f or f.startswith("/")) for f in sample_folder_values)
check("Step0", "REGRESSION: 'folder' field is relative, not an absolute path",
      not has_absolute_path, "found absolute path in index!" if has_absolute_path else "clean")

sensor_counts = []
for sid, info in list(samples_index.items())[:20]:
    with open(STEP0_DIR / info["folder"] / "sensor_meta.json") as f:
        meta = json.load(f)
    sensor_counts.append(len(meta["sensor_channels"]))
check("Step0", "All 12 sensor channels present (sampled 20 files)",
      all(c == 12 for c in sensor_counts), f"counts seen: {set(sensor_counts)}")
print()

=== STEP 0 – Dataset Preparation ===
✓ [Step0] Sample count == 404 (nuScenes-mini) – got 404
✓ [Step0] Every sample has sensor_meta.json – 0 missing
✓ [Step0] REGRESSION: 'folder' field is relative, not an absolute path – clean
✓ [Step0] All 12 sensor channels present (sampled 20 files) – counts seen: {12}

```

```

In [3]: # — STEP 1.1/1.2/1.3 audit —————
print("=== STEP 1.1 – Camera Preprocessing ===")
with open(STEP1_DIR / "camera_meta.json") as f:
    camera_meta = json.load(f)
check("Step1.1", "Camera samples count matches Step 0", len(camera_meta) == n_
samples,
      f"{len(camera_meta)} / {n_samples}")
cam_channel_counts = [len(v) for v in camera_meta.values()]
check("Step1.1", "All samples have 6 camera channels", all(c == 6 for c in cam
_channel_counts),
      f"counts seen: {set(cam_channel_counts)}")
has_calib = all("calibrated_sensor" in list(v.values())[0] for v in camera_met
a.values() if v)
check("Step1.1", "Calibration present per camera entry", has_calib)

print("\n=== STEP 1.2 – Radar Parsing ===")
radar_dir = STEP1_DIR / "radar"
radar_sample_folders = [p for p in radar_dir.iterdir() if p.is_dir()]
check("Step1.2", "Radar sample folders count matches Step 0", len(radar_sample
_folders) == n_samples,
      f"{len(radar_sample_folders)} / {n_samples}")
sample_radar_file = next(radar_sample_folders[0].glob("RADAR_*.json"))
with open(sample_radar_file) as f:
    radar_sample_data = json.load(f)
check("Step1.2", "REGRESSION: radar file has dict structure with 'points' + 'c
alibration'",
      "points" in radar_sample_data and "calibration" in radar_sample_data)

print("\n=== STEP 1.3 – LiDAR Parsing ===")
lidar_dir = STEP1_DIR / "lidar"
lidar_sample_folders = [p for p in lidar_dir.iterdir() if p.is_dir()]
check("Step1.3", "LiDAR sample folders count matches Step 0", len(lidar_sample
_folders) == n_samples,
      f"{len(lidar_sample_folders)} / {n_samples}")
sample_lidar = np.load(lidar_sample_folders[0] / "lidar_raw.npy")
check("Step1.3", "lidar_raw.npy has 7 columns (x,y,z,intensity,distance,azimut
h,elevation)",
      sample_lidar.shape[1] == 7, f"got shape {sample_lidar.shape}")
with open(lidar_sample_folders[0] / "lidar_meta.json") as f:
    lidar_meta_sample = json.load(f)
check("Step1.3", "lidar_meta.json has ego_pose + calibration",
      "ego_pose" in lidar_meta_sample and "sensor_to_ego_translation" in lidar
_meta_sample)
print()

```

=== STEP 1.1 – Camera Preprocessing ===

- ✓ [Step1.1] Camera samples count matches Step 0 – 404 / 404
- ✓ [Step1.1] All samples have 6 camera channels – counts seen: {6}
- ✓ [Step1.1] Calibration present per camera entry

=== STEP 1.2 – Radar Parsing ===

- ✓ [Step1.2] Radar sample folders count matches Step 0 – 404 / 404
- ✓ [Step1.2] REGRESSION: radar file has dict structure with 'points' + 'calibration'

=== STEP 1.3 – LiDAR Parsing ===

- ✓ [Step1.3] LiDAR sample folders count matches Step 0 – 404 / 404
- ✓ [Step1.3] lidar_raw.npy has 7 columns (x,y,z,intensity,distance,azimuth,elevation) – got shape (34688, 7)
- ✓ [Step1.3] lidar_meta.json has ego_pose + calibration

```

In [4]: # — STEP 2.1/2.2/2.3/2.3.1 audit —————
print("=== STEP 2.1 – LiDAR Ground Removal + Clustering ===")
lidar2_dir = STEP2_DIR / "lidar"
sample_lidar2 = next(p for p in lidar2_dir.iterdir() if p.is_dir())

has_ground = (sample_lidar2 / "lidar_ground.npy").exists()
has_nonground = (sample_lidar2 / "lidar_nonground.npy").exists()
check("Step2.1", "REGRESSION: lidar_ground.npy exists (this was the original m
issing-file bug)",
      has_ground)
check("Step2.1", "lidar_nonground.npy exists", has_nonground)

if has_nonground and (sample_lidar2 / "lidar_cluster_labels.npy").exists():
    nonground = np.load(sample_lidar2 / "lidar_nonground.npy")
    labels = np.load(sample_lidar2 / "lidar_cluster_labels.npy")
    check("Step2.1", "REGRESSION: cluster_labels length matches nonground poin
ts (exact membership, not crude match)",
          len(labels) == len(nonground), f"{len(labels)} labels vs {len(nongro
und)} points")

print("\n=== STEP 2.2 – Radar Filtering ===")
radar2_dir = STEP2_DIR / "radar"
sample_radar2_folder = next(p for p in radar2_dir.iterdir() if p.is_dir())
sample_radar2_file = next(sample_radar2_folder.glob("RADAR_*.json"))
with open(sample_radar2_file) as f:
    radar2_data = json.load(f)
check("Step2.2", "REGRESSION: filtered radar output preserves dict structure
('points' + 'calibration')",
      "points" in radar2_data and "calibration" in radar2_data)
if radar2_data["points"]:
    has_both_vel = "vx" in radar2_data["points"][0] and "vx_comp" in radar2_da
ta["points"][0]
    check("Step2.2", "REGRESSION: both raw (vx) and compensated (vx_comp) velo
city present", has_both_vel)

print("\n=== STEP 2.3 / 2.3.1 – YOLO Detection + Global 3D Projection ===")
yolo_global_dir = STEP2_DIR / "yolo_global"
sample_yg_folder = next(p for p in yolo_global_dir.iterdir() if p.is_dir())
sample_yg_file = next(sample_yg_folder.glob("CAM_*.json"), None)
if sample_yg_file:
    with open(sample_yg_file) as f:
        yg_data = json.load(f)
    if yg_data:
        has_3d_field = "has_3d_position" in yg_data[0]
        check("Step2.3.1", "has_3d_position field present on detections", has_
3d_field)
print()

```

=== STEP 2.1 – LiDAR Ground Removal + Clustering ===

- ✓ [Step2.1] REGRESSION: lidar_ground.npy exists (this was the original missing-file bug)
- ✓ [Step2.1] lidar_nonground.npy exists
- ✓ [Step2.1] REGRESSION: cluster_labels length matches nonground points (exact membership, not crude match) – 20206 labels vs 20206 points

=== STEP 2.2 – Radar Filtering ===

- ✓ [Step2.2] REGRESSION: filtered radar output preserves dict structure ('points' + 'calibration')
- ✓ [Step2.2] REGRESSION: both raw (vx) and compensated (vx_comp) velocity present

=== STEP 2.3 / 2.3.1 – YOLO Detection + Global 3D Projection ===

- ✓ [Step2.3.1] has_3d_position field present on detections

```

In [5]: # — STEP 3.1/3.2/3.3 audit — the most important regression checks —
print("=== STEP 3.1/3.2/3.3 — Tracking ===")

def audit_tuple_tracks(track_dir, sensor_name):
    files = list(track_dir.glob("track_*.json"))
    duplicate_sample_tracks = 0
    length_2_count = 0
    total = len(files)
    for f in files:
        with open(f) as fh:
            points = json.load(fh)
            sample_ids_in_track = [p[0] for p in points]
            if len(sample_ids_in_track) != len(set(sample_ids_in_track)):
                duplicate_sample_tracks += 1 # REGRESSION signal: double-assignment bug
            if len(points) == 2:
                length_2_count += 1
    return total, duplicate_sample_tracks, length_2_count

for sensor_name, track_dir in [("lidar", STEP3_DIR / "lidar"), ("radar", STEP3_DIR / "radar")]:
    total, dupes, len2 = audit_tuple_tracks(track_dir, sensor_name)
    check(f"Step3.{sensor_name}", f"REGRESSION: no track has the same sample_id twice (double-assignment bug)",
          dupes == 0, f"{dupes}/{total} tracks affected")
    warn(f"Step3.{sensor_name}", "Fraction of minimum-length (2-frame) tracks not dominant",
         len2 / total < 0.5 if total > 0 else True, f"{len2}/{total} ({len2/total*100:.0f}%)") if total > 0 else "n/a")

# Camera uses a different (dict/trajectory) format
cam_track_dir = STEP3_DIR / "camera"
cam_files = list(cam_track_dir.glob("track_*.json"))
cam_dupes = 0
for f in cam_files:
    with open(f) as fh:
        data = json.load(fh)
        sids = [pt["sample_id"] for pt in data["trajectory"]]
        if len(sids) != len(set(sids)):
            cam_dupes += 1
check("Step3.camera", "REGRESSION: no camera track has the same sample_id twice", cam_dupes == 0,
      f"{cam_dupes}/{len(cam_files)} tracks affected")

cross_cam_tracks = sum(
    1 for f in cam_files
    if len({pt["camera"] for pt in json.load(open(f))["trajectory"]}) > 1
)
check("Step3.camera", "Unified tracker produces some cross-camera tracks (design goal)",
      cross_cam_tracks > 0, f"{cross_cam_tracks} tracks span 2+ cameras")
print()

```

=== STEP 3.1/3.2/3.3 – Tracking ===

- ✓ [Step3.lidar] REGRESSION: no track has the same sample_id twice (double-as signment bug) – 0/2327 tracks affected
- ✓ [Step3.lidar] Fraction of minimum-length (2-frame) tracks not dominant – 5 10/2327 (22%)
- ✓ [Step3.radar] REGRESSION: no track has the same sample_id twice (double-as signment bug) – 0/7283 tracks affected
- ✓ [Step3.radar] Fraction of minimum-length (2-frame) tracks not dominant – 2 331/7283 (32%)
- ✓ [Step3.camera] REGRESSION: no camera track has the same sample_id twice – 0/1131 tracks affected
- ✓ [Step3.camera] Unified tracker produces some cross-camera tracks (design goal) – 522 tracks span 2+ cameras

```
In [6]: # — STEP 4 audit —————
print("=== STEP 4 – Fusion ===")

fused_df = pd.read_csv(STEP4_DIR / "fused_tracks_all.csv")

# FIXED: stale set, predating the camera_mono swap (CLAUDE.md "Second audit
# round" -- Step 4's CAMERA_TRACKS_DIR now points at camera_mono, the
# genuinely independent monocular baseline, not the LiDAR-assisted
# "camera" baseline, which deliberately never appears in fused output
# anymore). Was flagging that correct, intentional state as a failure.
valid_sensor_tokens = {"lidar", "radar", "camera_mono"}
all_tokens = set()
for s in fused_df["sensors"].unique():
    all_tokens.update(s.split("+"))
check("Step4", "'sensors' column only contains known sensor names",
      all_tokens.issubset(valid_sensor_tokens), f"found: {all_tokens}")

dupe_pairs = fused_df.duplicated(subset=["fused_id", "sample_id"]).sum()
check("Step4", "REGRESSION: no (fused_id, sample_id) pair appears twice (fusio
n-level double-assignment)",
      dupe_pairs == 0, f"{dupe_pairs} duplicates found")

multi_sensor_pct = fused_df["sensors"].str.contains(r"\+").mean() * 100
print(f" Info: {multi_sensor_pct:.1f}% of fused points combine 2+ sensors "
      f"(context for the merge-rate discussion – current INTRA_FRAME_MERGE_THR
ESH)")
print()
```

=== STEP 4 – Fusion ===

- ✓ [Step4] 'sensors' column only contains known sensor names – found: {'lidar', 'radar', 'camera_mono'}
- ✓ [Step4] REGRESSION: no (fused_id, sample_id) pair appears twice (fusion-level double-assignment) – 0 duplicates found
- Info: 10.5% of fused points combine 2+ sensors (context for the merge-rate discussion – current INTRA_FRAME_MERGE_THRESHOLD)

```
In [7]: # — STEP 5 audit —————
print("=== STEP 5 – TTC ===")

for name in ["lidar", "radar", "camera", "camera_mono", "fused"]:
    ttc_df = pd.read_csv(STEP5_DIR / f"ttc_{name}.csv")
    check(f"Step5.{name}", "REGRESSION: timestamp column present", "timestamp"
in ttc_df.columns)

    finite_ttc = ttc_df[np.isfinite(ttc_df["ttc"])]["ttc"]
    if len(finite_ttc) > 0:
        check(f"Step5.{name}", "REGRESSION: no TTC value exceeds MAX_VALID_TTC
cap (60s) – outlier bug check",
            finite_ttc.max() <= 60.01, f"max seen: {finite_ttc.max():.1f}s")
        warn(f"Step5.{name}", "Mean and median TTC are reasonably close (no ex
treme skew)",
            abs(finite_ttc.mean() - finite_ttc.median()) < 5,
            f"mean={finite_ttc.mean():.2f}, median={finite_ttc.median():.2
f}")
    print()
```

```
=== STEP 5 – TTC ===
```

```
✓ [Step5.lidar] REGRESSION: timestamp column present
✓ [Step5.lidar] REGRESSION: no TTC value exceeds MAX_VALID_TTC cap (60s) – o
utlier bug check – max seen: 60.0s
⚠ [Step5.lidar] Mean and median TTC are reasonably close (no extreme skew) –
mean=10.74, median=5.72
✓ [Step5.radar] REGRESSION: timestamp column present
✓ [Step5.radar] REGRESSION: no TTC value exceeds MAX_VALID_TTC cap (60s) – o
utlier bug check – max seen: 59.9s
✓ [Step5.radar] Mean and median TTC are reasonably close (no extreme skew) –
mean=10.38, median=6.80
✓ [Step5.camera] REGRESSION: timestamp column present
✓ [Step5.camera] REGRESSION: no TTC value exceeds MAX_VALID_TTC cap (60s) –
outlier bug check – max seen: 59.9s
✓ [Step5.camera] Mean and median TTC are reasonably close (no extreme skew)
– mean=8.53, median=5.27
✓ [Step5.camera_mono] REGRESSION: timestamp column present
✓ [Step5.camera_mono] REGRESSION: no TTC value exceeds MAX_VALID_TTC cap (60
s) – outlier bug check – max seen: 59.4s
✓ [Step5.camera_mono] Mean and median TTC are reasonably close (no extreme s
kew) – mean=8.13, median=4.91

✓ [Step5.fused] REGRESSION: timestamp column present
✓ [Step5.fused] REGRESSION: no TTC value exceeds MAX_VALID_TTC cap (60s) – o
utlier bug check – max seen: 59.9s
✓ [Step5.fused] Mean and median TTC are reasonably close (no extreme skew) –
mean=10.97, median=7.06
```

```
In [8]: # — STEP 6 audit —————
print("=== STEP 6 – Evaluation Metrics ===")

metrics_path = STEP6_DIR / "evaluation_metrics.csv"
if metrics_path.exists():
    metrics_df = pd.read_csv(metrics_path)
    check("Step6", "All 5 sensors present in evaluation table",
          set(metrics_df["Sensor"]) == {"lidar", "radar", "camera", "camera_mono", "fused"})
    check("Step6", "REGRESSION: no MAE/RMSE in the tens-of-thousands (near-zero closing-speed bug)",
          metrics_df["MAE"].max() < 100, f"max MAE: {metrics_df['MAE'].max():.1f}")
    warn("Step6", "n_matched_pairs is a reasonable fraction of total tracks (locked-identity fix working)",
          (metrics_df["n_matched_pairs"] > 0).all())
else:
    print(" Step 6 evaluation_metrics.csv not found – run Step 6 first.")
print()

=== STEP 6 – Evaluation Metrics ===
✅ [Step6] All 5 sensors present in evaluation table
✅ [Step6] REGRESSION: no MAE/RMSE in the tens-of-thousands (near-zero closing-speed bug) – max MAE: 3.8
✅ [Step6] n_matched_pairs is a reasonable fraction of total tracks (locked-identity fix working)
```

```
In [9]: # — FINAL SUMMARY —————
audit_df = pd.DataFrame(audit_log, columns=["step", "check", "status", "detail"])

n_pass = (audit_df["status"] == "PASS").sum()
n_fail = (audit_df["status"] == "FAIL").sum()
n_ok = (audit_df["status"] == "OK").sum()
n_warn = (audit_df["status"] == "WARN").sum()

print("=" * 55)
print(f"AUDIT SUMMARY: {n_pass} PASS, {n_fail} FAIL, {n_ok} OK, {n_warn} WARN")
print("=" * 55)

if n_fail > 0:
    print("\n\u274c FAILED CHECKS (fix these first):")
    display(audit_df[audit_df["status"] == "FAIL"])

if n_warn > 0:
    print("\n\u26a0\u200c WARNINGS (worth a look, not necessarily broken):")
    display(audit_df[audit_df["status"] == "WARN"])

audit_path = OUTPUT_ROOT / "step_7_audit_report.csv"
audit_df.to_csv(audit_path, index=False)
print(f"\nFull report saved: {audit_path}")
```

```
=====
AUDIT SUMMARY: 36 PASS, 0 FAIL, 7 OK, 1 WARN
=====
```

⚠ WARNINGS (worth a look, not necessarily broken):

	step	check	status	detail
28	Step5.lidar	Mean and median TTC are reasonably close (no e...	WARN	mean=10.74, median=5.72

Full report saved: F:\Sensor fusion Research\output\step_7_audit_report.csv